01418331 Theory of Computation

Theory of Computation ฉบับรวมเล่ม

จาก Finite Automata ถึง Undecidability

DFA · NFA · Regular Expression · Pumping Lemma
CFG · PDA · Turing Machine · Decidability

สอนละเอียดทีละหัวข้อ เขียนเป็น bullet อ่านง่าย
มีตัวอย่างพร้อม diagram กล่องสูตร กล่อง “Claude สอนเพิ่ม” และกล่องจุดที่ผิดบ่อย

ภาค 1 (บทที่ 0–8) : Regular languages เรียงตาม Sipser บทที่ 0–1
ภาค 2 (บทที่ 9–18) : Context-free languages และ Computability เรียงตาม Sipser บทที่ 2–4
บทที่ 19 : รวมสูตรทั้งเล่ม
ภาค 3 (บทที่ S0–S16) : สรุปตามหนังสือ Sipser เรียงตามรายการ Suggested reading เลข Definition / Theorem / Example ตรงกับหนังสือ
บทที่ 8 (Minimization, Myhill–Nerode) เป็นเนื้อหาเสริมนอก Sipser บทที่ 1
เฉลยการบ้านอยู่ที่ `Homework5_solution.pdf`

สารบัญ

ภาค 1 : Regular Languages

0 พื้นฐาน : เซต สตริง และภาษา **1**

0.1 เซต . . . 1
0.2 Alphabet และ String . . . 1
0.3 Language . . . 2
0.4 Operation บนภาษา . . . 2
0.5 เทคนิคการพิสูจน์ที่ใช้ทั้งวิชา . . . 2

1 Deterministic Finite Automata (DFA) **3**

1.1 แนวคิด . . . 3
1.2 นิยาม . . . 3
1.3 การอ่าน state diagram . . . 3
1.4 การคำนวณอย่างเป็นทางการ . . . 4
1.5 วิธีออกแบบ DFA . . . 4
1.6 รูปแบบที่เจอบ่อย . . . 5

2 Nondeterministic Finite Automata (NFA) **6**

2.1 NFA ต่างจาก DFA อย่างไร . . . 6
2.2 นิยาม . . . 6
2.3 วิธีคิดเรื่อง nondeterminism . . . 6

3 แปลง NFA เป็น DFA (Subset Construction) **8**

3.1 ทฤษฎีบท . . . 8
3.2 แนวคิด . . . 8
3.3 สูตร . . . 8
3.4 ขั้นตอนทำด้วยตาราง . . . 8

4 Closure ของ Regular Languages **11**

4.1 ความหมายของ “ปิด” . . . 11
4.2 Product construction (ใช้ DFA) . . . 11
4.3 Complement . . . 12
4.4 Union, Concatenation, Star (ใช้ NFA) . . . 12
4.5 สรุป closure และการนำไปใช้ . . . 12

5 Regular Expression **14**

5.1 นิยาม . . . 14
5.2 ตัวอย่าง ($\Sigma = \{0, 1\}$) . . . 15
5.3 Identity ที่ใช้ลดรูป . . . 15

6 Regular Expression ⇔ Finite Automata **16**
6.1 ทฤษฎีบท . . . 16
6.2 Regular expression → NFA . . . 16
6.3 DFA → Regular expression ด้วย GNFA . . . 17

7 Pumping Lemma สำหรับ Regular Languages **19**
7.1 ตัวบท . . . 19
7.2 ทำไมจึงจริง . . . 19
7.3 วิธีใช้ : มองเป็นเกม . . . 19
7.4 ตัวอย่าง . . . 20

8 เสริม : DFA Minimization และ Myhill–Nerode **22**
8.1 State ที่แยกกันได้ . . . 22
8.2 Table-filling algorithm . . . 22
8.3 Myhill–Nerode theorem . . . 23

ภาค 2 : Context-Free Languages และ Computability

9 ภาพรวม : ลำดับชั้นของภาษา **24**
9.1 ลำดับชั้น . . . 24
9.2 ความจำของแต่ละเครื่อง . . . 24

10 Context-Free Grammar (CFG) **25**
10.1 นิยาม . . . 25
10.2 คำศัพท์ . . . 25
10.3 สูตรสำเร็จในการออกแบบ CFG . . . 25
10.4 เทคนิค : แตกภาษาเป็นชิ้น . . . 26
10.5 วิธีพิสูจน์ว่า CFG ถูกต้อง . . . 26
10.6 เทคนิค : แปลง DFA เป็น CFG . . . 26
10.7 Ambiguity . . . 26
10.8 Chomsky normal form . . . 26

11 Pushdown Automata (PDA) **28**
11.1 นิยาม . . . 28
11.2 วิธีอ่านป้ายบนเส้น : $a,\ b \to c$ . . . 28
11.3 เงื่อนไข accept . . . 28
11.4 เทคนิคที่ใช้ประจำ . . . 28
11.5 ตัวอย่าง : PDA สำหรับ $\{0^n 1^n : n \geq 0\}$ . . . 29

12 แปลง CFG → PDA **30**
12.1 แนวคิด . . . 30
12.2 สูตร . . . 30
12.3 การ push หลายตัว . . . 30

12.4 นับจำนวน . . . 31
12.5 ขั้นตอนทำโจทย์ . . . 31

13 แปลง PDA → CFG 32
13.1 ขั้นที่ 0 : ปรับ PDA ให้ตรง 3 เงื่อนไข . . . 32
13.2 ความหมายของตัวแปร . . . 32
13.3 สูตร . . . 32
13.4 วิธีจำกฎแบบที่ 1 . . . 32
13.5 ขั้นตอนทำโจทย์ . . . 33

14 Pumping Lemma สำหรับ Context-Free Language 34
14.1 ตัวบท . . . 34
14.2 เทียบกับ pumping lemma ของ regular . . . 34
14.3 โครงบทพิสูจน์ (ใช้ได้ทุกข้อ) . . . 34
14.4 การแบ่งกรณีมาตรฐาน . . . 34
14.5 เคล็ดลับการเลือก s . . . 35

15 Turing Machine (TM) 36
15.1 นิยาม . . . 36
15.2 สิ่งที่ต่างจาก DFA / PDA . . . 36
15.3 Configuration . . . 36
15.4 ตัวอย่าง : TM ที่ decide $\{0^n1^n : n \geq 0\}$ . . . 37
15.5 เทคนิคออกแบบ TM . . . 37

16 Variants of Turing Machines 38
16.1 Variants of Turing machines . . . 38
16.2 Multitape Turing machines . . . 38
16.3 Nondeterministic Turing machines . . . 38
16.4 Enumerators . . . 39
16.5 Equivalence with other models . . . 39

17 Decidable กับ Recognizable 40
17.1 นิยาม . . . 40
17.2 โครงบทพิสูจน์ closure (ใช้ได้ทุกข้อ) . . . 40
17.3 สูตรการสร้าง M สำหรับภาษา decidable . . . 40
17.4 กรณี recognizable : ต้องระวัง loop . . . 41
17.5 ปัญหา decidable ของ regular และ context-free languages . . . 41

18 Undecidability ของ A_{TM} 43
18.1 ภาษา . . . 43
18.2 ทำไม recognizable . . . 43
18.3 ทำไมไม่ decidable : diagonalization . . . 43

18.4 ภาพของ diagonalization . . . 43
18.5 โครงพิสูจน์ undecidable ด้วย reduction . . . 44
18.6 Countable sets และ diagonalization ของ Cantor . . . 44
18.7 ภาษาที่ไม่ Turing-recognizable . . . 44

สรุป

19 รวมสูตรทั้งเล่ม และแผนที่ตัดสินใจ 45
19.1 ภาค 1 : Regular languages . . . 45
19.2 ภาค 2 : Context-free languages และ Computability . . . 46

ภาค 3 : สรุปตามหนังสือ Sipser (เรียงตาม Suggested reading)

S0 Math background 49
S0.1 Sets . . . 49
S0.2 Sequences and tuples . . . 49
S0.3 Functions and relations . . . 49
S0.4 Graphs . . . 50
S0.5 Strings and languages . . . 50
S0.6 Boolean logic . . . 51
S0.7 Summary of mathematical terms . . . 51
S0.8 §0.3 Definitions, theorems, and proofs . . . 51
S0.9 §0.4 Types of proof . . . 52

S1 Deterministic Finite Automata (DFA) 53
S1.1 Finite automata . . . 53
S1.2 Formal definition of a finite automaton . . . 53
S1.3 Examples of finite automata . . . 54

S2 Regular Languages 55
S2.1 Formal definition of computation . . . 55
S2.2 Designing finite automata . . . 55
S2.3 The regular operations . . . 56

S3 Non-deterministic Finite Automata (NFA) 57
S3.1 Nondeterminism . . . 57
S3.2 Formal definition of a nondeterministic finite automaton . . . 58

S4 Equivalence of DFA and NFA 59
S4.1 Equivalence of NFAs and DFAs . . . 59

S5 Closure Properties of Regular Languages 60
S5.1 Closure under the regular operations . . . 60

S6 Regular Expressions **61**
S6.1 Regular expressions . . . 61
S6.2 Formal definition of a regular expression . . . 61

S7 Equivalence of RE and DFA/NFA **63**
S7.1 Equivalence with finite automata . . . 63

S8 Proving Non-regularity **65**
S8.1 Nonregular languages . . . 65
S8.2 The pumping lemma for regular languages . . . 65

S9 Context-Free Grammar (CFG) **67**
S9.1 Context-free grammars . . . 67
S9.2 Formal definition of a context-free grammar . . . 67
S9.3 Examples of context-free grammars . . . 68
S9.4 Designing context-free grammars . . . 68

S10 Context-Free Languages (CFL) **69**
S10.1 Ambiguity . . . 69
S10.2 Chomsky normal form . . . 69

S11 Pushdown Automata (PDA) **71**
S11.1 Pushdown automata . . . 71
S11.2 Formal definition of a pushdown automaton . . . 71
S11.3 Examples of pushdown automata . . . 72

S12 Equivalence of PDA and CFG **73**
S12.1 Equivalence with context-free grammars . . . 73

S13 Non-Context-Free Languages **75**
S13.1 The pumping lemma for context-free languages . . . 75

S14 Turing Machines **76**
S14.1 Turing machines . . . 76
S14.2 Formal definition of a Turing machine . . . 76
S14.3 Examples of Turing machines . . . 77

S15 Variants of Turing Machines **79**
S15.1 Variants of Turing machines . . . 79
S15.2 Multitape Turing machines . . . 79
S15.3 Nondeterministic Turing machines . . . 79
S15.4 Enumerators . . . 80
S15.5 Equivalence with other models . . . 80

S16 Decidability and Undecidability **81**

S16.1 §4.1 Decidable languages 81

S16.2 §4.2 Undecidability 82

วิธีใช้เล่มนี้

- อ่านเรียงบท แต่ละบทเริ่มด้วย Objectives และจบด้วยกล่องสรุปสูตร
- คำสำคัญพิมพ์**สีน้ำเงินตัวหนา**
- กล่องสี : **เขียวเข้ม** = นิยาม **ม่วง** = ทฤษฎีบท **ส้ม** = สูตร/วิธีสร้าง **น้ำเงิน** = คำอธิบายเพิ่ม **แดง** = จุดที่ผิดบ่อย **เทา** = ตัวอย่าง
- ก่อนสอบ : อ่านบทที่ 19 (รวมสูตร) แล้วย้อนดูตัวอย่างของบทที่ยังไม่แม่น

บทที่ 0

พื้นฐาน : เซต สตริง และภาษา

Objectives — อ่านจบแล้วต้องทำได้

- อ่านและเขียนสัญลักษณ์ของเซตได้
- แยก alphabet, string, language ออกจากกันได้
- คำนวณ operation บนสตริงและบนภาษาได้
- แยก $\emptyset$, ε, $\{\varepsilon\}$ ออกจากกันได้

0.1 เซต

- **เซต (set)** = กลุ่มของสิ่งของ ไม่สนลำดับ ไม่นับตัวซ้ำ เช่น $\{0,1\} = \{1,0\}$
- $a \in A$: a เป็นสมาชิกของ A
- $A \subseteq B$: ทุกสมาชิกของ A อยู่ใน B
- $\emptyset$ = เซตว่าง
- $|A|$ = จำนวนสมาชิก
- $A \cup B$ (union), $A \cap B$ (intersection), $\overline{A}$ (complement), $A \setminus B$ (difference)
- **Cartesian product** : $A \times B = \{(a,b) : a \in A,\ b \in B\}$
- **Power set** : $\mathcal{P}(A)$ = เซตของ subset ทั้งหมดของ A

สูตร

$|A \times B| = |A| \cdot |B|$ $\quad$ $|\mathcal{P}(A)| = 2^{|A|}$ $\quad$ De Morgan : $\overline{A \cup B} = \overline{A} \cap \overline{B}$, $\quad \overline{A \cap B} = \overline{A} \cup \overline{B}$

0.2 Alphabet และ String

- **Alphabet** Σ = เซตจำกัดที่ไม่ว่างของสัญลักษณ์ เช่น $\Sigma = \{0,1\}$
- **String** = ลำดับจำกัดของสัญลักษณ์จาก Σ เช่น $w = 0110$
- $|w|$ = ความยาว
- ε = **สตริงว่าง** มีความยาว 0
- **Concatenation** : xy = เอา y มาต่อท้าย x เช่น $x = 01$, $y = 110$ ได้ $xy = 01110$
- w^k = w ต่อกัน k ครั้ง และ $w^0 = \varepsilon$
- w^R = **reverse** ของ w เช่น $(011)^R = 110$
- **prefix** / **suffix** / **substring** : ส่วนหน้า / ส่วนท้าย / ส่วนที่ติดกันของ w
- Σ^n = สตริงยาว n ทั้งหมด
- Σ^* = สตริงทั้งหมด (รวม ε)
- $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$

สูตร

$|xy| = |x| + |y|$ $\quad$ $\varepsilon w = w\varepsilon = w$ $\quad$ $(xy)^R = y^R x^R$ $\quad$ $|\Sigma^n| = |\Sigma|^n$
สตริงยาว n มี prefix $n+1$ ตัว suffix $n+1$ ตัว substring ไม่เกิน $\frac{n(n+1)}{2} + 1$ ตัว

0.3 Language

- **Language** = เซตของสตริง คือ $L \subseteq \Sigma^*$
- ภาษาอาจมีสมาชิกจำกัดหรือไม่จำกัดก็ได้
- ตัวอย่าง : $L = \{w \in \{0,1\}^* : w \text{ ลงท้ายด้วย } 1\} = \{1, 01, 11, 001, \dots\}$
- "เครื่อง M **recognize** ภาษา L" หมายถึง M accept สตริงใน L ทุกตัว และไม่ accept ตัวอื่นเลย

0.4 Operation บนภาษา

Regular operations

- **Union** : $A \cup B = \{x : x \in A \text{ หรือ } x \in B\}$
- **Concatenation** : $A \circ B = AB = \{xy : x \in A \text{ และ } y \in B\}$
- **Star** : $A^* = \{x_1 x_2 \cdots x_k : k \geq 0 \text{ และทุก } x_i \in A\}$

และ $A^+ = AA^*$ (อย่างน้อย 1 ชิ้น) A^k = A ต่อกัน k ครั้ง $A^0 = \{\varepsilon\}$

ตัวอย่าง : $A = \{0, 01\}$ และ $B = \{1, 11\}$

- $A \cup B = \{0, 01, 1, 11\}$
- $AB = \{01,\ 011,\ 0111\}$ (ได้ $01{\cdot}1 = 011$ และ $0{\cdot}11 = 011$ ซ้ำกัน จึงเหลือ 3 ตัว)
- $A^* = \{\varepsilon,\ 0,\ 01,\ 00,\ 001,\ 010,\ 0101, \dots\}$

ระวัง — จุดที่ผิดบ่อย

- $\emptyset$ = ภาษาที่ไม่มีสตริงเลย $\{\varepsilon\}$ = ภาษาที่มีสตริง 1 ตัว (คือสตริงว่าง) ε = สตริง ไม่ใช่ภาษา
- $A \circ \emptyset = \emptyset$ แต่ $A \circ \{\varepsilon\} = A$
- $\emptyset^* = \{\varepsilon\}$ เพราะ $k = 0$ ได้ ε เสมอ ดังนั้น $\varepsilon \in A^*$ ทุกภาษา A
- $|AB| \leq |A| \cdot |B|$ ไม่จำเป็นต้องเท่ากัน (ดูตัวอย่างข้างบน)

0.5 เทคนิคการพิสูจน์ที่ใช้ทั้งวิชา

- **Proof by construction** : จะพิสูจน์ว่า "มีเครื่อง" ก็สร้างเครื่องให้ดู
- **Proof by contradiction** : สมมติสิ่งตรงข้าม แล้วหาข้อขัดแย้ง (ใช้กับ pumping lemma)
- **Proof by induction** : ฐาน + ขั้น induction (ใช้พิสูจน์ความถูกต้องของเครื่อง/grammar)
- **Pigeonhole principle** : นก $n+1$ ตัว เข้า n รัง ต้องมีรังที่มีนกอย่างน้อย 2 ตัว (ที่มาของ pumping lemma)

บทที่ 1

Deterministic Finite Automata (DFA)

Objectives — อ่านจบแล้วต้องทำได้

- เขียนนิยาม 5-tuple ของ DFA ได้
- อ่าน state diagram และ transition table ได้
- trace การทำงานของ DFA บน input ได้
- ออกแบบ DFA จากคำบรรยายภาษาได้

1.1 แนวคิด

- DFA = เครื่องที่มี**ความจำจำกัด** ความจำทั้งหมดคือ “ตอนนี้อยู่ state ไหน”
- อ่าน input ทีละตัวจากซ้ายไปขวา อ่านแล้วย้อนกลับไม่ได้
- อ่านหมดแล้วอยู่ที่ **accept state** = accept ไม่งั้น = reject
- **Deterministic** : จากแต่ละ state อ่านแต่ละสัญลักษณ์ มีทางไป**ทางเดียวพอดี**

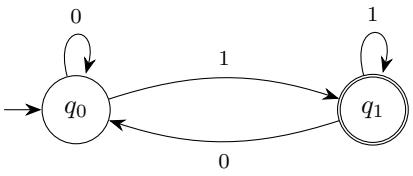
1.2 นิยาม

นิยาม DFA

$M = (Q, \Sigma, \delta, q_0, F)$

- Q = เซตจำกัดของ state
- Σ = alphabet
- $\delta : Q \times \Sigma \to Q$ = **transition function**
- $q_0 \in Q$ = **start state** (มีตัวเดียว)
- $F \subseteq Q$ = เซตของ **accept state** (มีกี่ตัวก็ได้ รวมทั้ง 0 ตัว)

1.3 การอ่าน state diagram



- วงกลม = state ลูกศรลอยเข้า = start state
- **วงกลมสองชั้น** = accept state
- เส้นจาก p ไป q ป้าย a หมายถึง $\delta(p, a) = q$
- เครื่องนี้ recognize $\{w : w \text{ ลงท้ายด้วย } 1\}$ เพราะ state จำแค่ “ตัวล่าสุดเป็น 1 หรือไม่”

Transition table ของเครื่องเดียวกัน

δ	0	1
$\to q_0$	q_0	q_1
$* q_1$	q_0	q_1

($\to$ = start state $*$ = accept state)

1.4 การคำนวณอย่างเป็นทางการ

นิยาม accept

M accept $w = w_1 w_2 \cdots w_n$ ถ้ามีลำดับ state $r_0, r_1, \ldots, r_n$ ที่

1. $r_0 = q_0$
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ ทุก $i = 0, \ldots, n-1$
3. $r_n \in F$

Extended transition function : $\hat{\delta}(q, \varepsilon) = q$ $\hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$

$L(M) = \{w \in \Sigma^* : \hat{\delta}(q_0, w) \in F\}$ ภาษา L เป็น regular ถ้ามี DFA ที่ recognize L

ตัวอย่าง : trace เครื่องข้างบน ด้วย input 0110 และ 011

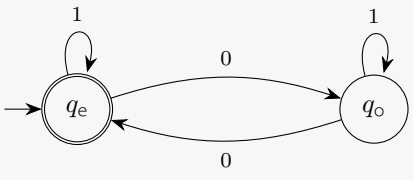
- 0110 :
  - $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_1 \xrightarrow{0} q_0$
  - จบที่ $q_0 \notin F$
  - **reject**
- 011 :
  - $q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{1} q_1$
  - จบที่ $q_1 \in F$
  - **accept**
- ε : ไม่ขยับ จบที่ q_0 **reject**

1.5 วิธีออกแบบ DFA

★ Claude สอนเพิ่ม : คิดว่าตัวเองเป็นเครื่อง

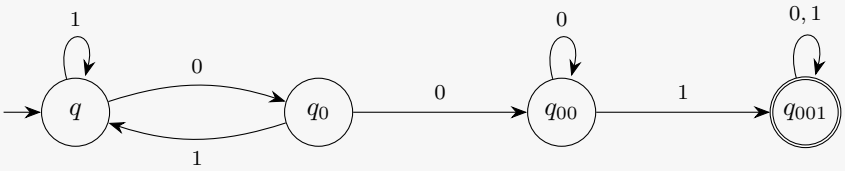
1. ถามตัวเอง : “ถ้า input หยุดตรงนี้ ต้องตอบได้ทันที ฉันต้อง**จำอะไร**เกี่ยวกับสิ่งที่อ่านมาแล้ว?”
2. สิ่งที่ต้องจำแต่ละแบบ = **1 state** (ต้องมีจำนวนจำกัด)
3. หา start state : สถานะตอนยังไม่อ่านอะไร
4. เติม transition : จากแต่ละ state อ่านแต่ละสัญลักษณ์ แล้วความจำเปลี่ยนเป็นแบบไหน
5. เลือก accept state : state ที่ความจำบอกว่า “สตริงอยู่ในภาษา”
6. ทดสอบด้วย ε, สตริงสั้น ๆ ทั้งที่ควร accept และควร reject

ตัวอย่าง : จำนวน 0 เป็นเลขคู่



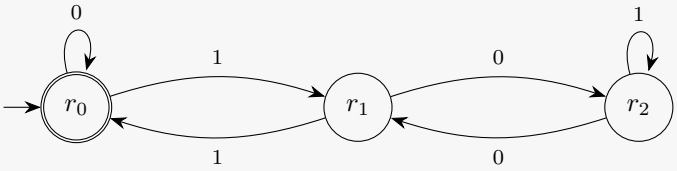
- สิ่งที่ต้องจำ : จำนวน 0 ที่เห็นมา เป็นคู่ (q_e) หรือคี่ (q_o)
- start state เป็น accept state ด้วย เพราะ ε มี 0 อยู่ 0 ตัว ซึ่งเป็นเลขคู่

ตัวอย่าง : มี 001 เป็น substring



- สิ่งที่ต้องจำ : “ตอนนี้ท้ายสตริงตรงกับ 001 ไปแล้วกี่ตัว” $(0,\ 1,\ 2$ ตัว หรือเจอครบแล้ว$)$
- q_{00} อ่าน 0 อยู่ที่เดิม เพราะท้ายสตริงยังเป็น 00
- q_{001} เป็น**กับดักฝั่ง accept** : เจอแล้วไม่มีวันหลุด

ตัวอย่าง : เลขฐานสองที่หาร 3 ลงตัว



- สิ่งที่ต้องจำ : เศษจากการหาร 3 ของเลขที่อ่านมา $(r_i$ = เศษ $i)$
- อ่านบิต b ต่อท้าย : ค่าใหม่ $= 2 \times$ ค่าเดิม $+ b$ ดังนั้น $\delta(r_i, b) = r_{(2i+b) \bmod 3}$
- ตรวจ : $110_2 = 6$ $r_0 \xrightarrow{1} r_1 \xrightarrow{1} r_0 \xrightarrow{0} r_0$ accept
- เครื่องนี้ accept ε ด้วย (ถือว่า ε มีค่า 0) ถ้าไม่ต้องการ ให้เพิ่ม start state แยกอีกตัว

1.6 รูปแบบที่เจอบ่อย

ภาษา	สิ่งที่ state จำ	จำนวน state
ลงท้ายด้วย u	ท้ายสตริงตรงกับ u ไปแล้วกี่ตัว	$\lvert u\rvert + 1$
มี u เป็น substring	เหมือนข้างบน + กับดัก accept	$\lvert u\rvert + 1$
ขึ้นต้นด้วย u	ตรงกับ u ไปแล้วกี่ตัว + กับดัก reject	$\lvert u\rvert + 2$
จำนวน a หาร k เหลือเศษ r	จำนวน $a \bmod k$	k
ความยาวหาร k ลงตัว	ความยาว $\bmod k$	k
ค่าฐานสองหาร k ลงตัว	ค่า $\bmod k$	k
ไม่มีสมบัติ P	สร้าง DFA ของ P แล้ว**สลับ** accept/non-accept	เท่าเดิม

ระวัง — จุดที่ผิดบ่อย

- DFA ต้องมีเส้นออก**ครบทุกสัญลักษณ์จากทุก state** ถ้า “ผิดแล้วผิดเลย” ให้ส่งไป dead state (กับดัก reject)
- อย่าลืมตรวจ ε : ถ้า ε อยู่ในภาษา start state ต้องเป็น accept state
- เลือก accept state ผิดบ่อย : accept ดูที่ state **ตอนอ่านจบ**เท่านั้น ระหว่างทางผ่าน accept state ไม่มีผล
- DFA นับไม่จำกัดไม่ได้ : ถ้าต้องจำ “จำนวนเท่าไรก็ได้” (เช่น 0^n1^n) แปลว่าภาษาไม่ regular

บทที่ 2

Nondeterministic Finite Automata (NFA)

Objectives — อ่านจบแล้วต้องทำได้

- บอกความต่างของ NFA กับ DFA ได้ 3 ข้อ
- เขียนนิยาม 5-tuple ของ NFA ได้
- trace NFA โดยติดตาม “เซตของ state ที่เป็นไปได้”
- ออกแบบ NFA ด้วยเทคนิค “เดา”

2.1 NFA ต่างจาก DFA อย่างไร

	DFA	NFA
เส้นออกต่อสัญลักษณ์	พอดี 1 เส้น	0, 1 หรือหลายเส้น
ε-transition	ไม่มี	มีได้ (ย้าย state โดยไม่อ่าน input)
δ ให้ผลเป็น	state เดียว	**เซต**ของ state
accept เมื่อ	เส้นทางเดียวจบที่ accept state	**มีอย่างน้อย 1 เส้นทาง**จบที่ accept state
ภาษาที่รับได้	regular	regular (เท่ากัน)

2.2 นิยาม

นิยาม NFA

$N = (Q, \Sigma, \delta, q_0, F)$ เหมือน DFA ทุกอย่าง ยกเว้น

$$\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q) \qquad \text{โดย } \Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$$

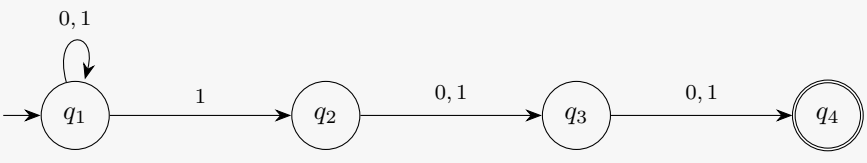
N **accept** w ถ้าเขียน $w = y_1 y_2 \cdots y_m$ (แต่ละ $y_i \in \Sigma_\varepsilon$) และมีลำดับ state $r_0, \ldots, r_m$ ที่

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, y_{i+1})$
3. $r_m \in F$

2.3 วิธีคิดเรื่อง nondeterminism

- **มุมมองแตกกิ่ง** : เจอหลายทางให้ “แยกร่าง” ไปทุกทางพร้อมกัน ร่างที่ไม่มีทางไปก็ตาย
- **มุมมองเดา** : เครื่องเดาทางถูกเสมอ ถ้ามีทางถูกอยู่
- **มุมมองเซต** : ติดตาม “เซตของ state ที่เป็นไปได้ตอนนี้” (ใช้ trace จริง และเป็นที่มาของ subset construction)

ตัวอย่าง : ตัวที่ 3 นับจากท้ายเป็น 1



- q_1 วนรอ แล้ว**เดา**ว่า 1 ตัวนี้คือตัวที่ 3 จากท้าย จากนั้นนับอีก 2 ตัวพอดี

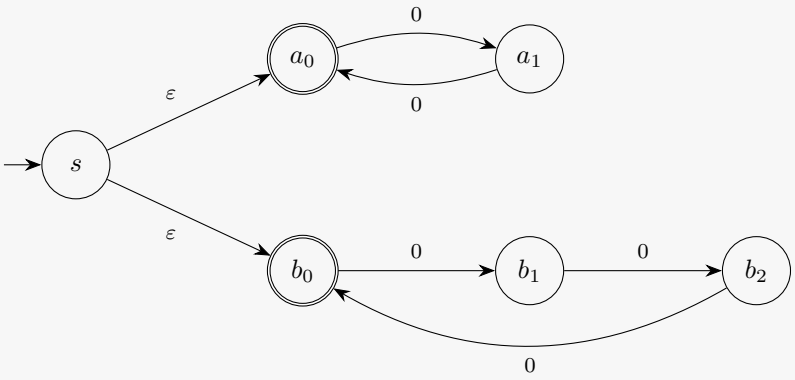
- q_4 ไม่มีเส้นออก : ถ้ายังมี input เหลือ ร่างนั้นตาย

trace input 1011 แบบเซต :

อ่านไปแล้ว	ε	1	10	101	1011
เซตของ state	$\{q_1\}$	$\{q_1, q_2\}$	$\{q_1, q_3\}$	$\{q_1, q_2, q_4\}$	$\{q_1, q_2, q_3\}$

- จบที่ $\{q_1, q_2, q_3\}$ ไม่มี q_4 **reject** (ตัวที่ 3 จากท้ายของ 1011 คือ 0)
- ถ้า input เป็น 101 จบที่ $\{q_1, q_2, q_4\}$ มี q_4 **accept**

ตัวอย่าง : ε-transition : $\{0^k : k$ หาร 2 หรือ 3 ลงตัว$\}$



- ε สองเส้นแรก = เดาว่าจะตรวจ “หาร 2 ลงตัว” (สายบน) หรือ “หาร 3 ลงตัว” (สายล่าง)
- นี่คือสูตรสร้าง union ด้วย NFA : start state ใหม่ + ε ไปยังเครื่องย่อยแต่ละเครื่อง

★ Claude สอนเพิ่ม : เมื่อไรควรใช้ NFA

- โจทย์มีคำว่า “มี ... อยู่ที่ไหนสักแห่ง”, “ลงท้ายด้วย”, “ตัวที่ k จากท้าย”, “หรือ” NFA จะเล็กกว่า DFA มาก
- โจทย์ถามเรื่อง closure (union, concatenation, star) ใช้ NFA ต่อเครื่องด้วย ε
- โจทย์มีคำว่า “ไม่มี”, “ทุกตำแหน่ง” ทำ DFA ง่ายกว่า (เพราะ NFA สลับ accept state แล้ว**ไม่ได้** complement)

ระวัง — จุดที่ผิดบ่อย

- NFA **ไม่ต้อง**มีเส้นออกครบ ไม่มีเส้น = ร่างนั้นตาย (ไม่ใช่ reject ทั้งเครื่อง)
- NFA reject ก็ต่อเมื่อ**ทุก**เส้นทางล้มเหลว
- สลับ accept/non-accept ของ NFA ไม่ได้ภาษา complement ต้องแปลงเป็น DFA ก่อน
- DFA ทุกเครื่องเป็น NFA อยู่แล้ว (เป็นกรณีพิเศษที่ทุกเซตมีสมาชิกเดียว)

บทที่ 3

แปลง NFA เป็น DFA (Subset Construction)

Objectives — อ่านจบแล้วต้องทำได้

- คำนวณ ε-closure ได้
- แปลง NFA (ทั้งแบบมีและไม่มี ε) เป็น DFA ด้วยตารางได้
- อธิบายได้ว่าทำไมจำนวน state อาจโตถึง 2^n

3.1 ทฤษฎีบท

NFA = DFA

NFA ทุกเครื่องมี DFA ที่ recognize ภาษาเดียวกัน ดังนั้น

$$L \text{ regular} \iff \text{มี DFA recognize } L \iff \text{มี NFA recognize } L$$

3.2 แนวคิด

- DFA จำลอง NFA โดยให้ **1 state ของ DFA = 1 เซตของ state ของ NFA**
- คือทำ “trace แบบเซต” ของบทที่ 2 ไว้ล่วงหน้าทุกกรณี

3.3 สูตร

Subset construction

ให้ NFA $N = (Q, \Sigma, \delta, q_0, F)$ สร้าง DFA $M = (Q', \Sigma, \delta', q_0', F')$

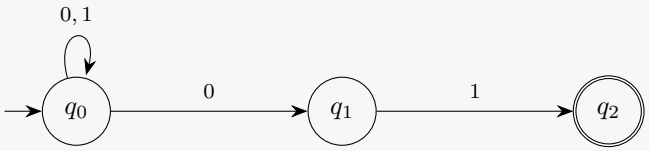
- ε-closure : $E(R)$ = state ทั้งหมดที่ไปถึงได้จาก R ด้วย ε จำนวน 0 เส้นขึ้นไป
- $Q' = \mathcal{P}(Q)$
- $q_0' = E(\{q_0\})$
- $\delta'(R, a) = E\Big(\bigcup_{r \in R} \delta(r, a) \Big)$
- $F' = \{R \in Q' : R \cap F \neq \emptyset\}$ (เซตที่มี accept state ของ N อย่างน้อย 1 ตัว)

ถ้า N ไม่มี ε-transition : $E(R) = R$ สูตรจึงเหลือ $\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$

3.4 ขั้นตอนทำด้วยตาราง

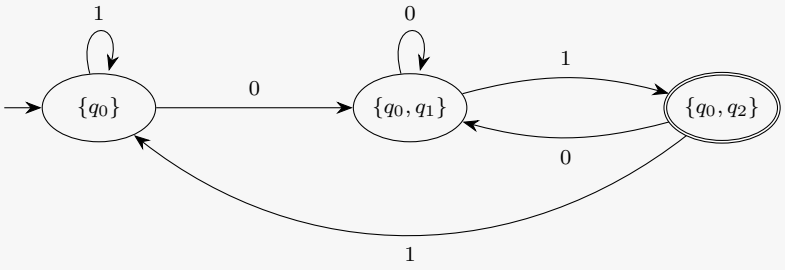
1. เริ่มที่ $q_0' = E(\{q_0\})$ เขียนเป็นแถวแรก
2. สำหรับแต่ละแถว R และแต่ละสัญลักษณ์ a : รวมปลายทางของทุก state ใน R แล้วทำ $E(\cdot)$
3. ถ้าได้เซตที่ยังไม่เคยมี ให้เพิ่มเป็นแถวใหม่
4. ทำซ้ำจนไม่มีแถวใหม่ (สร้างเฉพาะเซตที่**ไปถึงได้** ไม่ต้องไล่ครบ 2^n)
5. เซตว่าง $\emptyset$ คือ dead state : $\delta'(\emptyset, a) = \emptyset$
6. ทำเครื่องหมาย accept ให้ทุกเซตที่มี accept state ของ NFA

ตัวอย่าง : NFA ไม่มี ε : ลงท้ายด้วย 01

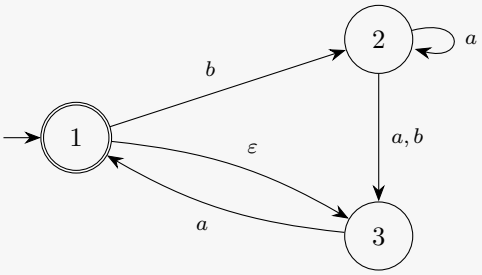


δ'	0	1
$\to \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$*\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$

- ตัวอย่างการคิด 1 ช่อง : $\delta'(\{q_0, q_1\}, 1) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$
- ได้ DFA 3 state (จากที่เป็นไปได้ $2^3 = 8$) :



ตัวอย่าง : NFA มี ε (Sipser Example 1.41)



- ε-closure : $E(\{1\}) = \{1, 3\}$ $E(\{2\}) = \{2\}$ $E(\{3\}) = \{3\}$
- start state ของ DFA คือ $E(\{1\}) = \{1, 3\}$ (**ไม่ใช่** $\{1\}$)

δ'	a	b
$\to *\{1, 3\}$	$\{1, 3\}$	$\{2\}$
$\{2\}$	$\{2, 3\}$	$\{3\}$
$\{2, 3\}$	$\{1, 2, 3\}$	$\{3\}$
$\{3\}$	$\{1, 3\}$	$\emptyset$
$*\{1, 2, 3\}$	$\{1, 2, 3\}$	$\{2, 3\}$
$\emptyset$	$\emptyset$	$\emptyset$

- คิดช่อง $\delta'(\{3\}, a)$: $\delta(3, a) = \{1\}$ แล้วทำ $E(\{1\}) = \{1, 3\}$
- คิดช่อง $\delta'(\{2, 3\}, a)$: $\delta(2, a) \cup \delta(3, a) = \{2, 3\} \cup \{1\} = \{1, 2, 3\}$ ทำ E แล้วได้เท่าเดิม
- accept state = เซตที่มี state 1 : คือ $\{1, 3\}$ และ $\{1, 2, 3\}$
- ได้ DFA 6 state (จากที่เป็นไปได้ $2^3 = 8$)

สูตรนับ

NFA มี n state $\Rightarrow$ DFA มีไม่เกิน 2^n state
กรณีแย่สุดเกิดจริง : ภาษา “ตัวที่ k จากท้ายเป็น 1” NFA ใช้ $k+1$ state DFA ต้องใช้อย่างน้อย 2^k state

ระวัง — จุดที่ผิดบ่อย

- ลืมทำ ε-closure ที่ start state และหลังอ่านแต่ละสัญลักษณ์
- ลืมแถว $\emptyset$ (DFA ต้องมีเส้นออกครบ)
- กำหนด accept ผิด : ต้องเป็นเซตที่มี accept state **อย่างน้อยหนึ่งตัว** ไม่ใช่ทุกตัว

บทที่ 4

Closure ของ Regular Languages

Objectives — อ่านจบแล้วต้องทำได้

- สร้างเครื่องสำหรับ union, intersection ด้วย product construction ได้
- สร้างเครื่องสำหรับ union, concatenation, star ด้วย NFA ได้
- ใช้ closure ลดงานพิสูจน์ได้

4.1 ความหมายของ “ปิด”

- class ของภาษา **ปิดภายใต้** operation หนึ่ง ถ้าเอาภาษาใน class ไปทำ operation แล้วผลลัพธ์ยังอยู่ใน class
- พิสูจน์ด้วย construction : ให้เครื่องของ A_1, A_2 มา แล้วสร้างเครื่องของผลลัพธ์

4.2 Product construction (ใช้ DFA)

Product construction

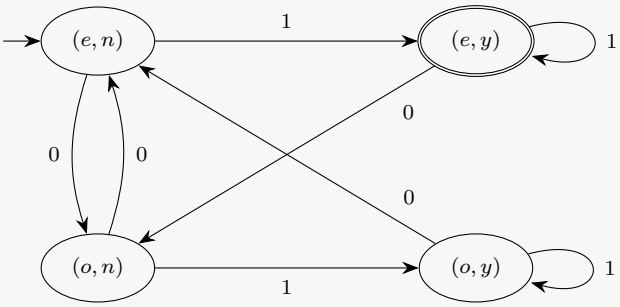
ให้ $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ และ $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ สร้าง $M = (Q, \Sigma, \delta, q_0, F)$

- $Q = Q_1 \times Q_2$
- $q_0 = (q_1, q_2)$
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a),\ \delta_2(r_2, a))$ (เดินสองเครื่องพร้อมกัน)
- Union : $F = \{(r_1, r_2) : r_1 \in F_1$ **หรือ** $r_2 \in F_2\}$
- Intersection : $F = \{(r_1, r_2) : r_1 \in F_1$ **และ** $r_2 \in F_2\} = F_1 \times F_2$
- Difference $A_1 \setminus A_2$: $F = \{(r_1, r_2) : r_1 \in F_1$ และ $r_2 \notin F_2\}$

จำนวน state $= |Q_1| \cdot |Q_2|$

ตัวอย่าง : (จำนวน 0 เป็นคู่) ∩ (ลงท้ายด้วย 1)

- M_1 : state e (คู่), o (คี่) M_2 : state n (ไม่ลงท้ายด้วย 1), y (ลงท้ายด้วย 1)
- อ่าน 0 : สลับ $e \leftrightarrow o$ และไป n
- อ่าน 1 : e/o คงเดิม และไป y



- intersection : accept เฉพาะ (e, y) (ตามรูป)
- ถ้าต้องการ union : accept (e, n), (e, y), (o, y) โดยใช้ diagram เดิม เปลี่ยนแค่ F

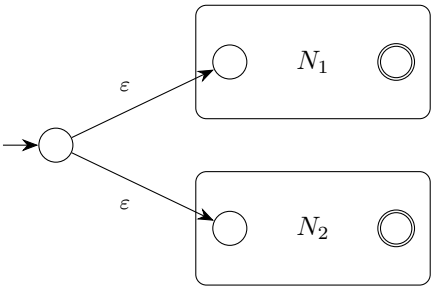
4.3 Complement

Complement

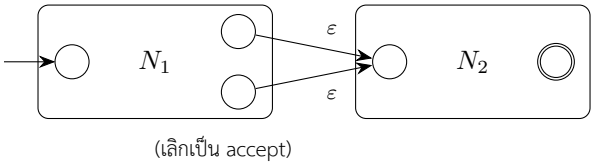
ให้ DFA $M = (Q, \Sigma, \delta, q_0, F)$ recognize A แล้ว $\overline{M} = (Q, \Sigma, \delta, q_0, Q \setminus F)$ recognize $\overline{A}$

- ใช้ได้กับ **DFA เท่านั้น** ถ้ามี NFA ต้องทำ subset construction ก่อน
- ได้ intersection อีกทางหนึ่งจาก De Morgan : $A \cap B = \overline{\overline{A} \cup \overline{B}}$

4.4 Union, Concatenation, Star (ใช้ NFA)

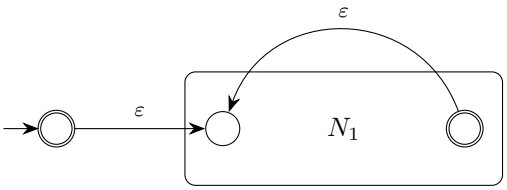


Union $A_1 \cup A_2$



(เลิกเป็น accept)

Concatenation $A_1 A_2$



Star A_1^*

สูตรสร้างด้วย NFA

- **Union** : เพิ่ม start state ใหม่ ลาก ε ไป start state ของ N_1 และ N_2 accept state คงเดิมทั้งหมด
- **Concatenation** : start state ของ N_1 ลาก ε จาก**ทุก** accept state ของ N_1 ไป start state ของ N_2 accept state ของ N_1 **เลิกเป็น** accept เหลือแต่ของ N_2
- **Star** : เพิ่ม start state ใหม่**ที่เป็น accept** ลาก ε ไป start state เดิม ลาก ε จากทุก accept state กลับไป start state เดิม

จำนวน state : union $= n_1 + n_2 + 1$ concatenation $= n_1 + n_2$ star $= n_1 + 1$

ระวัง — Star

- ต้องเพิ่ม start state **ใหม่** เพื่อรับ ε ห้ามแค่เปลี่ยน start state เดิมให้เป็น accept
- เหตุผล : ถ้า start state เดิมมีเส้นวนกลับมา จะ accept สตริงที่ไม่ควร accept

4.5 สรุป closure และการนำไปใช้

Regular languages ปิดภายใต้

union · intersection · complement · difference · concatenation · star · reverse

★ Claude สอนเพิ่ม : ใช้ closure ทำอะไรได้

- **พิสูจน์ว่า regular :** แตกภาษาเป็นชิ้นที่รู้ว่า regular แล้วประกอบด้วย operation ที่ปิด
  เช่น “จำนวน 0 เป็นคู่ และไม่มี 11” $= A \cap \overline{B}$
- **พิสูจน์ว่าไม่ regular :** สมมติว่า L regular เอาไป $\cap$ กับภาษา regular แล้วได้ภาษาที่รู้ว่าไม่ regular ขัดแย้ง
  เช่น $\{w : \#0 = \#1\} \cap 0^*1^* = \{0^n1^n\}$
- **reverse :** กลับทิศลูกศรทุกเส้น start state เดิมเป็น accept เพิ่ม start state ใหม่ลาก ε ไป accept state เดิมทุกตัว

บทที่ 5

Regular Expression

Objectives — อ่านจบแล้วต้องทำได้

- เขียนนิยาม regular expression 6 กรณีได้
- ใช้ลำดับความสำคัญของ operator ได้ถูก
- แปลงคำบรรยายภาษาเป็น regular expression และกลับกันได้
- ใช้ identity ลดรูปได้

5.1 นิยาม

Regular expression

R เป็น regular expression ถ้า R คือ

1. a สำหรับ $a \in \Sigma$ — ภาษา $\{a\}$
2. ε — ภาษา $\{\varepsilon\}$
3. $\emptyset$ — ภาษาว่าง
4. $(R_1 \cup R_2)$ — union
5. $(R_1 \circ R_2)$ เขียนย่อ R_1R_2 — concatenation
6. (R_1^*) — star

โดย R_1, R_2 เป็น regular expression เขียน $L(R)$ แทนภาษาของ R

ลำดับความสำคัญ และตัวย่อ

star ก่อน แล้ว concatenation แล้ว union เช่น $ab^* \cup c = (a(b^*)) \cup c$

$R^+ = RR^*$ $R^k = \underbrace{R \cdots R}_{k}$ Σ = สัญลักษณ์ตัวใดก็ได้ 1 ตัว เช่น $\Sigma = (0 \cup 1)$

- บางตำราเขียน $+$ หรือ | แทน $\cup$ เช่น $(0+1)^*$ หรือ `(0|1)*`

5.2 ตัวอย่าง ($\Sigma = \{0, 1\}$)

Regular expression	ภาษา
0^*10^*	มี 1 พอดี 1 ตัว
$\Sigma^*1\Sigma^*$	มี 1 อย่างน้อย 1 ตัว
$\Sigma^*001\Sigma^*$	มี 001 เป็น substring
Σ^*01	ลงท้ายด้วย 01
$\Sigma^*1\Sigma\Sigma$	ตัวที่ 3 จากท้ายเป็น 1
$(\Sigma\Sigma)^*$	ความยาวเป็นเลขคู่
$(\Sigma\Sigma\Sigma)^*$	ความยาวหาร 3 ลงตัว
$(1 \cup 01^*0)^*$	จำนวน 0 เป็นเลขคู่
$1^*(01^+)^*$	หลัง 0 ทุกตัวมี 1 อย่างน้อย 1 ตัว
$(1 \cup 01)^*(0 \cup \varepsilon)$	ไม่มี 00 เป็น substring
$0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1$	ขึ้นต้นและลงท้ายด้วยตัวเดียวกัน
$(0 \cup \varepsilon)(1 \cup \varepsilon)$	$\{\varepsilon, 0, 1, 01\}$
$1^*\emptyset$	$\emptyset$
$\emptyset^*$	$\{\varepsilon\}$

5.3 Identity ที่ใช้ลดรูป

Identity

$R \cup \emptyset = R$	$R \circ \varepsilon = \varepsilon \circ R = R$	$R \circ \emptyset = \emptyset \circ R = \emptyset$
$R \cup R = R$	$R \cup S = S \cup R$	$R(S \cup T) = RS \cup RT$
$\emptyset^* = \varepsilon$	$\varepsilon^* = \varepsilon$	$(R^*)^* = R^*$
$R^* = \varepsilon \cup R^+$	$R^*R^* = R^*$	$(R \cup S)^* = (R^*S^*)^*$
$(RS)^*R = R(SR)^*$	$(\varepsilon \cup R)^* = R^*$	$R^* = \varepsilon \cup RR^*$

ระวัง — จุดที่ผิดบ่อย

- $R \cup \varepsilon$ **ไม่เท่ากับ** R เสมอไป เช่น $R = 0 : L(R) = \{0\}$ แต่ $L(R \cup \varepsilon) = \{0, \varepsilon\}$
- $R \circ \emptyset$ **ไม่เท่ากับ** R ต่อกับภาษาว่างได้ภาษาว่าง
- $RS \neq SR$ โดยทั่วไป concatenation ไม่สลับที่
- $(R \cup S)^* \neq R^* \cup S^*$ เช่น $01 \in (0 \cup 1)^*$ แต่ $01 \notin 0^* \cup 1^*$
- $(RS)^* \neq R^*S^*$ เช่น $0101 \in (01)^*$ แต่ $0101 \notin 0^*1^*$

★ Claude สอนเพิ่ม : วิธีเขียน regular expression

- “มี u” : $\Sigma^*u\Sigma^*$
- “ขึ้นต้นด้วย u” : $u\Sigma^*$
- “ลงท้ายด้วย u” : Σ^*u
- “นับ mod k” : จัดกลุ่มละ k แล้วใส่ star เช่น $(\Sigma^k)^*$
- “ไม่มี u” : เขียนตรง ๆ ยาก ให้คิดว่า “สตริงถูกสร้างจากบล็อกอะไรได้บ้าง” หรือสร้าง DFA แล้วแปลงตามบทที่ 6
- ตรวจคำตอบด้วยสตริงสั้น : ε, สตริงยาว 1, และสตริงที่อยู่ “ขอบ” ของเงื่อนไข

บทที่ 6

Regular Expression ⇔ Finite Automata

Objectives — อ่านจบแล้วต้องทำได้

- แปลง regular expression เป็น NFA ทีละชิ้นได้
- แปลง DFA เป็น regular expression ด้วย GNFA (state elimination) ได้
- จำสูตร $R_1 R_2^* R_3 \cup R_4$ ได้

6.1 ทฤษฎีบท

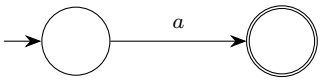
Kleene's theorem

ภาษา L เป็น regular $\iff$ มี regular expression ที่บรรยาย L

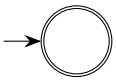
Regular expression $\xrightarrow{\text{ต่อชิ้น NFA}}$ NFA $\xrightarrow{\text{subset construction}}$ DFA $\xrightarrow{\text{GNFA}}$ Regular expression

6.2 Regular expression → NFA

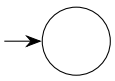
- สร้างตามนิยาม 6 กรณี 3 กรณีฐานคือเครื่องข้างล่าง อีก 3 กรณีใช้สูตร union / concatenation / star ของบทที่ 4



$R = a$



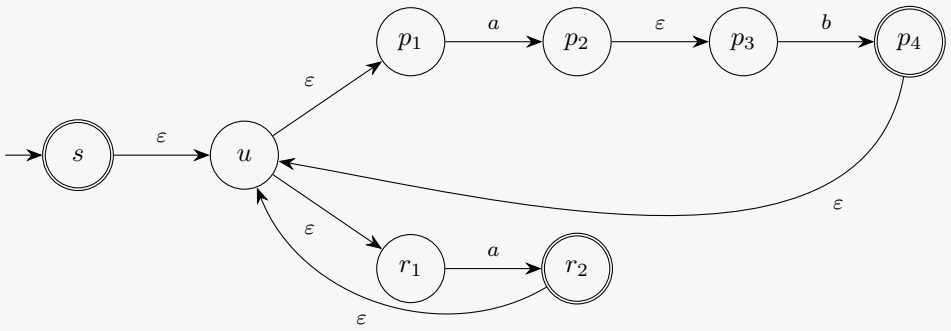
$R = \varepsilon$



$R = \emptyset$

ตัวอย่าง : $(ab \cup a)^*$

1. สร้าง a และ b ต่อกันด้วย ε ได้ ab (สายบน)
2. สร้าง a อีกตัว (สายล่าง)
3. union : เพิ่ม state u ลาก ε ไปทั้งสองสาย
4. star : เพิ่ม start state ใหม่ s ที่เป็น accept ลาก ε จาก accept state ทุกตัวกลับไป u



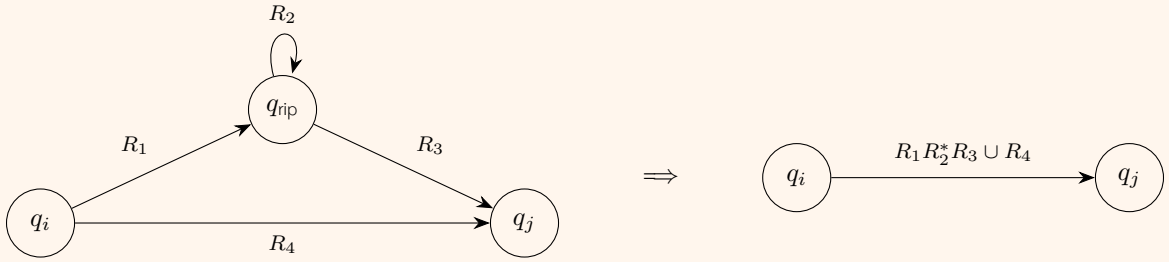
- สร้างแบบกลไก ไม่ต้องลดรูป มี ε เยอะเป็นเรื่องปกติ
- ตรวจ : aba $s \to u \to p_1 \xrightarrow{a} p_2 \to p_3 \xrightarrow{b} p_4 \to u \to r_1 \xrightarrow{a} r_2$ accept

6.3 DFA → Regular expression ด้วย GNFA

- GNFA = NFA ที่ป้ายบนเส้นเป็น **regular expression** ได้
- แนวคิด : ตัด state ออกทีละตัว แล้ว "ซ่อม" เส้นให้ภาษาคงเดิม จนเหลือ 2 state กับเส้นเดียว

สูตรตัด state q_{rip}

สำหรับทุกคู่ $q_i \to q_j$ ที่เดินผ่าน q_{rip} ได้ : ป้ายใหม่ของ $q_i \to q_j \quad = \quad R_1\, R_2^*\, R_3 \;\cup\; R_4$

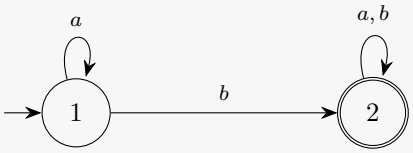


ถ้าไม่มีเส้นใด ให้ถือว่าป้ายเป็น $\emptyset$ (ไม่มี loop : $R_2 = \emptyset$ และ $\emptyset^* = \varepsilon$)

ขั้นตอน

1. เพิ่ม start state ใหม่ s : ลาก ε ไป start state เดิม
2. เพิ่ม accept state ใหม่ t : ลาก ε จาก accept state เดิม**ทุกตัว**มา แล้ว state เดิมเลิกเป็น accept
3. รวมเส้นขนาน : เส้น a และ b จาก p ไป q รวมเป็น $a \cup b$
4. เลือก state (ที่ไม่ใช่ s, t) มาตัด ใช้สูตรกับ**ทุกคู่** (เข้า, ออก) ของ state นั้น
5. ทำซ้ำจนเหลือแค่ s กับ t ป้ายบนเส้น $s \to t$ คือคำตอบ

ตัวอย่าง : DFA 2 state (Sipser Example 1.66)



1. เพิ่ม s และ t : $s \xrightarrow{\varepsilon} 1 \qquad 2 \xrightarrow{\varepsilon} t$ รวมเส้นขนานที่ state 2 เป็น $a \cup b$
2. **ตัด state 1** : คู่ $(s, 2)$ มี $R_1 = \varepsilon,\ R_2 = a,\ R_3 = b,\ R_4 = \emptyset$

$$s \to 2 : \qquad \varepsilon\, a^*\, b \cup \emptyset = a^* b$$

3. **ตัด state 2** : คู่ (s, t) มี $R_1 = a^*b,\ R_2 = a \cup b,\ R_3 = \varepsilon,\ R_4 = \emptyset$

$$s \to t : \qquad a^* b\, (a \cup b)^*\, \varepsilon \cup \emptyset = a^* b (a \cup b)^*$$

คำตอบ : $a^*b(a \cup b)^*$ (สตริงที่มี b อย่างน้อย 1 ตัว)

ตัวอย่าง : DFA "จำนวน 0 เป็นเลขคู่"

- DFA : $q_{\text{e}} \xrightarrow{1} q_{\text{e}},\quad q_{\text{e}} \xrightarrow{0} q_{\text{o}},\quad q_{\text{o}} \xrightarrow{1} q_{\text{o}},\quad q_{\text{o}} \xrightarrow{0} q_{\text{e}}$ start และ accept คือ q_{e}
- เพิ่ม $s \xrightarrow{\varepsilon} q_{\text{e}}$ และ $q_{\text{e}} \xrightarrow{\varepsilon} t$

1. **ตัด** q_{o} : คู่ $(q_{\text{e}}, q_{\text{e}})$ มี $R_1 = 0,\ R_2 = 1,\ R_3 = 0,\ R_4 = 1$

$$q_{\text{e}} \to q_{\text{e}} : \qquad 0\,1^*\,0 \cup 1$$

2. **ตัด** q_e : คู่ (s,t) มี $R_1 = \varepsilon$, $R_2 = 01^*0 \cup 1$, $R_3 = \varepsilon$, $R_4 = \emptyset$

$$s \to t : \quad (01^*0 \cup 1)^*$$

คำตอบ : $(1 \cup 01^*0)^*$

ระวัง — จุดที่ผิดบ่อย

- ลืมเพิ่ม s และ t ก่อนเริ่ม (ทำให้ start/accept state มีเส้นเข้าออกที่จัดการไม่ได้)
- ตัด state แล้วลืมบางคู่ : state ที่มี m เส้นเข้า และ n เส้นออก ต้องซ่อม $m \times n$ คู่ (รวมคู่ที่ $q_i = q_j$ ซึ่งกลายเป็น loop)
- ลืม $\cup R_4$ เส้นตรงที่มีอยู่เดิมต้องเก็บไว้
- ลำดับการตัดต่างกัน ได้นิพจน์หน้าตาต่างกัน แต่ภาษาเดียวกัน ตัด state ที่มีเส้นน้อยก่อนจะได้คำตอบสั้นกว่า

บทที่ 7

Pumping Lemma สำหรับ Regular Languages

Objectives — อ่านจบแล้วต้องทำได้

- เขียนตัวบท pumping lemma ได้ครบ 3 เงื่อนไข
- อธิบายที่มาจาก pigeonhole principle ได้
- พิสูจน์ว่าภาษาไม่ regular ด้วยโครง 6 ขั้นได้
- เลือกสตริง s และค่า i ที่เหมาะสมได้

7.1 ตัวบท

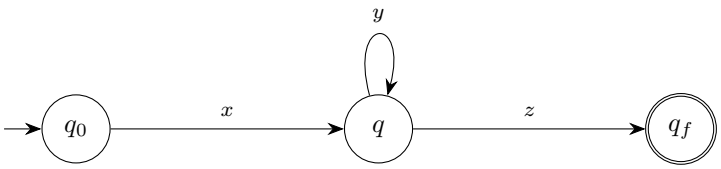
Pumping lemma (regular)

ถ้า A เป็น regular จะมีจำนวน p (pumping length) ที่ทำให้ ทุก $s \in A$ ที่ $|s| \geq p$ แบ่งได้เป็น $s = xyz$ โดย

1. $xy^iz \in A$ ทุก $i \geq 0$
2. $|y| > 0$
3. $|xy| \leq p$

7.2 ทำไมจึงจริง

- ให้ p = จำนวน state ของ DFA ที่ recognize A
- อ่านสตริงยาว $\geq p$ จะผ่าน state อย่างน้อย $p+1$ ครั้ง มีแค่ p state
- โดย pigeonhole ต้องมี state ที่**ซ้ำ**ภายใน p ตัวอักษรแรก
- ส่วนของสตริงระหว่างการซ้ำสองครั้งคือ y เป็น**วงวน**ที่จะเดินกี่รอบก็ได้ (รวม 0 รอบ)



7.3 วิธีใช้ : มองเป็นเกม

ตา	ใครเลือก	เลือกอะไร
1	ฝ่ายตรงข้าม	ค่า p (เราไม่รู้ค่า ต้องใช้เป็นตัวแปร)
2	**เรา**	สตริง $s \in A$ ที่ $\|s\| \geq p$
3	ฝ่ายตรงข้าม	การแบ่ง $s = xyz$ ที่ $\|y\| > 0$ และ $\|xy\| \leq p$
4	**เรา**	ค่า i ที่ทำให้ $xy^iz \notin A$

- เราชนะ (ภาษาไม่ regular) ถ้ามีวิธีชนะ**ทุก**การแบ่งที่ฝ่ายตรงข้ามเลือกได้

โครงบทพิสูจน์

1. สมมติว่า A regular ให้ p เป็น pumping length

2. เลือก $s = \ldots$ (เขียนในรูปของ p) ตรวจว่า $s \in A$ และ $|s| \geq p$
3. ให้ $s = xyz$ เป็นการแบ่งใด ๆ ที่ $|y| > 0$ และ $|xy| \leq p$
4. ใช้ $|xy| \leq p$ บอกว่า y หน้าตาเป็นอย่างไร (เช่น $y = 0^k$, $k \geq 1$)
5. เลือก i แล้วแสดงว่า $xy^iz \notin A$
6. ขัดกับ pumping lemma ดังนั้น A ไม่ regular

7.4 ตัวอย่าง

ตัวอย่าง : $B = \{0^n1^n : n \geq 0\}$

- เลือก $s = 0^p1^p$ ($s \in B$, $|s| = 2p \geq p$)
- เพราะ $|xy| \leq p$ xy อยู่ใน 0 ช่วงแรกทั้งหมด ดังนั้น $y = 0^k$ โดย $k \geq 1$
- เลือก $i = 2$:
  - $xy^2z = 0^{p+k}1^p$
  - มี 0 มากกว่า 1
  - จึง $\notin B$

ตัวอย่าง : $F = \{ww : w \in \{0,1\}^*\}$

- เลือก $s = 0^p10^p1$ ($w = 0^p1$)
- เพราะ $|xy| \leq p$ ได้ $y = 0^k$ โดย $k \geq 1$
- เลือก $i = 2$: $xy^2z = 0^{p+k}10^p1$
- สตริงนี้มี 1 สองตัว ถ้าเป็นรูป ww แต่ละครึ่งต้องลงท้ายด้วย 1 คือ $0^{p+k}1$ กับ 0^p1 ซึ่งไม่เท่ากัน จึง $\notin F$
- หมายเหตุ : ถ้าเลือก $s = 0^p0^p$ จะพิสูจน์ไม่ได้ เพราะ pump แล้วยังอยู่ใน F ได้

ตัวอย่าง : $D = \{1^{n^2} : n \geq 0\}$

- เลือก $s = 1^{p^2}$
- $1 \leq |y| \leq p$ เลือก $i = 2$: $|xy^2z| = p^2 + |y|$
- $p^2 < p^2 + |y| \leq p^2 + p < (p+1)^2$ ความยาวอยู่ระหว่างกำลังสองสมบูรณ์สองตัวที่ติดกัน จึง $\notin D$

ตัวอย่าง : $E = \{0^i1^j : i > j\}$ (ต้อง pump ลง)

- เลือก $s = 0^{p+1}1^p$
- $y = 0^k$ โดย $k \geq 1$ pump ขึ้นไม่ช่วย เพราะ 0 ยิ่งมากยิ่งอยู่ใน E
- เลือก $i = 0$:
  - $xz = 0^{p+1-k}1^p$
  - และ $p + 1 - k \leq p$
  - จึง $\notin E$

ตัวอย่าง : $C = \{w : w$ มี 0 และ 1 จำนวนเท่ากัน$\}$ (ใช้ closure)

- สมมติว่า C regular 0^*1^* ก็ regular
- regular ปิดภายใต้ intersection ดังนั้น $C \cap 0^*1^* = \{0^n1^n\}$ ต้อง regular
- แต่ $\{0^n1^n\}$ ไม่ regular ขัดแย้ง
- (ใช้ pumping lemma ตรง ๆ ด้วย $s = 0^p1^p$ ก็ได้เช่นกัน)

ระวัง — จุดที่ผิดบ่อย

- **เลือก p เป็นตัวเลข** เช่น $p = 5$ ผิด : p ต้องเป็นตัวแปร
- **เลือกการแบ่งเอง** เช่น "ให้ $x = \varepsilon, y = 0$" ผิด : ต้องครอบคลุมทุกการแบ่งที่เป็นไปได้
- **ไม่ใช้เงื่อนไข** $|xy| \leq p$ ทำให้ต้องแบ่งกรณีเยอะโดยไม่จำเป็น
- **เลือก s ที่ pump ได้** เช่น $(01)^p$ สำหรับภาษา "0 กับ 1 เท่ากัน" ($y = 01$ pump ได้ตลอด)
- **ใช้ pumping lemma พิสูจน์ว่า regular** ผิด : pumping lemma เป็นเงื่อนไขจำเป็นเท่านั้น มีภาษาที่ไม่ regular แต่ pump ได้ จะพิสูจน์ว่า regular ต้องสร้าง DFA / NFA / regular expression

★ Claude สอนเพิ่ม : ลางสังหรณ์ : ภาษาแบบไหนไม่ regular

- ต้อง**นับไม่จำกัด**แล้วเทียบกัน : 0^n1^n, จำนวน 0 = จำนวน 1, 0^i1^j ที่ $i > j$
- ต้อง**จำสตริงยาวไม่จำกัด** : ww, ww^R, palindrome
- ความยาวโต**ไม่เป็นเชิงเส้น** : 1^{n^2}, 1^{2^n}, 1^p ที่ p เป็นจำนวนเฉพาะ
- ระวังกับดัก : $\{w : \#(01) = \#(10)\}$ ดูเหมือนต้องนับ แต่ **regular** (เท่ากับ "ตัวแรกและตัวท้ายเหมือนกัน หรือเป็น ε")

บทที่ 8

เสริม : DFA Minimization และ Myhill–Nerode

Objectives — อ่านจบแล้วต้องทำได้

- บอกได้ว่า state สองตัว distinguishable หรือ equivalent
- ลดรูป DFA ด้วย table-filling algorithm ได้
- ใช้ Myhill–Nerode พิสูจน์ว่าไม่ regular หรือหาจำนวน state ต่ำสุดได้

8.1 State ที่แยกกันได้

นิยาม

- state p, q **distinguishable** ถ้ามีสตริง z ที่ $\hat{\delta}(p,z)$ กับ $\hat{\delta}(q,z)$ ตัวหนึ่งเป็น accept อีกตัวไม่เป็น
- ถ้าไม่มี z แบบนั้นเลย p, q **equivalent** รวมเป็น state เดียวได้โดยภาษาไม่เปลี่ยน

8.2 Table-filling algorithm

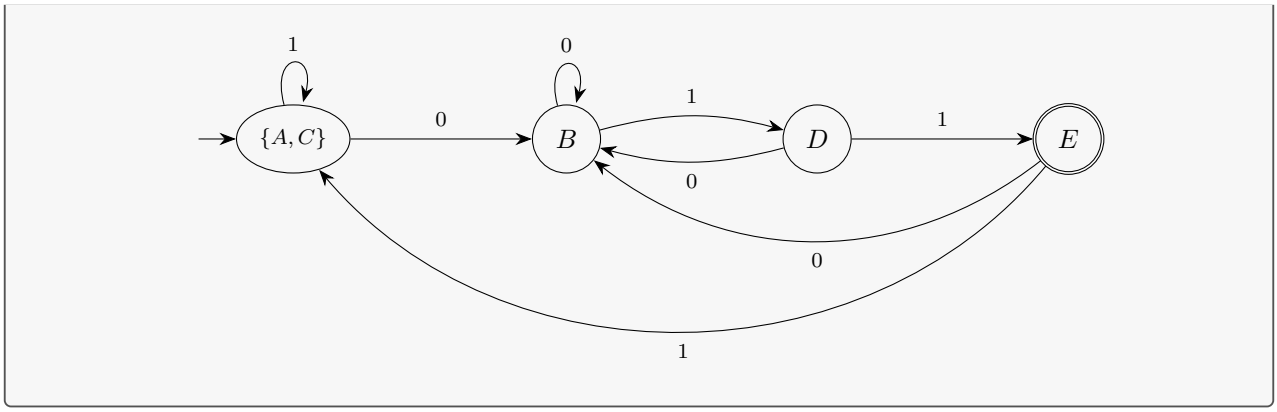
1. ลบ state ที่ไปไม่ถึงจาก start state
2. ทำตารางของทุกคู่ $\{p,q\}$
3. **รอบ 0** : mark ทุกคู่ที่ตัวหนึ่งเป็น accept อีกตัวไม่เป็น (แยกได้ด้วย $z=\varepsilon$)
4. **รอบถัดไป** : คู่ $\{p,q\}$ ที่ยังไม่ mark ถ้ามีสัญลักษณ์ a ที่คู่ $\{\delta(p,a),\delta(q,a)\}$ ถูก mark แล้ว ให้ mark $\{p,q\}$
5. ทำซ้ำจนไม่มีการ mark เพิ่ม
6. คู่ที่**ไม่ถูก mark** คือ equivalent รวมเป็น state เดียว

ตัวอย่าง : ลดรูป DFA 5 state (ภาษา : ลงท้ายด้วย 011)

δ	0	1
$\to A$	B	C
B	B	D
C	B	C
D	B	E
$*E$	B	C

	A	B	C	D
B	2			
C	–	2		
D	1	1	1	
E	0	0	0	0

- ตัวเลขในตารางขวา = รอบที่คู่นั้นถูก mark “–” = ไม่ถูก mark
- **รอบ 0** : E เป็น accept ตัวเดียว mark ทุกคู่ที่มี E
- **รอบ 1** : อ่าน 1 $\{A,D\}\to\{C,E\}$, $\{B,D\}\to\{D,E\}$, $\{C,D\}\to\{C,E\}$ ซึ่ง mark แล้วทั้งหมด
- **รอบ 2** : อ่าน 1 $\{A,B\}\to\{C,D\}$, $\{B,C\}\to\{D,C\}$ ซึ่ง mark แล้วในรอบ 1
- **รอบ 3** : $\{A,C\}$ อ่าน 0 ไป $\{B,B\}$
  - อ่าน 1 ไป $\{C,C\}$
  - ไม่มีทาง mark
  - จบ
- ผล : $A\equiv C$ รวมเป็น state เดียว เหลือ 4 state



8.3 Myhill–Nerode theorem

Myhill–Nerode

- สตริง x, y **distinguishable โดย** L ถ้ามี z ที่ xz กับ yz ตัวหนึ่งอยู่ใน L อีกตัวไม่อยู่
- เขียน $x \equiv_L y$ ถ้าแยกกันไม่ได้ จำนวน equivalence class เรียกว่า **index** ของ L
- L regular $\iff$ index ของ L มีค่าจำกัด
- index ของ L = จำนวน state ของ **minimal DFA** ของ L (และ minimal DFA มีแบบเดียว)

★ Claude สอนเพิ่ม : วิธีใช้

- **พิสูจน์ไม่ regular :** หาเซตสตริง**อนันต์**ที่ทุกคู่ distinguishable
  เช่น $L = \{0^n 1^n\}$: เซต $\{0^i : i \geq 0\}$ ถ้า $i \neq j$ ใช้ $z = 1^i$ ได้ $0^i 1^i \in L$ แต่ $0^j 1^i \notin L$
- **หาขอบล่างของจำนวน state :** หาสตริง k ตัวที่ทุกคู่ distinguishable DFA ต้องมีอย่างน้อย k state
  เช่น “ตัวที่ 3 จากท้ายเป็น 1” : สตริงยาว 3 ทั้ง 8 ตัวแยกกันได้หมด DFA ต้องมีอย่างน้อย $2^3 = 8$ state
- ข้อดีเหนือ pumping lemma : เป็นเงื่อนไข “ก็ต่อเมื่อ” ใช้ได้กับทุกภาษา

บทที่ 9

ภาพรวม : ลำดับชั้นของภาษา

★ Claude สอนเพิ่ม : จากภาค 1 สู่ภาค 2

- ภาค 1 จบที่ข้อจำกัดของ FA : ภาษาอย่าง $\{0^n1^n\}$ ไม่ regular เพราะ state มีจำนวนจำกัด นับไม่ได้
- ภาค 2 เพิ่มความจำให้เครื่องทีละขั้น : stack (PDA) แล้วเทป (TM)
- สุดท้ายจะเห็นว่าแม้แต่ TM ก็มีปัญหาที่ตัดสินไม่ได้ (A_{TM})

9.1 ลำดับชั้น

จำให้ได้

$$\text{Regular} \subsetneq \text{Context-free} \subsetneq \text{Decidable} \subsetneq \text{Turing-recognizable}$$

ชั้นของภาษา	เครื่อง	ตัวสร้าง	ตัวอย่างที่ “เกิน” ชั้นนี้
Regular	DFA / NFA	Regular expression	$\{0^n1^n\}$
Context-free	PDA	CFG	$\{0^n1^n2^n\}$, $\{ww\}$
Decidable	TM ที่หยุดเสมอ (decider)	–	A_{TM}
Recognizable	TM	–	$\overline{A_{TM}}$

9.2 ความจำของแต่ละเครื่อง

- **DFA/NFA** : มีแค่ state จำได้จำกัด นับไม่ได้
- **PDA** : state + **stack 1 อัน** นับ/จับคู่ได้ “ทีละคู่” แบบซ้อนกัน (เข้าหลังออกก่อน)
- **TM** : state + **เทปอ่านเขียนได้ เดินซ้ายขวาได้** ทำได้ทุกอย่างที่อัลกอริทึมทำได้

บทที่ 10

Context-Free Grammar (CFG)

Objectives — อ่านจบแล้วต้องทำได้

- อ่านและเขียน CFG ในรูป 4-tuple ได้
- เขียน derivation และ parse tree ของสตริงได้
- ออกแบบ CFG จากสูตรสำเร็จ และแตกภาษาเป็นชิ้นได้
- พิสูจน์ว่า CFG ที่ออกแบบถูกต้องได้

10.1 นิยาม

นิยาม

$G = (V, \Sigma, R, S)$

- V = เซตตัวแปร (variable / nonterminal)
- Σ = เซตตัวอักษร (terminal) โดย $V \cap \Sigma = \emptyset$
- R = เซตกฎ รูป $A \to w$ โดย $A \in V$ และ $w \in (V \cup \Sigma)^*$
- $S \in V$ = ตัวแปรเริ่มต้น

ภาษาของ G : $L(G) = \{w \in \Sigma^* : S \Rightarrow^* w\}$

10.2 คำศัพท์

- $u \Rightarrow v$: ใช้กฎ 1 ครั้ง เปลี่ยน u เป็น v
- $u \Rightarrow^* v$: ใช้กฎ 0 ครั้งขึ้นไป
- **derivation** : ลำดับการแทนจาก S จนเหลือแต่ terminal
- **ambiguous** : มีสตริงที่มี parse tree (หรือ leftmost derivation) มากกว่า 1 แบบ

10.3 สูตรสำเร็จในการออกแบบ CFG

ต้องการ	กฎ	หมายเหตุ
$a^n b^n \quad (n \geq 0)$	$S \to aSb \mid \varepsilon$	จับคู่นอกเข้าใน
$a^n b^n \quad (n \geq 1)$	$S \to aSb \mid ab$	เปลี่ยนฐาน
$a^n b^{2n}$	$S \to aSbb \mid \varepsilon$	อัตราส่วนคงที่
$a^i b^j, \quad i \geq j$	$S \to aSb \mid aS \mid \varepsilon$	ฝั่งหนึ่งเกินได้
palindrome	$S \to 0S0 \mid 1S1 \mid 0 \mid 1 \mid \varepsilon$	
ww^R	$S \to 0S0 \mid 1S1 \mid \varepsilon$	
จำนวน 0 = จำนวน 1	$S \to 0S1S \mid 1S0S \mid \varepsilon$	ลำดับใดก็ได้
วงเล็บสมดุล	$S \to (S)S \mid \varepsilon$	
$L_1 \cup L_2$	$S \to S_1 \mid S_2$	
$L_1 L_2$	$S \to S_1 S_2$	
L_1^*	$S \to S_1 S \mid \varepsilon$	

10.4 เทคนิค : แตกภาษาเป็นชิ้น

- ถ้าเงื่อนไขเป็น “ผลบวก” ให้แยกเป็นการต่อกัน เช่น

$$0^n1^{n+k}2^k \;=\; (0^n1^n)(1^k2^k) \quad \Rightarrow \quad S \to XY,\; X \to 0X1 \mid 01,\; Y \to 1Y2 \mid 12$$

- ถ้าเงื่อนไขเป็น “หรือ” ให้ใช้ union เช่น $\{a^ib^jc^k : i = j \text{ หรือ } j = k\}$
- CFG จับคู่ได้เฉพาะแบบ **ซ้อนกัน** หรือ **ต่อกัน** จับคู่แบบ**ไขว้กัน**ไม่ได้ (เช่น $a^nb^mc^nd^m$)

10.5 วิธีพิสูจน์ว่า CFG ถูกต้อง

1. $L(G) \subseteq L$: แสดงว่ากฎทุกข้อ “รักษาสมบัติ” ของภาษา (induction บนจำนวนขั้น derive)
2. $L \subseteq L(G)$: induction บนความยาวสตริง แตกสตริงเป็นชิ้นที่สั้นลงและยังอยู่ใน L

10.6 เทคนิค : แปลง DFA เป็น CFG

- ภาษา regular ทุกภาษาเป็น context-free สร้าง CFG จาก DFA ได้ตรง ๆ
  1. สร้าง variable R_i ต่อ state q_i
  2. ใส่กฎ $R_i \to aR_j$ ถ้า $\delta(q_i, a) = q_j$
  3. ใส่กฎ $R_i \to \varepsilon$ ถ้า q_i เป็น accept state
  4. start variable คือ R_0 (ของ start state q_0)

10.7 Ambiguity

- grammar อาจสร้าง string เดียวกันได้หลายแบบ : หลาย parse tree : หลายความหมาย ไม่ดีสำหรับภาษาโปรแกรม
- ตัวอย่าง G_5 : $\langle\text{EXPR}\rangle \to \langle\text{EXPR}\rangle + \langle\text{EXPR}\rangle \mid \langle\text{EXPR}\rangle \times \langle\text{EXPR}\rangle \mid (\langle\text{EXPR}\rangle) \mid a$
  สร้าง $a + a \times a$ ได้ 2 parse tree (G_5 ไม่บังคับลำดับความสำคัญ ต่างจาก G_4 ที่ไม่ ambiguous)
- derivation ต่างกันแต่ parse tree เดียวกันไม่นับว่า ambiguous จึงกำหนดลำดับการแทน :
- Leftmost derivation : ทุกขั้นแทน variable **ตัวซ้ายสุด**

Definition 2.7

w ถูก **derived ambiguously** ใน G ถ้ามี leftmost derivation ต่างกัน 2 แบบขึ้นไป G เป็น **ambiguous** ถ้าสร้างบาง string แบบ ambiguous

- บางครั้งหา grammar ที่ไม่ ambiguous สำหรับภาษาเดียวกันได้ แต่บางภาษาสร้างได้ด้วย grammar ที่ ambiguous เท่านั้น เรียก **inherently ambiguous** เช่น $\{a^ib^jc^k \mid i = j \text{ หรือ } j = k\}$

10.8 Chomsky normal form

Definition 2.8

CFG อยู่ใน **Chomsky normal form (CNF)** ถ้าทุกกฎอยู่ในรูป

$$A \to BC \qquad \text{หรือ} \qquad A \to a$$

a เป็น terminal ; A, B, C เป็น variable ; B, C ห้ามเป็น start variable อนุญาต $S \to \varepsilon$ เมื่อ S เป็น start variable

Theorem 2.9

ทุก context-free language สร้างได้ด้วย CFG ใน Chomsky normal form

ขั้นตอนแปลง (จากบทพิสูจน์)

1. **เพิ่ม start variable ใหม่** $S_0 \to S$ (รับประกันว่า start variable ไม่อยู่ฝั่งขวา)
2. **ตัด** ε**-rule** $A \to \varepsilon$ (A ไม่ใช่ start) : ทุกกฎที่มี A ฝั่งขวา เพิ่มกฎที่ลบ A ออก (ทุกชุดของตำแหน่ง)
   เช่น $R \to uAvAw$ เพิ่ม $R \to uvAw \mid uAvw \mid uvw$ ถ้ามี $R \to A$ เพิ่ม $R \to \varepsilon$ (เว้นแต่เคยตัดไปแล้ว)
3. **ตัด unit rule** $A \to B$: ทุกกฎ $B \to u$ เพิ่ม $A \to u$ (เว้นแต่เป็น unit rule ที่ตัดไปแล้ว)
4. **แปลงกฎที่เหลือ** : $A \to u_1 u_2 \cdots u_k$, $k \geq 3$ แทนด้วย $A \to u_1 A_1$, $A_1 \to u_2 A_2$, ..., $A_{k-2} \to u_{k-1} u_k$
   terminal u_i ในกฎที่ยาว ≥ 2 แทนด้วย variable ใหม่ U_i พร้อมกฎ $U_i \to u_i$

ตัวอย่าง : Example 2.10 : แปลง G_6 เป็น CNF

- G_6 : $S \to ASA \mid aB$
- $A \to B \mid S$
- $B \to b \mid \varepsilon$
- ขั้น 1 : เพิ่ม $S_0 \to S$
- ขั้น 2 : ตัด $B \to \varepsilon$ ได้ $S \to ASA \mid aB \mid a$, $A \to B \mid S \mid \varepsilon$ แล้วตัด $A \to \varepsilon$ ได้ $S \to ASA \mid aB \mid a \mid SA \mid AS \mid S$
- ขั้น 3 : ตัด $S \to S$,
  - $S_0 \to S$,
  - $A \to B$,
  - $A \to S$
- ขั้น 4 : แตกกฎยาว ผลลัพธ์

$$S_0 \to AA_1 \mid UB \mid a \mid SA \mid AS \qquad S \to AA_1 \mid UB \mid a \mid SA \mid AS$$
$$A \to b \mid AA_1 \mid UB \mid a \mid SA \mid AS \qquad A_1 \to SA \qquad U \to a \qquad B \to b$$

★ Claude สอนเพิ่ม : ทำไม CNF สำคัญ

- ใน CNF ทุก derivation ของ string ยาว $n \geq 1$ ใช้ $2n-1$ **ขั้นพอดี** ใช้พิสูจน์ว่า A_{CFG} decidable (Theorem 4.7)

บทที่ 11

Pushdown Automata (PDA)

Objectives — อ่านจบแล้วต้องทำได้

- อ่านและเขียน PDA ในรูป 6-tuple ได้
- อ่านป้าย $a,\ b \to c$ บน diagram ได้
- ใช้ stack นับและจับคู่สัญลักษณ์ได้
- ออกแบบ PDA สำหรับภาษาแบบ $0^n 1^n$ ได้

11.1 นิยาม

นิยาม

$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$

- Q = เซต state $\quad \Sigma$ = input alphabet $\quad \Gamma$ = stack alphabet
- $q_0 \in Q$ = start state $\quad F \subseteq Q$ = เซต accept state
- transition function (nondeterministic) :

$$\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \ \to\ \mathcal{P}(Q \times \Gamma_\varepsilon) \qquad \text{โดย } \Sigma_\varepsilon = \Sigma \cup \{\varepsilon\},\ \Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$$

11.2 วิธีอ่านป้ายบนเส้น : $a,\ b \to c$

ป้าย	ความหมาย	ชื่อเรียก
$a,\ \varepsilon \to c$	อ่าน a push c	push
$a,\ b \to \varepsilon$	อ่าน a pop b (บนสุดต้องเป็น b)	pop
$a,\ b \to c$	อ่าน a pop b แล้ว push c	replace
$\varepsilon,\ \varepsilon \to c$	ไม่อ่าน input push c	ε-move
$a,\ \varepsilon \to \varepsilon$	อ่าน a ไม่แตะ stack	

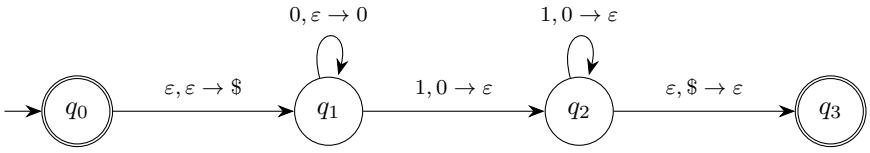
11.3 เงื่อนไข accept

- อ่าน input **หมด** และอยู่ใน accept state
- มี **อย่างน้อยหนึ่ง** เส้นทางการคำนวณที่ accept ก็พอ (เพราะ nondeterministic)
- PDA มองไม่เห็นว่า stack ว่างหรือไม่ จึงต้องใช้เทคนิค \$

11.4 เทคนิคที่ใช้ประจำ

- **ตัวบอกก้น stack :** เริ่มด้วย $\varepsilon, \varepsilon \to \$$ จบด้วย $\varepsilon, \$ \to \varepsilon$ ถ้า pop \$ ได้แปลว่า stack ว่างพอดี
- **นับ :** push ตอนอ่านกลุ่มแรก pop ตอนอ่านกลุ่มที่ต้องเท่ากัน
- **บังคับอย่างน้อย 1 ตัว :** ให้เส้นที่ “ข้าม state” เป็นเส้นที่อ่านตัวอักษรนั้นเอง
- **เดา :** ใช้ ε-move เพื่อเดาจุดกึ่งกลางหรือจุดเปลี่ยนช่วง

11.5 ตัวอย่าง : PDA สำหรับ $\{0^n1^n : n \geq 0\}$



- q_0 เป็น accept state ด้วย เพื่อรับ ε (กรณี $n = 0$)
- configuration ของ PDA เขียนเป็น (state, input ที่เหลือ, stack) เช่น

$$(q_0, 0011, \varepsilon) \vdash (q_1, 0011, \$) \vdash (q_1, 011, 0\$) \vdash (q_1, 11, 00\$) \vdash (q_2, 1, 0\$) \vdash (q_2, \varepsilon, \$) \vdash (q_3, \varepsilon, \varepsilon)$$

ทฤษฎีบทหลัก

ภาษา L เป็น context-free $\iff$ มี CFG สร้าง L $\iff$ มี PDA รับ L

บทที่ 12

แปลง CFG → PDA

Objectives — อ่านจบแล้วต้องทำได้

- สร้าง PDA 3 state จาก CFG ใดก็ได้
- จัดการกฎที่ต้อง push หลายตัวได้
- นับจำนวน state และ transition ที่ได้

12.1 แนวคิด

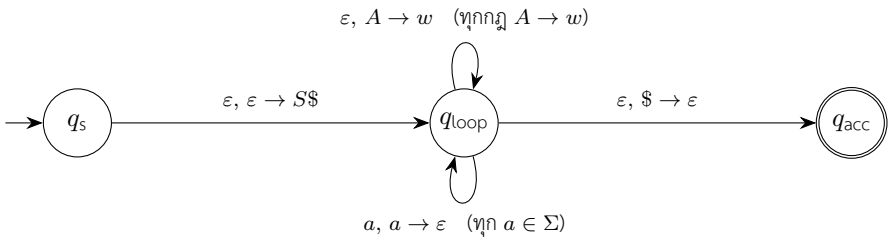
- PDA จำลอง **leftmost derivation** ไว้บน stack
- บนสุดเป็นตัวแปร : เดาว่าใช้กฎไหน แล้วแทน
- บนสุดเป็น terminal : เทียบกับ input แล้ว pop

12.2 สูตร

สูตร

ให้ $G = (V, \Sigma, R, S)$ สร้าง PDA ที่มี 3 state หลัก $q_s,\ q_{\text{loop}},\ q_{\text{acc}}$ และ $\Gamma = V \cup \Sigma \cup \{\$\}$

1. $q_s \to q_{\text{loop}}$: $\varepsilon,\ \varepsilon \to S\$$
2. loop ที่ q_{loop} สำหรับ**ทุกกฎ** $A \to w$ ใน R : $\varepsilon,\ A \to w$
3. loop ที่ q_{loop} สำหรับ**ทุก terminal** $a \in \Sigma$: $a,\ a \to \varepsilon$
4. $q_{\text{loop}} \to q_{\text{acc}}$: $\varepsilon,\ \$ \to \varepsilon$



12.3 การ push หลายตัว

- นิยาม PDA push ได้ครั้งละ 1 ตัว ป้าย $\varepsilon, A \to w$ ที่ $|w| > 1$ จึงเป็นตัวย่อ
- ของจริง : ใช้ state เสริม $|w| - 1$ ตัว push **จากขวาไปซ้าย** เพื่อให้ตัวซ้ายสุดของ w อยู่บนสุด
- ตัวอย่าง $w = 0S1S$:

$$q_{\text{loop}} \xrightarrow{\varepsilon, A \to S} r_1 \xrightarrow{\varepsilon, \varepsilon \to 1} r_2 \xrightarrow{\varepsilon, \varepsilon \to S} r_3 \xrightarrow{\varepsilon, \varepsilon \to 0} q_{\text{loop}}$$

12.4 นับจำนวน

สูตรนับ

- จำนวน loop ที่ $q_{\text{loop}} = |R| + |\Sigma|$
- จำนวน state ทั้งหมด (แบบขยายเต็ม) $= 3 + \sum_{A \to w \in R} \max(|w| - 1, 0) + 1$ (ตัว $+1$ มาจากการ push $S\$$)

12.5 ขั้นตอนทำโจทย์

1. เขียนกฎของ CFG ออกมาทีละข้อ (แยก | ออกเป็นกฎเดี่ยว)
2. วาด 3 state และเส้นหลัก 2 เส้น
3. ใส่ loop ตัวแปร 1 เส้นต่อ 1 กฎ
4. ใส่ loop terminal 1 เส้นต่อ 1 ตัวอักษร
5. (ถ้าโจทย์ต้องการ) ขยาย push หลายตัวเป็น state เสริม
6. ลอง trace สตริงสั้น ๆ 1 ตัวเพื่อตรวจ

บทที่ 13

แปลง PDA → CFG

Objectives — อ่านจบแล้วต้องทำได้

- ปรับ PDA ให้ตรง 3 เงื่อนไขก่อนแปลงได้
- อธิบายความหมายของตัวแปร A_{pq} ได้
- เขียนกฎทั้ง 3 แบบ และตัดกฎที่ไม่ได้ใช้ออกได้

13.1 ขั้นที่ 0 : ปรับ PDA ให้ตรง 3 เงื่อนไข

เงื่อนไข	ถ้ายังไม่ตรง แก้โดย
1. มี accept state เดียว	เพิ่ม state ใหม่ ลาก $\varepsilon, \varepsilon \to \varepsilon$ จาก accept state เดิมทุกตัว
2. stack ว่างก่อน accept	เพิ่ม loop pop ทุกสัญลักษณ์ก่อนเข้า accept state
3. ทุกเส้น push อย่างเดียว หรือ pop อย่างเดียว	เส้น replace $a, b \to c$: แตกเป็น $a, b \to \varepsilon$ แล้ว $\varepsilon, \varepsilon \to c$ เส้นที่ไม่แตะ stack $a, \varepsilon \to \varepsilon$: แตกเป็น $a, \varepsilon \to z$ แล้ว $\varepsilon, z \to \varepsilon$

13.2 ความหมายของตัวแปร

- A_{pq} สร้างสตริง x ทุกตัวที่พา PDA จาก state p ด้วย stack ว่าง ไปถึง state q ด้วย stack ว่าง
- ระหว่างทาง stack จะสูงขึ้นแล้วกลับมาเท่าเดิม มี 2 แบบ :
  - ตัวที่ push ตอนแรก ถูก pop ตอนท้ายสุด ⇒ กฎแบบที่ 1
  - stack ว่างกลางทางที่ state r ⇒ กฎแบบที่ 2

13.3 สูตร

สูตร

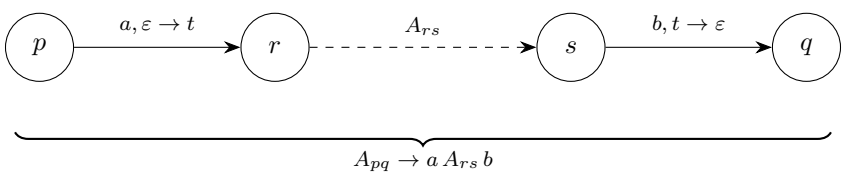
$V = \{A_{pq} : p, q \in Q\}$ ตัวแปรเริ่มต้น $= A_{q_0\, q_{\text{acc}}}$

1. ถ้า $(r, t) \in \delta(p, a, \varepsilon)$ และ $(q, \varepsilon) \in \delta(s, b, t)$ (push t แล้ว pop t ตัวเดียวกัน) :

$$A_{pq} \;\to\; a\, A_{rs}\, b$$

2. ทุก $p, q, r \in Q$: $A_{pq} \to A_{pr}\, A_{rq}$
3. ทุก $p \in Q$: $A_{pp} \to \varepsilon$

13.4 วิธีจำกฎแบบที่ 1



- ดัชนีของตัวแปรด้านขวา = **ปลายทางของเส้น push** และ **ต้นทางของเส้น pop**
- ดัชนีของตัวแปรด้านซ้าย = **ต้นทางของเส้น push** และ **ปลายทางของเส้น pop**

สูตรนับ

- จำนวนตัวแปร $= |Q|^2$
- กฎแบบที่ 2 $= |Q|^3$ ข้อ
- กฎแบบที่ 3 $= |Q|$ ข้อ
- กฎแบบที่ 1 $= \sum_{t \in \Gamma}$(จำนวนเส้น push t) $\times$ (จำนวนเส้น pop t)

13.5 ขั้นตอนทำโจทย์

1. ตรวจ 3 เงื่อนไข แก้ PDA ถ้าจำเป็น
2. ทำตาราง : แยกเส้น push กับเส้น pop ตามสัญลักษณ์ stack
3. จับคู่ push–pop ของสัญลักษณ์เดียวกัน**ทุกคู่** เขียนกฎแบบที่ 1
4. เขียนกฎแบบที่ 2 และ 3 เป็นรูปทั่วไป (ไม่ต้องไล่ครบทุกข้อ)
5. (แนะนำ) ตัดตัวแปรที่สร้างสตริงไม่ได้หรือไปไม่ถึง แล้วเปลี่ยนชื่อให้อ่านง่าย

บทที่ 14

Pumping Lemma สำหรับ Context-Free Language

Objectives — อ่านจบแล้วต้องทำได้

- เขียนตัวบท pumping lemma ของ CFL ได้ครบ 3 เงื่อนไข
- บอกความต่างจาก pumping lemma ของ regular ได้
- พิสูจน์ว่าภาษาไม่เป็น CFL โดยแบ่งกรณีของ vxy ได้
- เลือกสตริง s ที่ทำให้พิสูจน์ง่ายได้

14.1 ตัวบท

ทฤษฎีบท

ถ้า L เป็น context-free จะมีจำนวน p (pumping length) ที่ทำให้ ทุก $s \in L$ ที่ $|s| \geq p$ แบ่งได้เป็น $s = uvxyz$ โดย

1. $uv^ixy^iz \in L$ ทุก $i \geq 0$
2. $|vy| > 0$ (v กับ y ไม่ว่างพร้อมกัน)
3. $|vxy| \leq p$

14.2 เทียบกับ pumping lemma ของ regular

	Regular	Context-free
การแบ่ง	$s = xyz$	$s = uvxyz$
ส่วนที่ pump	y (1 ชิ้น)	v และ y (2 ชิ้น พร้อมกัน)
ไม่ว่าง	$\|y\| > 0$	$\|vy\| > 0$
ขอบเขต	$\|xy\| \leq p$ (อยู่ต้นสตริง)	$\|vxy\| \leq p$ (อยู่ตรงไหนก็ได้)

14.3 โครงบทพิสูจน์ (ใช้ได้ทุกข้อ)

1. สมมติว่า L เป็น context-free ให้ p เป็น pumping length
2. **เราเลือก** $s \in L$ ที่ $|s| \geq p$ (ต้องเขียนในรูปของ p)
3. ให้ $s = uvxyz$ เป็นการแบ่ง**ใด ๆ** ที่ตรงเงื่อนไข 2 และ 3 (เราเลือกการแบ่งเองไม่ได้)
4. แบ่งกรณีตามตำแหน่งของ v และ y ให้ครบทุกกรณี
5. แต่ละกรณี **เราเลือก** i (มักใช้ $i = 2$ หรือ $i = 0$) แล้วแสดงว่า $uv^ixy^iz \notin L$
6. ขัดกับเงื่อนไข 1 สรุปว่า L ไม่เป็น context-free

14.4 การแบ่งกรณีมาตรฐาน

- **กรณี A :** v หรือ y คร่อมรอยต่อ (มีสัญลักษณ์มากกว่า 1 ชนิด หรือมีตัวคั่นเช่น #)
  - pump แล้ว **รูปแบบ/ลำดับ** ของสตริงเสีย
- **กรณี B :** v และ y แต่ละตัวอยู่ในบล็อกเดียว
  - v, y แตะได้อย่างมาก 2 บล็อก pump แล้ว**จำนวน**ไม่สมดุล
  - ถ้าต้องการให้แตะได้แค่ “2 บล็อกที่ติดกัน” ให้ใช้เงื่อนไข $|vxy| \leq p$

14.5 เคล็ดลับการเลือก s

- ให้ทุกบล็อกยาวอย่างน้อย p เพื่อให้ vxy คร่อมได้ไม่เกิน 2 บล็อก
- เลือกสตริงที่ “ตึง” ที่สุด คือเปลี่ยนนิดเดียวก็หลุดจากภาษา
- ระวัง : สตริงบางตัว pump ได้ เช่น $\{ww\}$ ถ้าเลือก 0^p10^p1 จะพิสูจน์ไม่สำเร็จ ต้องใช้ $0^p1^p0^p1^p$

ภาษาที่ไม่เป็น context-free (ควรจำ)

$\{a^nb^nc^n\}$ $\{ww : w \in \{0,1\}^*\}$ $\{0^n1^n0^n1^n\}$ $\{a^ib^jc^k : i \le j \le k\}$ $\{a^{n^2}\}$ $\{a^{2^n}\}$

Closure ของ CFL

- **ปิด** ภายใต้ : union, concatenation, star, reverse, intersection กับ regular language
- **ไม่ปิด** ภายใต้ : intersection, complement

บทที่ 15

Turing Machine (TM)

Objectives — อ่านจบแล้วต้องทำได้

- อ่านและเขียน TM ในรูป 7-tuple ได้
- เขียนลำดับ configuration ของการคำนวณได้
- ออกแบบ TM ด้วยเทคนิค mark แล้วเดินกลับได้

15.1 นิยาม

นิยาม

$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$

- Σ = input alphabet (ไม่มี $\sqcup$)
- Γ = tape alphabet โดย $\Sigma \subseteq \Gamma$ และ $\sqcup \in \Gamma$
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\text{L}, \text{R}\}$ (deterministic)
- $q_{\text{acc}} \neq q_{\text{rej}}$ เข้าแล้ว**หยุดทันที**

15.2 สิ่งที่ต่างจาก DFA / PDA

- เขียนเทปได้ และเดินได้ทั้งซ้ายและขวา
- เทปยาวไม่จำกัดทางขวา ช่องที่ยังไม่ใช้เป็น $\sqcup$
- หยุดเมื่อเข้า q_{acc} หรือ q_{rej} เท่านั้น ไม่ต้องอ่าน input ให้หมด
- ผลลัพธ์มี 3 แบบ : **accept**, **reject**, **loop** (ไม่หยุด)
- อยู่ช่องซ้ายสุดแล้วสั่งไปซ้าย : หัวอ่านอยู่ที่เดิม

15.3 Configuration

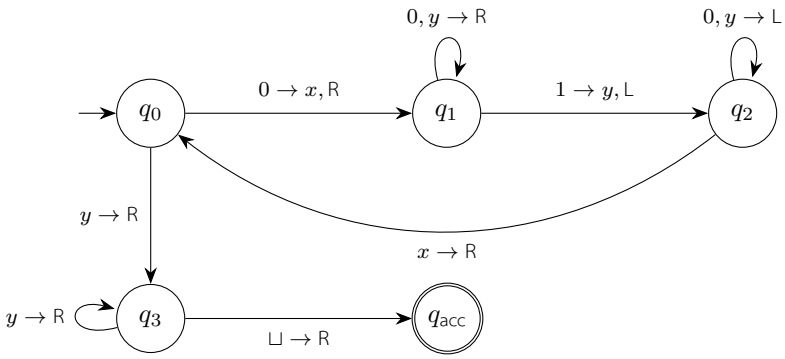
สูตร

เขียน $u\, q\, v$ หมายถึง เทปคือ uv state คือ q หัวอ่านอยู่ที่ตัวแรกของ v
กฎ yields (ให้ $a, b, c \in \Gamma$ และ $u, v \in \Gamma^*$) :

- ถ้า $\delta(q_i, b) = (q_j, c, \text{R})$:
- $u\, a\, q_i\, b\, v \;\vdash\; u\, a\, c\, q_j\, v$
- ถ้า $\delta(q_i, b) = (q_j, c, \text{L})$:
- $u\, a\, q_i\, b\, v \;\vdash\; u\, q_j\, a\, c\, v$
- ขอบซ้าย, ไปซ้าย :
- $q_i\, b\, v \;\vdash\; q_j\, c\, v$
- ขอบขวา : ถือว่า $u\, a\, q_i$ เหมือน $u\, a\, q_i\, \sqcup$

start configuration : $q_0\, w$ accepting : มี q_{acc} rejecting : มี q_{rej}

15.4 ตัวอย่าง : TM ที่ decide $\{0^n 1^n : n \geq 0\}$



- ป้าย $a \to b, \mathsf{R}$: อ่าน a เขียน b ไปขวา ป้าย $a \to \mathsf{R}$: อ่าน a ไม่แก้ ไปขวา
- เส้นที่ไม่ได้วาด เข้า q_{rej}
- ตัวอย่าง : $q_0\,01 \vdash x\,q_1\,1 \vdash q_2\,xy \vdash x\,q_0\,y \vdash xy\,q_3\,\sqcup \vdash xy\,\sqcup\,q_{\text{acc}}$

15.5 เทคนิคออกแบบ TM

- **ขีดฆ่า (mark) :** แทนสัญลักษณ์ด้วย x, y เพื่อจำว่าใช้ไปแล้ว
- **จำใน state :** อ่านค่าแล้วแยกไปคนละสาย state (TM ไม่มีตัวแปร ต้องจำด้วย state)
- **หมุดกลับบ้าน :** ใช้เครื่องหมายที่ขีดล่าสุดเป็นจุดให้หัวอ่านเดินกลับมา
- **zig-zag :** จับคู่ทีละคู่ แล้ววนกลับ จบเมื่อไม่เหลือตัวที่ยังไม่ขีด
- **ตรวจรูปแบบ :** เช็กจำนวนตัวคั่น (เช่น #) ระหว่างเดินผ่าน
- **พิสูจน์ว่าเป็น decider :** บอกว่าแต่ละรอบขีดเพิ่มอย่างน้อย 1 ตัว จึงวนได้จำกัดรอบ

บทที่ 16

Variants of Turing Machines

16.1 Variants of Turing machines

- มีนิยามแบบอื่นของ TM หลายแบบ ทั้งหมด**กำลังเท่ากับแบบเดิม** : recognize ภาษา class เดียวกัน คุณสมบัตินี้เรียก robustness
- ตัวอย่าง : อนุญาตให้ head **อยู่กับที่** $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}, \mathrm{S}\}$ ไม่เพิ่มกำลัง เพราะแทน S ด้วยการเลื่อนขวาแล้วเลื่อนซ้ายได้
- จะแสดงว่าสอง model เท่ากัน : แสดงว่าแต่ละตัว **simulate** อีกตัวได้

16.2 Multitape Turing machines

- มีหลาย tape แต่ละ tape มี head ของตัวเอง input อยู่บน tape 1 ที่เหลือว่าง $\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{\mathrm{L}, \mathrm{R}, \mathrm{S}\}^k$

Theorem 3.13

ทุก multitape TM มี single-tape TM ที่ equivalent

- **พิสูจน์** : S เก็บเนื้อหา k tape ต่อกันบน tape เดียว คั่นด้วย # ใช้ symbol ที่มี**จุดข้างบน**บอกตำแหน่ง head เสมือนของแต่ละ tape
  S = "On input $w = w_1 \cdots w_n$:
  1. จัด tape ให้เป็น $\#\dot{w}_1 w_2 \cdots w_n \# \dot{\sqcup} \# \dot{\sqcup} \# \cdots \#$
  2. จำลอง 1 ขั้น : กวาดจาก # ตัวแรกถึงตัวที่ $k+1$ เพื่ออ่าน symbol ใต้ head เสมือนทุกตัว แล้วกวาดอีกรอบเพื่ออัปเดตตาม δ ของ M
  3. ถ้า head เสมือนเลื่อนขวาไปชน # : เขียน $\sqcup$ แล้วเลื่อนเนื้อหาที่เหลือทั้งหมดไปขวา 1 ช่อง"
- Corollary 3.15 : ภาษาเป็น Turing-recognizable $\iff$ มี multitape TM recognize มัน

16.3 Nondeterministic Turing machines

- $\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\})$ การคำนวณเป็น tree accept ถ้า**มีกิ่งใดกิ่งหนึ่ง**ถึง accept state

Theorem 3.16

ทุก nondeterministic TM มี deterministic TM ที่ equivalent

- **Proof idea** : D ลองทุกกิ่งของ N ต้องค้นแบบ breadth-first (depth-first อาจติดในกิ่งที่ไม่จบแล้วพลาดกิ่งที่ accept)
- **Proof** : D มี 3 tape
  - tape 1 : input ไม่เคยเปลี่ยน
  - tape 2 : สำเนา tape ของ N บนกิ่งที่กำลังจำลอง
  - tape 3 : address ของ node ใน tree เป็น string บน $\Gamma_b = \{1, \ldots, b\}$ (b = จำนวนทางเลือกสูงสุดของ δ) เช่น 231 = ลูกคนที่ 2 แล้ว 3 แล้ว 1

  1. tape 1 มี w tape 2, 3 ว่าง
  2. copy tape 1 ไป tape 2 ตั้ง tape 3 เป็น ε
  3. จำลอง N บน tape 2 โดยเลือกทางตาม symbol บน tape 3 ถ้า symbol หมด หรือทางเลือกไม่มี หรือเจอ rejecting configuration ไปขั้น 4 เจอ accepting configuration ให้ *accept*
  4. เปลี่ยน tape 3 เป็น string ถัดไปตาม string ordering กลับไปขั้น 2

- Corollary 3.18 : ภาษาเป็น Turing-recognizable $\iff$ มี nondeterministic TM recognize มัน
- NTM เป็น decider ถ้า**ทุกกิ่งหยุดบนทุก input**
- Corollary 3.19 : ภาษาเป็น decidable $\iff$ มี nondeterministic TM decide มัน

16.4 Enumerators

- Enumerator = TM ที่มี **printer** ต่ออยู่ เริ่มด้วย tape ว่าง พิมพ์ string ออกมาเรื่อย ๆ (อาจไม่สิ้นสุด ซ้ำได้ ลำดับใดก็ได้)
- ภาษาที่ E enumerate = เซตของ string ทั้งหมดที่มันพิมพ์ ที่มาของชื่อ recursively enumerable

Theorem 3.21

ภาษาเป็น Turing-recognizable $\iff$ มี enumerator enumerate มัน

- ($\Leftarrow$) M = "On input w :
  1. รัน E ทุกครั้งที่ E พิมพ์ string ให้เทียบกับ w
  2. ถ้า w ปรากฏ *accept*"
- ($\Rightarrow$) ให้ $s_1, s_2, s_3, \ldots$ เป็นรายการของทุก string ใน Σ^*
  E = "ไม่สนใจ input :
  1. ทำซ้ำสำหรับ $i = 1, 2, 3, \ldots$
  2. รัน M i **ขั้น**บนแต่ละ input $s_1, \ldots, s_i$
  3. ถ้าตัวใด accept ให้พิมพ์ s_j นั้น"
- ต้องจำกัดจำนวนขั้น เพราะ M อาจ loop บนบาง s_j

16.5 Equivalence with other models

- มี model การคำนวณทั่วไปอีกมาก ทุกตัวมีจุดร่วมคือ **เข้าถึงความจำไม่จำกัดได้อย่างไม่มีข้อจำกัด** และทำงานได้จำนวนจำกัดต่อขั้น
- model ที่มีคุณสมบัตินี้ simulate กันได้ทั้งหมด เหมือนภาษาโปรแกรม : อัลกอริทึมที่เขียนได้ในภาษาหนึ่ง เขียนในอีกภาษาได้เสมอ
- ผลสำคัญ : class ของอัลกอริทึมที่บรรยายได้**ไม่ขึ้นกับ model ที่เลือก**

★ Claude สอนเพิ่ม : §3.3 ที่ต้องรู้ก่อนอ่านบทถัดไป (นอกรายการ Suggested reading)

- Church–Turing thesis : แนวคิด "อัลกอริทึม" แบบ intuitive = อัลกอริทึมของ Turing machine
- บรรยาย TM ได้ 3 ระดับ : formal description (state, δ ครบ) implementation description (บอกการเลื่อน head และการเก็บข้อมูล) high-level description (บรรยายอัลกอริทึม)
- input ของ TM เป็น string เสมอ object O เข้ารหัสเป็น $\langle O \rangle$ หลาย object เป็น $\langle O_1, \ldots, O_k \rangle$

บทที่ 17

Decidable กับ Recognizable

Objectives — อ่านจบแล้วต้องทำได้

- แยก decider กับ recognizer ออกจากกันได้
- พิสูจน์ closure ของภาษา decidable ได้
- รู้ว่ากรณี recognizable ต้องรันขนานเพื่อเลี่ยง loop

17.1 นิยาม

นิยาม

- M **recognize** L : $w \in L \iff M$ accept w (ถ้า $w \notin L$ อาจ reject หรือ loop)
- M **decide** L : M recognize L **และ** M หยุดทุก input
- L เป็น **Turing-recognizable** ถ้ามี TM recognize L
- L เป็น **Turing-decidable** ถ้ามี TM decide L

	$w \in L$	$w \notin L$
Decider	accept	reject
Recognizer	accept	reject **หรือ** loop

ทฤษฎีบทสำคัญ

$$L \text{ decidable} \iff L \text{ recognizable} \quad \text{และ} \quad \overline{L} \text{ recognizable}$$

17.2 โครงบทพิสูจน์ closure (ใช้ได้ทุกข้อ)

1. ให้ M_1 decide L_1 และ M_2 decide L_2
2. สร้าง M ที่เรียก M_1, M_2 เป็น subroutine เขียนเป็นขั้นตอนภาษาอังกฤษ/ไทย
3. แสดงว่า M **หยุดเสมอ** (เรียก decider จำนวนจำกัดครั้ง)
4. แสดงว่า $w \in L \Rightarrow M$ accept และ $w \notin L \Rightarrow M$ reject

17.3 สูตรการสร้าง M สำหรับภาษา decidable

Operation	M บน input w
$L_1 \cup L_2$	run M_1 แล้ว run M_2 accept ถ้า**ตัวใดตัวหนึ่ง** accept
$L_1 \cap L_2$	run M_1 แล้ว run M_2 accept ถ้า**ทั้งคู่** accept
$\overline{L_1}$	run M_1 แล้ว**สลับ**คำตอบ
$L_1 L_2$	ลองทุกจุดตัด $w = xy$ ($\lvert w\rvert + 1$ แบบ) accept ถ้ามีแบบที่ M_1 accept x และ M_2 accept y
L_1^*	ลองทุกวิธีตัด $w = x_1 x_2 \cdots x_k$ ($2^{\lvert w\rvert - 1}$ แบบ) accept ถ้ามีแบบที่ M_1 accept ทุกชิ้น

17.4 กรณี recognizable : ต้องระวัง loop

- run ต่อกันไม่ได้ เพราะ M_1 อาจ loop แล้วไม่ได้ run M_2
- ต้อง run **ขนานกัน** : สลับทำทีละขั้น (M_1 หนึ่งขั้น, M_2 หนึ่งขั้น, ...) หรือใช้ nondeterminism

Closure

	∪	∩	complement	concatenation	star
Regular	ปิด	ปิด	ปิด	ปิด	ปิด
Context-free	ปิด	**ไม่ปิด**	**ไม่ปิด**	ปิด	ปิด
Decidable	ปิด	ปิด	ปิด	ปิด	ปิด
Recognizable	ปิด	ปิด	**ไม่ปิด**	ปิด	ปิด

17.5 ปัญหา decidable ของ regular และ context-free languages

- แสดงปัญหาเป็นภาษา : "DFA B accept w หรือไม่" เท่ากับ "$\langle B, w\rangle \in A_{DFA}$ หรือไม่"
- พิสูจน์ decidable : ให้ TM (อัลกอริทึม) ที่**หยุดเสมอ**
- ข้อสอบมักถาม : ให้ภาษาแบบนี้มา แล้วให้เขียน decider โดยเรียก decider ที่รู้จักแล้วเป็น subroutine

เกี่ยวกับ regular languages

Theorem 4.1–4.5

- 4.1 $A_{DFA} = \{\langle B, w\rangle \mid B$ เป็น DFA ที่ accept $w\}$: decidable
- 4.2 $A_{NFA} = \{\langle B, w\rangle \mid B$ เป็น NFA ที่ accept $w\}$: decidable
- 4.3 $A_{REX} = \{\langle R, w\rangle \mid R$ เป็น regular expression ที่สร้าง $w\}$: decidable
- 4.4 $E_{DFA} = \{\langle A\rangle \mid A$ เป็น DFA และ $L(A) = \emptyset\}$: decidable
- 4.5 $EQ_{DFA} = \{\langle A, B\rangle \mid A, B$ เป็น DFA และ $L(A) = L(B)\}$: decidable

- 4.1 M = "On input $\langle B, w\rangle$:
  1. simulate B บน w
  2. จบที่ accept state ให้ *accept* ไม่งั้น *reject*" (เก็บ state ปัจจุบันและตำแหน่งใน w บน tape)
- 4.2 N = "On input $\langle B, w\rangle$:
  1. แปลง NFA B เป็น DFA C (Theorem 1.39)
  2. รัน M บน $\langle C, w\rangle$
  3. ตอบตาม M" (ใช้ M เป็น **subroutine**)
- 4.3 P = "On input $\langle R, w\rangle$:
  1. แปลง R เป็น NFA A (Theorem 1.54)
  2. รัน N บน $\langle A, w\rangle$
  3. ตอบตาม N"
- 4.4 T = "On input $\langle A\rangle$:
  1. mark start state
  2. ทำซ้ำจนไม่มี state ใหม่ถูก mark :
  3. mark state ที่มี transition เข้าจาก state ที่ mark แล้ว
  4. ถ้าไม่มี accept state ถูก mark ให้ *accept* ไม่งั้น *reject*"

- 4.5 สร้าง DFA C ที่รับ **symmetric difference**

$$L(C) = \big(L(A) \cap \overline{L(B)}\big) \cup \big(\overline{L(A)} \cap L(B)\big)$$

$L(C) = \emptyset \iff L(A) = L(B)$ (regular ปิดภายใต้ complement, union, intersection)
F = "On input $\langle A, B\rangle$:

1. สร้าง C
2. รัน T บน $\langle C\rangle$
3. T accept ให้ *accept* ไม่งั้น *reject*"

เกี่ยวกับ context-free languages

Theorem 4.7–4.9

- 4.7 $A_{CFG} = \{\langle G, w\rangle \mid G$ เป็น CFG ที่สร้าง $w\}$: decidable
- 4.8 $E_{CFG} = \{\langle G\rangle \mid G$ เป็น CFG และ $L(G) = \emptyset\}$: decidable
- 4.9 ทุก context-free language เป็น decidable

- 4.7 ลองทุก derivation ไม่ได้ (มีไม่จำกัด ได้แค่ recognizer) ใช้ CNF : derivation ของ w ยาว n มี $2n-1$ ขั้น
S = "On input $\langle G, w\rangle$:
  1. แปลง G เป็น CNF
  2. ไล่ทุก derivation ที่มี $2n-1$ ขั้น ($n = |w|$) ถ้า $n = 0$ ไล่ derivation 1 ขั้น
  3. ถ้ามีตัวใดสร้าง w ให้ *accept* ไม่งั้น *reject*"
- 4.8 ลองทุก w ไม่ได้ ใช้ marking : หาว่า variable ใดสร้าง string ของ terminal ได้
R = "On input $\langle G\rangle$:
  1. mark terminal ทุกตัว
  2. ทำซ้ำจนไม่มี variable ใหม่ถูก mark :
  3. mark A ถ้ามีกฎ $A \to U_1 \cdots U_k$ ที่ทุก U_i ถูก mark แล้ว
  4. ถ้า start variable ไม่ถูก mark ให้ *accept* ไม่งั้น *reject*"
- $EQ_{CFG} = \{\langle G, H\rangle \mid L(G) = L(H)\}$: **ไม่ decidable** วิธีของ 4.5 ใช้ไม่ได้ เพราะ CFL ไม่ปิดภายใต้ complement และ intersection
- 4.9 ห้ามแปลง PDA เป็น TM ตรง ๆ (บางกิ่งของ PDA อาจไม่หยุด) ให้ G เป็น CFG ของ A
M_G = "On input w :
  1. รัน S บน $\langle G, w\rangle$
  2. ตอบตาม S"

Figure 4.10 : ความสัมพันธ์ของ class

regular $\subsetneq$ context-free $\subsetneq$ decidable $\subsetneq$ Turing-recognizable

บทที่ 18

Undecidability ของ A_{TM}

Objectives — อ่านจบแล้วต้องทำได้

- อธิบายได้ว่าทำไม A_{TM} recognizable
- พิสูจน์ว่า A_{TM} ไม่ decidable ด้วย diagonalization ได้
- ใช้ reduction พิสูจน์ว่าภาษาอื่น undecidable ได้

18.1 ภาษา

สูตร

$$A_{TM} = ACCEPT = \{\langle M, w\rangle : M \text{ เป็น TM และ } M \text{ accept } w\}$$

- $\langle o\rangle$ = การเข้ารหัส object o เป็นสตริง (TM, กราฟ, คู่อันดับ ฯลฯ เข้ารหัสได้หมด)
- A_{TM} เป็น **recognizable** แต่**ไม่ decidable**

18.2 ทำไม recognizable

- universal TM U : บน input $\langle M, w\rangle$ ให้จำลอง M บน w
- M accept : U accept M reject : U reject M loop : U ก็ loop
- U จึงเป็น recognizer แต่ไม่ใช่ decider

18.3 ทำไมไม่ decidable : diagonalization

1. สมมติมี decider H : $H(\langle M, w\rangle)$ = accept ถ้า M accept w มิฉะนั้น reject
2. สร้าง D : บน input $\langle M\rangle$ run H บน $\langle M, \langle M\rangle\rangle$ แล้ว**ตอบตรงข้าม**
3. run D บน $\langle D\rangle$:

$$D \text{ accept } \langle D\rangle \iff H \text{ reject } \langle D, \langle D\rangle\rangle \iff D \text{ ไม่ accept } \langle D\rangle$$

4. ขัดแย้ง ดังนั้น H ไม่มีอยู่จริง

18.4 ภาพของ diagonalization

	$\langle M_1\rangle$	$\langle M_2\rangle$	$\langle M_3\rangle$	$\cdots$	$\langle D\rangle$
M_1	**acc**	rej	acc	$\cdots$	
M_2	acc	**acc**	acc	$\cdots$	
M_3	rej	rej	**rej**	$\cdots$	
$\vdots$				$\ddots$	
D	rej	rej	acc	$\cdots$	**?**

- ช่อง (แถว M_i, คอลัมน์ $\langle M_j\rangle$) คือคำตอบของ $H(\langle M_i, \langle M_j\rangle\rangle)$
- แถวของ D = **กลับค่าเส้นทแยงมุม**
- ช่อง “?” ต้องเป็นค่าตรงข้ามกับตัวเอง จึงเป็นไปไม่ได้

ผลที่ตามมา

- $\overline{A_{TM}}$ **ไม่ recognizable** (ถ้า recognizable ทั้งคู่ A_{TM} จะ decidable)
- $HALT_{TM} = \{\langle M, w\rangle : M \text{ หยุดบน } w\}$ ไม่ decidable เช่นกัน

18.5 โครงพิสูจน์ undecidable ด้วย reduction

1. สมมติว่าภาษา B decidable มี decider R
2. ใช้ R สร้าง decider S สำหรับ A_{TM}
3. แต่ A_{TM} ไม่ decidable ขัดแย้ง ดังนั้น B ไม่ decidable

18.6 Countable sets และ diagonalization ของ Cantor

- เทคนิคของ Georg Cantor (1873) สำหรับเทียบขนาดของเซตอนันต์
- Definition 4.12 : $f : A \to B$ เป็น one-to-one ถ้า $f(a) \neq f(b)$ เมื่อ $a \neq b$ เป็น onto ถ้าทุก $b \in B$ มี a ที่ $f(a) = b$ A, B **ขนาดเท่ากัน**ถ้ามี f ที่เป็นทั้งสองอย่าง (correspondence)
- Example 4.13 : $\mathcal{N}$ กับเลขคู่ $\mathcal{E} = \{2, 4, 6, \dots\}$ ขนาดเท่ากัน $f(n) = 2n$
- Definition 4.14 : A เป็น countable ถ้าจำกัด หรือขนาดเท่ากับ $\mathcal{N}$
- Example 4.15 : $\mathcal{Q} = \{m/n \mid m, n \in \mathcal{N}\}$ เป็น countable วาง m/n ในตารางอนันต์แล้วไล่ตาม**แนวทแยง** ข้ามตัวที่ซ้ำ
- Theorem 4.17 : $\mathcal{R}$ เป็น uncountable
  - สมมติมี correspondence $f : \mathcal{N} \to \mathcal{R}$
  - สร้าง $x \in (0, 1)$ โดยให้ทศนิยมตำแหน่งที่ n ต่างจากตำแหน่งที่ n ของ $f(n)$
  - $x \neq f(n)$ ทุก n
  - ขัดแย้ง
  - (เลี่ยงเลข 0 และ 9)
- Corollary 4.18 : มีภาษาที่ไม่ Turing-recognizable
  - เซตของ TM ทั้งหมด countable : Σ^* countable และแต่ละ TM มี encoding $\langle M\rangle$
  - เซตของภาษาทั้งหมด $\mathcal{L}$ uncountable : เซต $\mathcal{B}$ ของ infinite binary sequence uncountable (diagonalization) และ $\mathcal{L}$ ขนาดเท่า $\mathcal{B}$ ผ่าน characteristic sequence χ_A (บิตที่ i เป็น 1 เมื่อ $s_i \in A$)
  - TM แต่ละตัว recognize ภาษาเดียว ภาษามีมากกว่า TM บางภาษาไม่มี TM recognize

18.7 ภาษาที่ไม่ Turing-recognizable

- ภาษาเป็น co-Turing-recognizable ถ้าเป็น complement ของภาษาที่ Turing-recognizable

Theorem 4.22 และ Corollary 4.23

- 4.22 ภาษาเป็น decidable $\iff$ เป็นทั้ง Turing-recognizable และ co-Turing-recognizable
- 4.23 $\overline{A_{TM}}$ ไม่ Turing-recognizable

- 4.22 ($\Rightarrow$) A decidable : A และ $\overline{A}$ decidable จึง recognizable ทั้งคู่
  ($\Leftarrow$) ให้ M_1 recognize A, M_2 recognize $\overline{A}$
  M = "On input w :
  1. รัน M_1 และ M_2 บน w **ขนานกัน** (สลับทีละขั้น)
  2. M_1 accept ให้ *accept* ; M_2 accept ให้ *reject*"

  ทุก w อยู่ใน A หรือ $\overline{A}$: ตัวใดตัวหนึ่งต้อง accept M จึงหยุดเสมอ
- 4.23 A_{TM} recognizable ถ้า $\overline{A_{TM}}$ recognizable ด้วย จะได้ A_{TM} decidable ขัดกับ Theorem 4.11

บทที่ 19

รวมสูตรทั้งเล่ม และแผนที่ตัดสินใจ

19.1 ภาค 1 : Regular languages

นิยามเครื่อง

- DFA : $M = (Q, \Sigma, \delta, q_0, F)$
- $\delta : Q \times \Sigma \to Q$
- NFA : $N = (Q, \Sigma, \delta, q_0, F)$
- $\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q)$
- $\hat{\delta}(q, \varepsilon) = q, \quad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$
- $L(M) = \{w : \hat{\delta}(q_0, w) \in F\}$

สตริงและภาษา

$|\Sigma^n| = |\Sigma|^n$ $|\mathcal{P}(Q)| = 2^{|Q|}$ $(xy)^R = y^R x^R$ $A\emptyset = \emptyset$ $A\{\varepsilon\} = A$ $\emptyset^* = \{\varepsilon\}$ $A^+ = AA^*$

Subset construction

$q_0' = E(\{q_0\})$ $\delta'(R, a) = E\big(\bigcup_{r \in R} \delta(r, a)\big)$ $F' = \{R : R \cap F \neq \emptyset\}$ ไม่เกิน 2^n state

Closure

- Product : $Q = Q_1 \times Q_2, \quad \delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$ มี $|Q_1||Q_2|$ state
- Union : $r_1 \in F_1$ หรือ $r_2 \in F_2$
- Intersection : $F_1 \times F_2$
- Complement (DFA) : $Q \setminus F$
- NFA : union $n_1 + n_2 + 1$ state concatenation $n_1 + n_2$ state star $n_1 + 1$ state
- Regular ปิดภายใต้ : $\cup$, $\cap$, complement, difference, concatenation, star, reverse

Regular expression

ลำดับ : star > concatenation > union $R \cup \emptyset = R$ $R\varepsilon = R$ $R\emptyset = \emptyset$ $(R^*)^* = R^*$ $(R \cup S)^* = (R^* S^*)^*$

GNFA ตัด q_{rip} : ป้ายใหม่ของ $q_i \to q_j = R_1 R_2^* R_3 \cup R_4$

Pumping lemma (regular)

$s \in A, |s| \geq p \quad \Rightarrow \quad s = xyz$ โดย

1. $xy^i z \in A \ \forall i \geq 0$
2. $|y| > 0$
3. $|xy| \leq p$

p = จำนวน state ของ DFA สตริงมาตรฐาน : $0^p 1^p$, $0^p 10^p 1$, 1^{p^2}, $0^{p+1} 1^p$

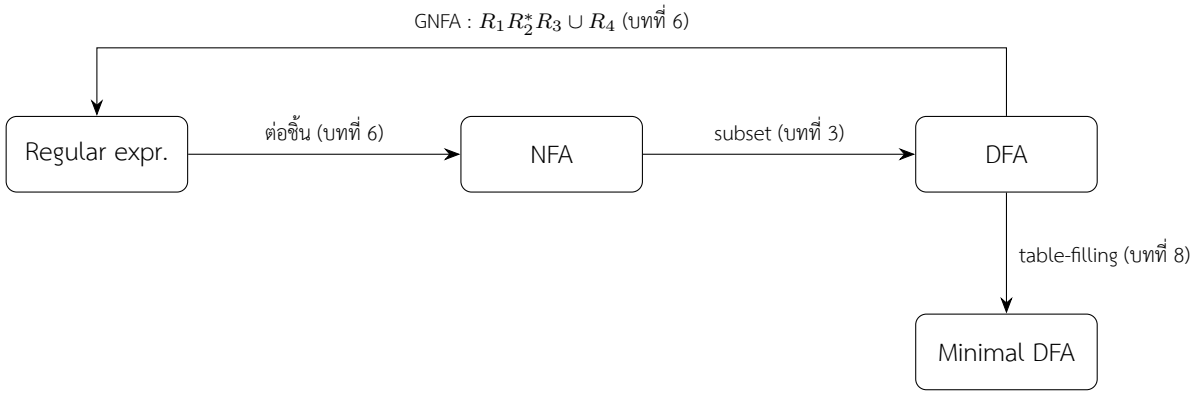
Myhill–Nerode

L regular $\iff$ index ของ $\equiv_L$ จำกัด จำนวน state ของ minimal DFA = index

แผนที่ตัดสินใจ : ภาษานี้ regular หรือไม่

1. ถามว่า “ต้องจำอะไร” ถ้าจำนวนสิ่งที่ต้องจำ**จำกัด** : น่าจะ regular
2. จะพิสูจน์ว่า **regular** เลือกทางที่ง่ายสุด :
   - สร้าง DFA (เงื่อนไขแบบ “ทุกตำแหน่ง”, “ไม่มี”, นับ mod)
   - สร้าง NFA (เงื่อนไขแบบ “มีที่ไหนสักแห่ง”, “จากท้าย”, “หรือ”)
   - เขียน regular expression (รูปแบบเป็นบล็อกชัดเจน)
   - ใช้ closure ประกอบจากภาษาที่รู้ว่า regular
3. จะพิสูจน์ว่า **ไม่ regular** :
   - pumping lemma (ทางหลัก)
   - closure + ภาษาที่รู้ว่าไม่ regular (สั้นกว่าถ้าใช้ได้)
   - Myhill–Nerode (หาเซตอนันต์ที่แยกกันได้ทุกคู่)

การแปลงทั้งหมดของ regular language



19.2 ภาค 2 : Context-free languages และ Computability

นิยามเครื่อง

- CFG : $G = (V, \Sigma, R, S)$
- กฎ $A \to w, \quad w \in (V \cup \Sigma)^*$
- PDA : $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$
- $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$
- TM : $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\text{L}, \text{R}\}$

CFG สำเร็จรูป

$a^n b^n$: $S \to aSb \mid \varepsilon$ $\#0 = \#1$: $S \to 0S1S \mid 1S0S \mid \varepsilon$ ww^R : $S \to 0S0 \mid 1S1 \mid \varepsilon$
union : $S \to S_1 \mid S_2$ concat : $S \to S_1S_2$ star : $S \to S_1S \mid \varepsilon$

CFG → PDA

$q_s \xrightarrow{\varepsilon,\varepsilon\to S\$} q_{\text{loop}} \xrightarrow{\varepsilon,\$\to\varepsilon} q_{\text{acc}}$ loop : $\varepsilon, A \to w$ (ทุกกฎ) และ $a, a \to \varepsilon$ (ทุก terminal)

PDA → CFG

เงื่อนไข : accept state เดียว, stack ว่างก่อน accept, ทุกเส้น push หรือ pop อย่างเดียว

1. $p \xrightarrow{a,\varepsilon\to t} r$ และ $s \xrightarrow{b,t\to\varepsilon} q \quad \Rightarrow \quad A_{pq} \to a\,A_{rs}\,b$
2. $A_{pq} \to A_{pr}A_{rq}$
3. $A_{pp} \to \varepsilon$ เริ่มที่ $A_{q_0 q_{\text{acc}}}$

จำนวน : ตัวแปร $|Q|^2$, แบบ 2 $|Q|^3$ ข้อ, แบบ 3 $|Q|$ ข้อ

Pumping lemma (CFL)

$s \in L, |s| \geq p \quad \Rightarrow \quad s = uvxyz$ โดย

1. $uv^i xy^i z \in L \ \forall i \geq 0$
2. $|vy| > 0$
3. $|vxy| \leq p$

พิสูจน์ไม่เป็น CFL : เลือก s เอง รับการแบ่งทุกแบบ เลือก i เอง (มัก $i = 2$ หรือ 0)

TM configuration

$\delta(q_i, b) = (q_j, c, \mathsf{R}) : ua\,q_i\,bv \vdash uac\,q_j\,v$ $\qquad$ $\delta(q_i, b) = (q_j, c, \mathsf{L}) : ua\,q_i\,bv \vdash u\,q_j\,acv$

Decidability

- Regular $\subsetneq$ CFL $\subsetneq$ Decidable $\subsetneq$ Recognizable
- L decidable $\iff$ L และ $\overline{L}$ recognizable
- Decidable ปิดภายใต้ $\cup$, $\cap$, complement, concatenation, star
- CFL ไม่ปิดภายใต้ $\cap$ และ complement
- Recognizable ไม่ปิดภายใต้ complement
- A_{TM} : recognizable แต่ไม่ decidable
- $\overline{A_{TM}}$: ไม่ recognizable
- จำนวนจุดตัด $w = xy$ มี $|w| + 1$ แบบ
- จำนวนวิธีตัดเป็นชิ้นไม่ว่าง มี $2^{|w|-1}$ แบบ

Chomsky normal form

รูปกฎ : $A \to BC$ หรือ $A \to a$ (B, C ไม่ใช่ start variable) อนุญาต $S \to \varepsilon$ string ยาว $n \geq 1$ ใช้ derivation $2n - 1$ ขั้นพอดี

แปลง : (1) เพิ่ม $S_0 \to S$ (2) ตัด ε-rule (3) ตัด unit rule (4) แตกกฎยาว และแทน terminal ในกฎยาวด้วย variable ใหม่

ภาษา decidable ที่ต้องจำ

- Decidable : A_{DFA}, A_{NFA}, A_{REX}, E_{DFA}, EQ_{DFA}, A_{CFG}, E_{CFG}, และทุก CFL
- ไม่ decidable : EQ_{CFG}, A_{TM}
- ไม่ recognizable : $\overline{A_{TM}}$
- Multitape TM, nondeterministic TM, enumerator : กำลังเท่ากับ TM เทปเดียว
- Countable : $\mathcal{N}$, $\mathcal{Q}$, Σ^*, เซตของ TM ทั้งหมด Uncountable : $\mathcal{R}$, เซตของภาษาทั้งหมด

ภาค 3

สรุปตามหนังสือ Sipser

Introduction to the Theory of Computation, 3rd ed.

- เรียงบทตามตารางเนื้อหาของวิชา 15 แถว บวกบทที่ S0 (math background)
- เก็บทุกหัวข้อย่อยของหนังสือในช่วงหน้านั้น และทุก Definition, Theorem, Lemma, Corollary, Example ที่มีเลขกำกับ
- ภาค 1–2 เป็นฉบับสอนละเอียด ภาค 3 เป็นฉบับสรุปไว้เปิดเทียบกับหนังสือ

บทที่ S0

Math background

อ้างอิง : Sipser 3rd ed. §0.2–0.4 (pp. 3–23)

S0.1 Sets

- **Set** = กลุ่มของ object ไม่สนลำดับ ไม่สนการซ้ำ object ในเซตเรียก **element** / **member** เขียน $7 \in \{7, 21, 57\}$, $8 \notin \{7, 21, 57\}$
- **Subset** $A \subseteq B$: สมาชิกทุกตัวของ A อยู่ใน B
- **Proper subset** $A \subsetneq B$: subset และ $A \neq B$
- **Multiset** : นับการซ้ำ $\{7\}$ กับ $\{7, 7\}$ ต่างกันเมื่อมองเป็น multiset
- **Infinite set** : $\mathcal{N} = \{1, 2, 3, \dots\}$ (natural numbers) $\mathcal{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ (integers)
- **Empty set** $\emptyset$
- **Singleton set** มีสมาชิก 1 ตัว
- **Unordered pair** มีสมาชิก 2 ตัว
- เขียนแบบกฎ : $\{n \mid n = m^2 \text{ สำหรับบาง } m \in \mathcal{N}\}$ = เซตของ perfect square
- **Union** $A \cup B$
- **Intersection** $A \cap B$
- **Complement** $\overline{A}$ แสดงด้วย **Venn diagram**

S0.2 Sequences and tuples

- **Sequence** = รายการที่**สนลำดับและการซ้ำ** $(7, 21, 57) \neq (57, 7, 21) \neq (7, 7, 21, 57)$
- sequence จำกัดเรียก **tuple** มี k ตัวเรียก **k-tuple** 2-tuple เรียก **ordered pair**
- **Power set** $\mathcal{P}(A)$ = เซตของ subset ทั้งหมดของ A เช่น $A = \{0, 1\} : \mathcal{P}(A) = \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$
- **Cartesian product** $A \times B$ = เซตของ ordered pair ที่ตัวแรกมาจาก A ตัวที่สองมาจาก B
- Example 0.5 : $A = \{1, 2\}, B = \{x, y, z\}$ $A \times B = \{(1, x), (1, y), (1, z), (2, x), (2, y), (2, z)\}$
- Example 0.6 : $A \times B \times A$ มี $2 \cdot 3 \cdot 2 = 12$ ตัว เช่น $(1, x, 1), (1, x, 2), \dots, (2, z, 2)$
- $A^k = \underbrace{A \times \cdots \times A}_{k}$
- Example 0.7 : $\mathcal{N}^2 = \mathcal{N} \times \mathcal{N} = \{(i, j) \mid i, j \geq 1\}$

S0.3 Functions and relations

- **Function** (mapping) : input เดียวกันให้ output เดียวกันเสมอ
  - $f(a) = b$
  - เขียน $f : D \to R$
  - D = **domain**, R = **range**
- **Onto** : ใช้สมาชิกของ range ครบทุกตัว
- Example 0.8 : $f : \{0, 1, 2, 3, 4\} \to \{0, 1, 2, 3, 4\}$, $f(n) = (n + 1) \bmod 5$ เขียนเป็นตาราง 1 มิติได้
- Example 0.9 : $g : \mathcal{Z}_4 \times \mathcal{Z}_4 \to \mathcal{Z}_4$,
  - $g(i, j) = (i + j) \bmod 4$

  - เขียนเป็นตาราง 2 มิติ
  - $(\mathcal{Z}_m = \{0, \ldots, m-1\})$
- domain เป็น $A_1 \times \cdots \times A_k$: input $(a_1, \ldots, a_k)$ เรียก **arguments** เป็น **k-ary function** $k = 1$ **unary**, $k = 2$ **binary**
- **Infix notation** : $a + b$
- **Prefix notation** : $add(a, b)$
- **Predicate / property** : function ที่ range คือ $\{\text{TRUE}, \text{FALSE}\}$
- **Relation** : property ที่ domain เป็นเซตของ k-tuple (k-ary relation) $k = 2$ เรียก **binary relation** เขียน aRb
- **Example 0.10** : relation *beats* ของ เกม Scissors–Paper–Stone เขียน เป็น เซต ได้ $\{(\text{SCISSORS}, \text{PAPER}), (\text{PAPER}, \text{STONE}), (\text{STONE}, \text{SCISSORS})\}$
- **Equivalence relation** : binary relation ที่ **reflexive** (xRx), **symmetric** $(xRy \Rightarrow yRx)$, **transitive** $(xRy, yRz \Rightarrow xRz)$
- **Example 0.11** : $i \equiv_7 j$ เมื่อ $i - j$ หารด้วย 7 ลงตัว เป็น equivalence relation (ตรวจครบ 3 ข้อ)

S0.4 Graphs

- **Undirected graph** $G = (V, E)$: **nodes** (vertices) และ **edges** edge เขียน (i, j) ไม่สนลำดับ
- **Degree** ของ node = จำนวน edge ที่ติดกับ node นั้น ไม่มี edge ซ้ำระหว่างคู่เดียวกัน อาจมี self-loop
- **Labeled graph** : มีป้ายบน node / edge
- **Subgraph** : เอา node บางส่วนและ edge ระหว่างมัน
- **Path** : ลำดับของ node ที่ต่อกันด้วย edge
- **Simple path** : ไม่ซ้ำ node
- **Connected** : ทุกคู่ node มี path ถึงกัน
- **Cycle** : path ที่เริ่มและจบ node เดียวกัน
- **Simple cycle** : มีอย่างน้อย 3 node ซ้ำแค่ตัวแรกกับตัวสุดท้าย
- **Tree** : connected และไม่มี simple cycle มี **root** ได้ node degree 1 ที่ไม่ใช่ root เรียก **leaves**
- **Directed graph** : edge มีทิศ (i, j) = จาก i ไป j **outdegree** / **indegree**
- **Directed path** : ลูกศรชี้ทางเดียวกันตลอด
- **Strongly connected** : ทุกคู่ node มี directed path ถึงกัน
- **Example 0.17** : directed graph ใช้แทน binary relation ได้ เช่น graph ของ relation *beats* ใน Example 0.10

S0.5 Strings and languages

- **Alphabet** = เซตจำกัดไม่ว่างของ **symbols**
  - เช่น $\Sigma_1 = \{0, 1\}$,
  - $\Sigma_2 = \{a, b, \ldots, z\}$,
  - $\Gamma = \{0, 1, x, y, z\}$
- **String** over Σ = ลำดับจำกัดของ symbol
- **Length** $|w|$
- **Empty string** ε (ยาว 0)
- **Reverse** $w^{\mathcal{R}}$
- **Substring** : z ปรากฏต่อเนื่องใน w เช่น `cad` เป็น substring ของ `abracadabra`
- **Concatenation** xy

- $x^k = \underbrace{xx\cdots x}_{k}$
- **Lexicographic order** = ลำดับแบบพจนานุกรม
- **Shortlex** (string order) = สั้น ก่อน ยาว ภายใน ความ ยาว เท่า กัน เรียง แบบ พจนานุกรม เหนือ $\{0,1\}$: $(\varepsilon, 0, 1, 00, 01, 10, 11, 000, \dots)$
- **Prefix** : x เป็น prefix ของ y ถ้ามี z ที่ $xz = y$
- **Proper prefix** : และ $x \neq y$
- **Language** = เซตของ string
- **Prefix-free** : ไม่มีสมาชิกตัวใดเป็น proper prefix ของอีกตัว

S0.6 Boolean logic

- **Boolean values** : TRUE/FALSE แทนด้วย 1/0
- **Boolean operations** :
- **Negation** (NOT) $\neg$
- **Conjunction** (AND) $\wedge$
- **Disjunction** (OR) $\vee$ ค่าที่นำมาคำนวณเรียก **operands**
- **Exclusive or** (XOR) $\oplus$: เป็น 1 เมื่อ operand ต่างกัน
- **Equality** $\leftrightarrow$: เป็น 1 เมื่อเท่ากัน **Implication** $\rightarrow$: เป็น 0 เฉพาะ $1 \rightarrow 0$
- ทุกตัวเขียนด้วย AND, OR, NOT ได้ : $P \vee Q = \neg(\neg P \wedge \neg Q)$ $P \rightarrow Q = \neg P \vee Q$ $P \leftrightarrow Q = (P \rightarrow Q) \wedge (Q \rightarrow P)$ $P \oplus Q = \neg(P \leftrightarrow Q)$
- **Distributive law** : $P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$
- $P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$

S0.7 Summary of mathematical terms

- หนังสือรวมศัพท์ทั้งหมดของ §0.2 ไว้เป็นตารางที่หน้า 16 ใช้เปิดทวนเมื่อเจอศัพท์ที่ลืม
- ศัพท์ที่ใช้บ่อยสุดในบทถัดไป : alphabet, string, language, concatenation, power set, Cartesian product, k-tuple, function, equivalence relation

S0.8 §0.3 Definitions, theorems, and proofs

- **Definition** : บอกความหมายของ object ให้ชัดเจน
- **Mathematical statement** : ข้อความที่จริงหรือเท็จ
- **Proof** : การให้เหตุผลที่ยืนยันว่า statement จริงแน่นอน
- **Theorem** : statement ที่พิสูจน์แล้ว
- **Lemma** : ผลที่พิสูจน์เพื่อใช้ช่วยพิสูจน์อย่างอื่น
- **Corollary** : ผลที่ตามมาง่าย ๆ จาก theorem

Finding proofs

- อ่าน statement ให้เข้าใจทุกสัญลักษณ์ก่อน แล้วเขียนใหม่ด้วยคำของตัวเอง
- "P if and only if Q" ($P \iff Q$) ต้องพิสูจน์ 2 ทิศ : forward ($P \Rightarrow Q$) และ reverse ($Q \Rightarrow P$)
- "ทุก object มี property" : ลองหา counterexample ถ้าหาไม่ได้จะเห็นว่าทำไมมันจริง
- ลองกรณีพิเศษก่อน คำแนะนำของหนังสือ : **be patient, come back to it, be neat, be concise**
- Example 0.19 : จะพิสูจน์ว่าผลรวม degree ของทุก graph เป็นเลขคู่
  - ลองวาด graph เล็ก ๆ
  - ลองหา counterexample
  - จะเห็นว่าเพิ่ม edge ทีละเส้นแล้วผลรวมเพิ่มทีละ 2
- Theorem 0.20 : $\overline{A \cup B} = \overline{A} \cap \overline{B}$ พิสูจน์เซตเท่ากันโดยแสดงว่าสมาชิกของฝั่งหนึ่งอยู่ในอีกฝั่ง ทั้ง 2 ทิศ
- Theorem 0.21 : ผลรวม degree ของทุก node เป็นเลขคู่ เพราะ edge แต่ละเส้นเพิ่มผลรวม 2 มี e edge ได้ผลรวม $2e$

S0.9 §0.4 Types of proof

Proof by construction

- จะพิสูจน์ว่า "มี object แบบนี้" : สร้างให้ดู
- Theorem 0.22 : ทุกเลขคู่ $n > 2$ มี 3-regular graph ที่มี n node (k-regular : ทุก node มี degree k)
  สร้าง : $V = \{0, \ldots, n-1\}$ วาง node เป็นวงกลม ต่อ node ติดกัน และต่อ node ตรงข้าม $\{i,\ i + n/2\}$

Proof by contradiction

- สมมติว่า theorem เท็จ แล้วแสดงว่านำไปสู่ข้อสรุปที่เท็จชัดเจน (contradiction)
- Example 0.23 : Jack เห็น Jill ตัวแห้ง จึงรู้ว่าฝนไม่ตก (ถ้าฝนตก Jill ต้องเปียก)
- Theorem 0.24 : $\sqrt{2}$ เป็น irrational
  - สมมติ $\sqrt{2} = m/n$ เป็นเศษส่วนอย่างต่ำ
  - ได้ $m^2 = 2n^2$
  - m คู่
  - ให้ $m = 2k$
  - ได้ $n^2 = 2k^2$
  - n คู่
  - ขัดกับเศษส่วนอย่างต่ำ

Proof by induction

- ใช้พิสูจน์ว่าสมาชิกทุกตัวของเซตอนันต์ (มัก $\mathcal{N}$) มี property $\mathcal{P}$
- Basis : พิสูจน์ $\mathcal{P}(1)$
- Induction step : ทุก $i \geq 1$ ถ้า $\mathcal{P}(i)$ จริง แล้ว $\mathcal{P}(i+1)$ จริง
- สมมติฐาน "$\mathcal{P}(i)$ จริง" เรียก induction hypothesis บางครั้งใช้แบบแรงกว่า : สมมติ $\mathcal{P}(j)$ จริงทุก $j \leq i$
- Theorem 0.25 (สูตรผ่อนบ้าน) : P_t = เงินต้นคงเหลือหลังเดือนที่ t, M = ตัวคูณดอกเบี้ยรายเดือน, Y = ค่างวด

$$P_t = PM^t - Y\left(\frac{M^t - 1}{M - 1}\right) \qquad \text{ทุก } t \geq 0$$

Basis $t = 0$: $P_0 = P$ Induction step : ใช้ $P_{k+1} = P_k M - Y$ แทนสูตรของ P_k แล้วจัดรูป

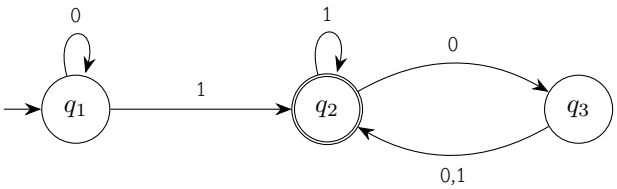
บทที่ S1

Deterministic Finite Automata (DFA)

อ้างอิง : Sipser 3rd ed. §1.1 (pp. 31–40)

S1.1 Finite automata

- **Finite automaton** (finite state machine) = model ของคอมพิวเตอร์ที่มี**ความจำจำกัดมาก**
- ตัวอย่างเปิดบท : ตัวควบคุมประตูอัตโนมัติ
  - 2 state (OPEN, CLOSED)
  - input 4 แบบ (FRONT, REAR, BOTH, NEITHER)
  - ใช้ความจำแค่ 1 bit
- **State diagram** ของ M_1 (Figure 1.4) : **start state** มีลูกศรชี้เข้าจากที่ว่าง **accept state** เป็นวงกลมสองชั้น ลูกศรระหว่าง state คือ **transition**
- อ่าน input ทีละตัวจากซ้ายไปขวา อ่านจบแล้วอยู่ที่ accept state : **accept** ไม่งั้น **reject**



M_1 : เช่น input 1101 ผ่าน q_1, q_2, q_2, q_3, q_2 แล้ว accept

S1.2 Formal definition of a finite automaton

Definition 1.5

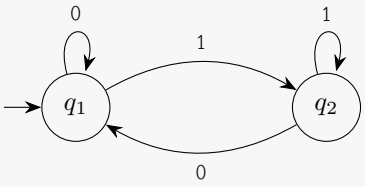
Finite automaton คือ 5-tuple $(Q, \Sigma, \delta, q_0, F)$

1. Q = เซตจำกัดของ **states**
2. Σ = เซตจำกัดเรียก **alphabet**
3. $\delta : Q \times \Sigma \to Q$ = **transition function**
4. $q_0 \in Q$ = **start state**
5. $F \subseteq Q$ = เซตของ **accept states** (final states)

- M_1 แบบ formal : $Q = \{q_1, q_2, q_3\}$,
  - $\Sigma = \{0, 1\}$,
  - start q_1,
  - $F = \{q_2\}$
- A = เซตของ string ที่ M accept : เรียก A ว่า **language of machine** M เขียน $L(M) = A$ พูดว่า M **recognizes** A
- เครื่องหนึ่ง accept ได้หลาย string แต่ recognize **ภาษาเดียว** ถ้าไม่ accept อะไรเลย มัน recognize $\emptyset$
- $L(M_1) = \{w \mid w$ มี 1 อย่างน้อย 1 ตัว และมี 0 จำนวนคู่ตามหลัง 1 ตัวสุดท้าย$\}$

S1.3 Examples of finite automata

ตัวอย่าง : Example 1.7 และ 1.9 : M_2, M_3



- M_2 : $F = \{q_2\}$
- $L(M_2) = \{w \mid w$ ลงท้ายด้วย $1\}$
- M_3 : diagram เดียวกัน แต่ $F = \{q_1\}$
- $L(M_3) = \{w \mid w = \varepsilon$ หรือลงท้ายด้วย $0\}$
- start state เป็น accept state : เครื่อง accept ε

ตัวอย่าง : Example 1.11 : M_4 (5 state, $\Sigma = \{a, b\}$)

	a	b
$\rightarrow s$	q_1	r_1
$*\, q_1$	q_1	q_2
q_2	q_1	q_2
$*\, r_1$	r_2	r_1
r_2	r_2	r_1

- ตัวแรกเป็น a ไปฝั่ง q, เป็น b ไปฝั่ง r แต่ละฝั่งจำว่าตัวล่าสุดเป็นตัวเดียวกับตัวแรกหรือไม่
- $L(M_4)$ = string ที่**ขึ้นต้นและลงท้ายด้วย symbol เดียวกัน**

ตัวอย่าง : Example 1.13 : M_5 ($\Sigma = \{\langle\text{RESET}\rangle, 0, 1, 2\}$)

- 3 state q_0, q_1, q_2
  - start q_0
  - $F = \{q_0\}$
  - q_j หมายถึง “ผลรวมสะสม mod 3 เท่ากับ j”
- อ่าน 0 อยู่ที่เดิม
  - อ่าน 1 เลื่อน $+1$
  - อ่าน 2 เลื่อน $+2$ (mod 3)
  - อ่าน $\langle$RESET$\rangle$ กลับ q_0
- accept เมื่อผลรวม (นับจาก $\langle$RESET$\rangle$ ล่าสุด) หารด้วย 3 ลงตัว

ตัวอย่าง : Example 1.15 : ทั่วไปเป็น B_i

- A_i = string ที่ผลรวมเป็นพหุคูณของ i (reset เมื่อเจอ $\langle$RESET$\rangle$)
- $B_i = (Q_i, \Sigma, \delta_i, q_0, \{q_0\})$, $Q_i = \{q_0, \ldots, q_{i-1}\}$
- $\delta_i(q_j, 0) = q_j$ $\delta_i(q_j, 1) = q_{(j+1) \bmod i}$ $\delta_i(q_j, 2) = q_{(j+2) \bmod i}$ $\delta_i(q_j, \langle\text{RESET}\rangle) = q_0$
- ประเด็น : บางเครื่องวาด diagram ไม่ไหว ต้องเขียนแบบ formal

บทที่ S2

Regular Languages

อ้างอิง : Sipser 3rd ed. §1.1 (pp. 40–46)

S2.1 Formal definition of computation

การ accept

ให้ $M = (Q, \Sigma, \delta, q_0, F)$ และ $w = w_1 w_2 \cdots w_n$ M **accepts** w ถ้ามีลำดับ state $r_0, r_1, \ldots, r_n$ ที่

1. $r_0 = q_0$
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ ทุก $i = 0, \ldots, n-1$
3. $r_n \in F$

M **recognizes** A ถ้า $A = \{w \mid M \text{ accepts } w\}$

Definition 1.16

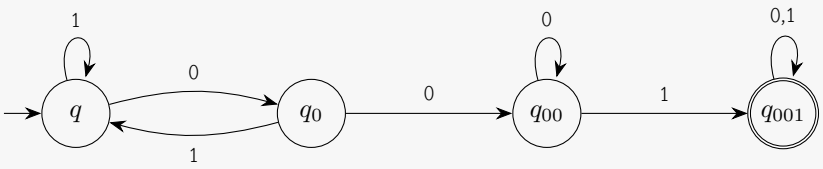
ภาษาเรียกว่า **regular language** ถ้ามี finite automaton บางตัว recognize มัน

- Example 1.17 : M_5 กับ $w = 10\langle\text{RESET}\rangle 22\langle\text{RESET}\rangle 012$
  - ลำดับ state คือ $q_0, q_1, q_1, q_0, q_2, q_1, q_0, q_0, q_1, q_0$
  - ครบ 3 เงื่อนไข จึง accept
  - $L(M_5)$ เป็น regular

S2.2 Designing finite automata

- วิธีคิด : **“reader as automaton”** สมมติตัวเองเป็นเครื่อง
  - เห็น input ทีละตัว
  - ไม่รู้ว่าจะจบเมื่อไร
  - ต้องพร้อมตอบเสมอ
- ถามว่า “ต้องจำอะไรบ้าง” ข้อมูลที่ต้องจำแต่ละแบบกลายเป็น 1 state แล้วใส่ transition, start state, accept state
- ตัวอย่าง E_1 : string ที่มี 1 จำนวนคี่
  - จำแค่ “คู่/คี่” : 2 state $q_{\text{even}}, q_{\text{odd}}$
  - อ่าน 1 สลับ, อ่าน 0 อยู่ที่เดิม
  - start q_{even}, accept q_{odd}

ตัวอย่าง : Example 1.21 : E_2 รับ string ที่มี 001 เป็น substring



- 4 ความเป็นไปได้ที่ต้องจำ : ยังไม่เห็นอะไร (q), เพิ่งเห็น 0 (q_0), เพิ่งเห็น 00 (q_{00}), เห็น 001 ครบแล้ว (q_{001})

S2.3 The regular operations

Definition 1.23

ให้ A, B เป็นภาษา **regular operations** คือ

- **Union** : $A \cup B = \{x \mid x \in A \text{ หรือ } x \in B\}$
- **Concatenation** : $A \circ B = \{xy \mid x \in A \text{ และ } y \in B\}$
- **Star** : $A^* = \{x_1 x_2 \cdots x_k \mid k \geq 0 \text{ และ } x_i \in A\}$ (unary operation ; $\varepsilon \in A^*$ เสมอ)

- Example 1.24 : $A = \{\texttt{good}, \texttt{bad}\}$, $B = \{\texttt{boy}, \texttt{girl}\}$
  $A \cup B = \{\texttt{good}, \texttt{bad}, \texttt{boy}, \texttt{girl}\}$ $A \circ B = \{\texttt{goodboy}, \texttt{goodgirl}, \texttt{badboy}, \texttt{badgirl}\}$
  $A^* = \{\varepsilon, \texttt{good}, \texttt{bad}, \texttt{goodgood}, \texttt{goodbad}, \texttt{badgood}, \texttt{badbad}, \dots\}$
- **Closed** : เซตปิดภายใต้ operation ถ้านำสมาชิกมาทำ operation แล้วผลยังอยู่ในเซต เช่น $\mathcal{N}$ ปิดภายใต้การคูณ แต่ไม่ปิดภายใต้การหาร

Theorem 1.25 : regular languages ปิดภายใต้ union

ให้ $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 และ $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ recognize A_2 สร้าง $M = (Q, \Sigma, \delta, q_0, F)$:

- $Q = Q_1 \times Q_2$
- $\delta((r_1, r_2), a) = (\delta_1(r_1, a), \delta_2(r_2, a))$
- $q_0 = (q_1, q_2)$
- $F = \{(r_1, r_2) \mid r_1 \in F_1 \text{ หรือ } r_2 \in F_2\} = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

- แนวคิด : รันสองเครื่อง**พร้อมกัน** โดยจำคู่ของ state (รันทีละเครื่องไม่ได้ เพราะย้อน input ไม่ได้)
- ระวัง : $F \neq F_1 \times F_2$ อันนั้นให้ intersection
- ถ้า alphabet ต่างกัน ใช้ $\Sigma = \Sigma_1 \cup \Sigma_2$
- Theorem 1.26 : regular languages ปิดภายใต้ concatenation พิสูจน์แบบเดิมไม่ได้ เพราะ M ไม่รู้ว่าต้องตัด input ตรงไหน ต้องใช้ nondeterminism (พิสูจน์จริงใน Theorem 1.47)

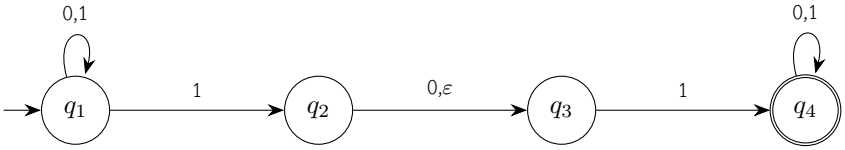
บทที่ S3

Non-deterministic Finite Automata (NFA)

อ้างอิง : Sipser 3rd ed. §1.2 (pp. 47–54)

S3.1 Nondeterminism

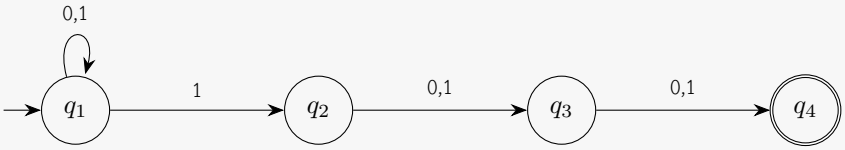
- **Deterministic** : รู้ state กับ input ถัดไป ก็รู้ state ถัดไปแน่นอน
- **Nondeterministic** : มีได้หลายทางเลือก
- ความต่างของ NFA จาก DFA :
  - แต่ละ state มีลูกศรออกสำหรับ symbol หนึ่ง **0, 1 หรือหลายเส้น**ก็ได้
  - มีลูกศรป้าย ε ได้ เดินได้โดยไม่อ่าน input
- การคำนวณ : เจอหลายทาง เครื่อง**แตกเป็นหลาย copy** เดินขนานกัน copy ที่ไม่มีทางไป ตาย เจอ ε arrow แตก copy ทันทีโดยไม่อ่าน input
- **accept ถ้ามี copy อย่างน้อยหนึ่งตัว**อยู่ที่ accept state ตอนจบ input
- มองได้ 3 แบบ : การคำนวณขนาน / tree ของความเป็นไปได้ / เครื่องที่ “เดาถูกเสมอ”



N_1 (Figure 1.27) : accept string ที่มี 101 หรือ 11 เป็น substring เช่น 010110 accept

- ทุก NFA แปลงเป็น DFA ที่เทียบเท่าได้ แต่ NFA มักเล็กกว่าและเข้าใจง่ายกว่า

ตัวอย่าง : Example 1.30 : N_2 ตัวที่ 3 จากท้ายเป็น 1

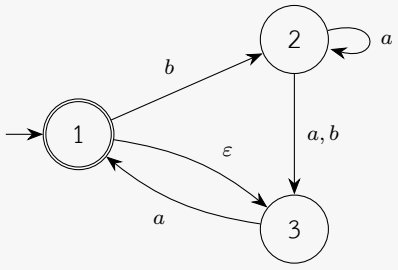


- อยู่ที่ q_1 จน “เดา” ว่าเหลืออีก 3 ตัว ถ้าตัวนั้นเป็น 1 ไป q_2 แล้วใช้ q_3, q_4 ตรวจว่าเดาถูก
- DFA ที่เทียบเท่าต้องมี **8 state** (จำ 3 ตัวล่าสุด)

ตัวอย่าง : Example 1.33 : N_3 บน unary alphabet $\{0\}$

- start state แตก ε ไป 2 วง : วงยาว 2 และวงยาว 3 accept 0^k เมื่อ k เป็นพหุคูณของ 2 หรือ 3
- accept : $\varepsilon, 00, 000, 0000, 000000$
- ไม่ accept : $0, 00000$
- แสดงประโยชน์ของ ε arrow

ตัวอย่าง : Example 1.35 : NFA 3 state (Figure 1.36)



- accept : $\varepsilon,\ a,\ baba,\ baa$
- ไม่ accept : $b,\ bb,\ babba$ (ใช้ต่อใน Example 1.41)

S3.2 Formal definition of a nondeterministic finite automaton

Definition 1.37

NFA คือ 5-tuple $(Q, \Sigma, \delta, q_0, F)$ เหมือน DFA ยกเว้น

$$\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q) \qquad \Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$$

N accepts w ถ้าเขียน $w = y_1 y_2 \cdots y_m$ $(y_i \in \Sigma_\varepsilon)$ และมี $r_0, \ldots, r_m$ ที่

1. $r_0 = q_0$
2. $r_{i+1} \in \delta(r_i, y_{i+1})$
3. $r_m \in F$

- **Example 1.38 :** N_1 แบบ formal $Q = \{q_1, q_2, q_3, q_4\}$, start q_1, $F = \{q_4\}$

	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

บทที่ S4

Equivalence of DFA and NFA

อ้างอิง : Sipser 3rd ed. §1.2 (pp. 54–58)

S4.1 Equivalence of NFAs and DFAs

- สองเครื่อง **equivalent** ถ้า recognize ภาษาเดียวกัน

Theorem 1.39

ทุก NFA มี DFA ที่ equivalent ให้ $N = (Q, \Sigma, \delta, q_0, F)$ สร้าง $M = (Q', \Sigma, \delta', q_0', F')$:

- $Q' = \mathcal{P}(Q)$ (N มี k state : M มี 2^k state)
- $E(R) = \{q \mid$ ไปถึง q ได้จากสมาชิกของ R ด้วย ε arrow 0 เส้นขึ้นไป$\}$
- $\delta'(R, a) = \{q \in Q \mid q \in E(\delta(r, a))$ สำหรับบาง $r \in R\} = \bigcup_{r \in R} E(\delta(r, a))$
- $q_0' = E(\{q_0\})$
- $F' = \{R \in Q' \mid R$ มี accept state ของ $N\}$

- แนวคิด : DFA จำ**เซตของ state ที่ copy ของ NFA อยู่ได้** ณ ขณะนั้น
- หนังสือพิสูจน์ 2 ขั้น : กรณีไม่มี ε ก่อน ($\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$, $q_0' = \{q_0\}$) แล้วเติม E
- Corollary 1.40 : ภาษาเป็น regular $\iff$ มี NFA recognize มัน

ตัวอย่าง : Example 1.41 : แปลง N_4 (NFA ของ Figure 1.36) เป็น DFA D

- $Q_D = \mathcal{P}(\{1, 2, 3\})$ มี 8 state
- start : $E(\{1\}) = \{1, 3\}$
- accept : เซตที่มี 1 คือ $\{1\}, \{1, 2\}, \{1, 3\}, \{1, 2, 3\}$

	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\to *\{1, 3\}$	$\{1, 3\}$	$\{2\}$
$\{2\}$	$\{2, 3\}$	$\{3\}$
$\{3\}$	$\{1, 3\}$	$\emptyset$
$\{2, 3\}$	$\{1, 2, 3\}$	$\{3\}$
$*\{1, 2, 3\}$	$\{1, 2, 3\}$	$\{2, 3\}$

- ไม่มีลูกศรชี้เข้า $\{1\}$ และ $\{1, 2\}$ ลบได้ เหลือ **6 state** (Figure 1.44)

บทที่ S5

Closure Properties of Regular Languages

อ้างอิง : Sipser 3rd ed. §1.2 (pp. 58–63)

S5.1 Closure under the regular operations

- ใช้ NFA พิสูจน์ใหม่ทั้ง 3 operation ง่ายกว่า product construction มาก

Theorem 1.45 : ปิดภายใต้ union

ให้ $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ สร้าง $N = (Q, \Sigma, \delta, q_0, F)$:

- $Q = \{q_0\} \cup Q_1 \cup Q_2$
- q_0 = start state ใหม่
- $F = F_1 \cup F_2$
- $\delta(q_0, \varepsilon) = \{q_1, q_2\}$, $\delta(q_0, a) = \emptyset$ เมื่อ $a \neq \varepsilon$
- state อื่นใช้ δ_1 / δ_2 ตามเดิม

Theorem 1.47 : ปิดภายใต้ concatenation

สร้าง $N = (Q_1 \cup Q_2, \Sigma, \delta, q_1, F_2)$:

- start state = q_1 ของ N_1
- accept states = F_2 ของ N_2
- $q \in F_1$: $\delta(q, \varepsilon) = \delta_1(q, \varepsilon) \cup \{q_2\}$ (เพิ่ม ε arrow จาก accept state ของ N_1 ไป start ของ N_2)
- ที่เหลือตามเดิม

Theorem 1.49 : ปิดภายใต้ star

สร้าง $N = (\{q_0\} \cup Q_1, \Sigma, \delta, q_0, \{q_0\} \cup F_1)$:

- q_0 = start state ใหม่ **ที่เป็น accept** (เพื่อรับ ε)
- $\delta(q_0, \varepsilon) = \{q_1\}$
- $q \in F_1$: $\delta(q, \varepsilon) = \delta_1(q, \varepsilon) \cup \{q_1\}$ (วนกลับ start เดิม)

ระวัง — จุดที่ผิดบ่อย

- star : ห้ามแค่ทำ start state เดิมให้เป็น accept เพราะอาจทำให้ accept string ที่ไม่ควรรับ ต้อง**เพิ่ม start state ใหม่**

บทที่ S6

Regular Expressions

อ้างอิง : Sipser 3rd ed. §1.3 (pp. 63–66)

S6.1 Regular expressions

- ใช้ regular operations สร้างนิพจน์บรรยายภาษา เช่น $(0 \cup 1)0^*$ = string ที่ขึ้นด้วย 0 หรือ 1 แล้วตามด้วย 0 กี่ตัวก็ได้
- ค่าของ regular expression คือ**ภาษา** ใช้ใน `grep`, `awk`, Perl, text editor
- **Example 1.51** : $(0 \cup 1)^*$ = string ทั้งหมดของ 0,1
  - เขียน Σ แทน $(0 \cup 1)$
  - Σ^* = ทุก string
  - Σ^*1 = ลงท้ายด้วย 1
  - $(0\Sigma^*) \cup (\Sigma^*1)$ = ขึ้นต้นด้วย 0 หรือลงท้ายด้วย 1
- **Precedence** : star ก่อน, แล้ว concatenation, แล้ว union (เว้นแต่มีวงเล็บ)

S6.2 Formal definition of a regular expression

Definition 1.52

R เป็น **regular expression** ถ้า R คือ

1. a สำหรับบาง $a \in \Sigma$ (ภาษา $\{a\}$)
2. ε (ภาษา $\{\varepsilon\}$)
3. $\emptyset$ (ภาษาว่าง)
4. $(R_1 \cup R_2)$
5. $(R_1 \circ R_2)$
6. (R_1^*)

โดย R_1, R_2 เป็น regular expression

- อย่าสับสน : ε แทนภาษาที่มี string เดียวคือ empty string
- $\emptyset$ แทนภาษาที่ไม่มี string เลย
- เป็น **inductive definition** (นิยามด้วยตัวที่เล็กกว่า) ไม่ใช่ circular
- ตัวย่อ : $R^+ = RR^*$ ดังนั้น $R^+ \cup \varepsilon = R^*$
- R^k = ต่อ R กัน k ครั้ง
- $L(R)$ = ภาษาของ R

ตัวอย่าง : Example 1.53 ($\Sigma = \{0, 1\}$)

1. $0^*10^* = \{w \mid w \text{ มี 1 ตัวเดียว}\}$
2. $\Sigma^*1\Sigma^* = \{w \mid w \text{ มี 1 อย่างน้อย 1 ตัว}\}$
3. $\Sigma^*001\Sigma^* = \{w \mid w \text{ มี 001 เป็น substring}\}$
4. $1^*(01^+)^* = \{w \mid 0 \text{ ทุกตัวใน } w \text{ ตามด้วย 1 อย่างน้อย 1 ตัว}\}$
5. $(\Sigma\Sigma)^* = \{w \mid |w| \text{ เป็นเลขคู่}\}$
6. $(\Sigma\Sigma\Sigma)^* = \{w \mid |w| \text{ เป็นพหุคูณของ 3}\}$
7. $01 \cup 10 = \{01, 10\}$

8. $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1 = \{w \mid w$ ขึ้นต้นและลงท้ายด้วย symbol เดียวกัน$\}$
9. $(0 \cup \varepsilon)1^* = 01^* \cup 1^*$
10. $(0 \cup \varepsilon)(1 \cup \varepsilon) = \{\varepsilon, 0, 1, 01\}$
11. $1^*\emptyset = \emptyset$ (concatenate กับเซตว่างได้เซตว่าง)
12. $\emptyset^* = \{\varepsilon\}$ (star ต่อ 0 ตัวได้ ε)

Identity

$R \cup \emptyset = R$ $R \circ \varepsilon = R$ แต่ $R \cup \varepsilon$ อาจ $\neq R$ (เช่น $R = 0$) $R \circ \emptyset$ อาจ $\neq R$ (เช่น $R = 0$ ได้ $\emptyset$)

- การใช้งาน : บรรยาย **token** ใน compiler
  - เช่นค่าคงที่ตัวเลข $(+ \cup - \cup \varepsilon)(D^+ \cup D^+.D^* \cup D^*.D^+)$,
  - $D = \{0, \ldots, 9\}$
  - สร้าง **lexical analyzer** อัตโนมัติจาก regular expression ได้

บทที่ S7

Equivalence of RE and DFA/NFA

อ้างอิง : Sipser 3rd ed. §1.3 (pp. 66–76)

S7.1 Equivalence with finite automata

Theorem 1.54

ภาษาเป็น regular $\iff$ มี regular expression บรรยายมัน (พิสูจน์เป็น 2 lemma)

Lemma 1.55 : regular expression $\Rightarrow$ regular

- แปลง R เป็น NFA ตาม 6 กรณีของนิยาม :
  1. $R = a$: 2 state $q_1 \xrightarrow{a} q_2$ (accept)
  2. $R = \varepsilon$: 1 state ที่เป็นทั้ง start และ accept
  3. $R = \emptyset$: 1 state ไม่มี accept
  4. $R_1 \cup R_2$,
  5. $R_1 \circ R_2$,
  6. R_1^* : ใช้ construction ของ Theorem 1.45, 1.47, 1.49
- **Example 1.56** : $(ab \cup a)^*$ สร้างจากเล็กไปใหญ่ : a, b, ab, $ab \cup a$, $(ab \cup a)^*$ ได้ NFA 8 state (ไม่เล็กสุด แต่ทำตามขั้นตอนได้เสมอ)
- **Example 1.58** : $(a \cup b)^*aba$ สร้าง a, b, $a \cup b$, $(a \cup b)^*$, aba แล้วต่อกัน

Lemma 1.60 : regular $\Rightarrow$ regular expression

- ทาง : DFA $\to$ GNFA $\to$ regular expression
- **Generalized NFA (GNFA)** : NFA ที่ป้ายบนลูกศรเป็น **regular expression** อ่าน input ทีละบล็อก
- รูปพิเศษที่ต้องใช้ :
  1. start state มีลูกศรไปทุก state ไม่มีลูกศรเข้า
  2. accept state มีตัวเดียว มีลูกศรจากทุก state ไม่มีลูกศรออก
  3. state อื่นทุกคู่มีลูกศรถึงกันทั้งสองทิศและถึงตัวเอง
- แปลง DFA เป็น GNFA : เพิ่ม start ใหม่ (ε ไป start เดิม)
  - เพิ่ม accept ใหม่ (ε จาก accept เดิมทุกตัว)
  - ลูกศรซ้ำรวมด้วย $\cup$
  - คู่ที่ไม่มีลูกศรใส่ $\emptyset$
- DFA มี k state : GNFA มี $k+2$ state ตัด state ออกทีละตัว (**rip**) จนเหลือ 2 state ป้ายที่เหลือคือคำตอบ

Definition 1.64 และ CONVERT

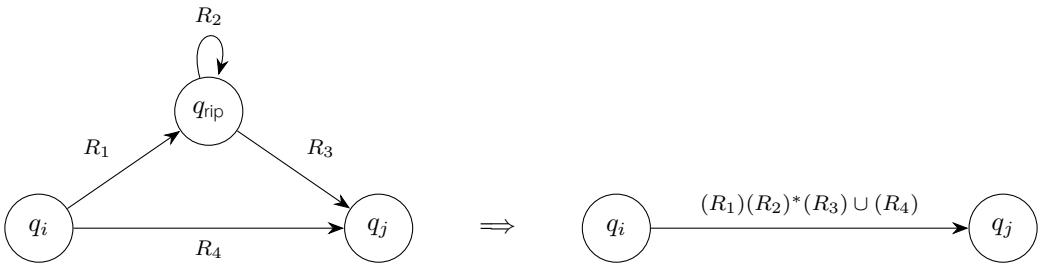
GNFA $= (Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$ $\delta : (Q - \{q_{\text{accept}}\}) \times (Q - \{q_{\text{start}}\}) \to \mathcal{R}$ ($\mathcal{R}$ = เซตของ regular expression)

CONVERT(G) : ให้ k = จำนวน state

1. ถ้า $k = 2$: คืนป้าย R ของลูกศรจาก q_{start} ไป q_{accept}
2. ถ้า $k > 2$: เลือก $q_{\text{rip}} \neq q_{\text{start}}, q_{\text{accept}}$ สร้าง G' ที่ $Q' = Q - \{q_{\text{rip}}\}$ และ

$$\delta'(q_i, q_j) = (R_1)(R_2)^*(R_3) \cup (R_4)$$

$R_1 = \delta(q_i, q_{\text{rip}}), \quad R_2 = \delta(q_{\text{rip}}, q_{\text{rip}}), \quad R_3 = \delta(q_{\text{rip}}, q_j), \quad R_4 = \delta(q_i, q_j)$ แล้วคืน CONVERT(G')



- **Claim 1.65** : ทุก GNFA G, CONVERT(G) equivalent กับ G พิสูจน์ด้วย induction บน k
- **Example 1.66** : DFA 2 state ($1 \xrightarrow{a} 1$, $1 \xrightarrow{b} 2$, $2 \xrightarrow{a,b} 2$, accept 2) ได้ $a^*b(a \cup b)^*$
- **Example 1.68** : DFA 3 state ($1 \xrightarrow{a} 2$, $1 \xrightarrow{b} 3$, $2 \xrightarrow{a} 1$, $2 \xrightarrow{b} 2$, $3 \xrightarrow{a} 2$, $3 \xrightarrow{b} 1$, accept 2,3) ตัด 1, 2, 3 ตามลำดับ ได้

$$(a(aa \cup b)^*ab \cup b)\big((ba \cup a)(aa \cup b)^*ab \cup bb\big)^*\big((ba \cup a)(aa \cup b)^* \cup \varepsilon\big) \cup a(aa \cup b)^*$$

บทที่ S8

Proving Non-regularity

อ้างอิง : Sipser 3rd ed. §1.4 (pp. 77–82)

S8.1 Nonregular languages

- $B = \{0^n 1^n \mid n \geq 0\}$: เครื่องต้องจำจำนวน 0 ซึ่งไม่จำกัด แต่ state มีจำกัด
- ระวัง : "ดูเหมือนต้องจำไม่จำกัด" ไม่ได้แปลว่า nonregular เสมอ
  - $C = \{w \mid w$ มี 0 และ 1 จำนวนเท่ากัน$\}$: nonregular
  - $D = \{w \mid w$ มี 01 และ 10 เป็น substring จำนวนเท่ากัน$\}$: **regular** (ขัดกับสัญชาตญาณ)
- จึงต้องมีเครื่องมือพิสูจน์ที่แน่นอน

S8.2 The pumping lemma for regular languages

Theorem 1.70 : Pumping lemma

ถ้า A เป็น regular แล้วมีจำนวน p (pumping length) ที่ทุก $s \in A$ ที่ $|s| \geq p$ แบ่งเป็น $s = xyz$ ได้ โดย

1. $xy^i z \in A$ ทุก $i \geq 0$
2. $|y| > 0$
3. $|xy| \leq p$

- y^i = ต่อ y กัน i ครั้ง $y^0 = \varepsilon$
- เงื่อนไข 2 : x หรือ z เป็น ε ได้ แต่ $y \neq \varepsilon$
- **พิสูจน์** : ให้ M เป็น DFA ที่ recognize A
  - ให้ p = จำนวน state
  - ให้ $s = s_1 \cdots s_n$, $n \geq p$
  - ลำดับ state $r_1, \ldots, r_{n+1}$ มี $n+1 \geq p+1$ ตัว
  - ใน $p+1$ ตัวแรกต้องมี 2 ตัวซ้ำ (pigeonhole principle)
  - ให้เป็น $r_j = r_l$, $j < l \leq p+1$

  ตั้ง $x = s_1 \cdots s_{j-1}$, $y = s_j \cdots s_{l-1}$, $z = s_l \cdots s_n$ ส่วน y พาเครื่องวนกลับ state เดิม จึงซ้ำกี่รอบก็ได้ $j \neq l$ ให้ $|y| > 0$ $l \leq p+1$ ให้ $|xy| \leq p$

วิธีใช้พิสูจน์ว่า B ไม่ regular

1. สมมติ B regular ได้ pumping length p
2. เลือก $s \in B$ ที่ $|s| \geq p$ (เลือกให้ "แสดงแก่นของความไม่ regular")
3. แสดงว่า**ทุกวิธีแบ่ง** $s = xyz$ ที่ตรงเงื่อนไข มี i ที่ $xy^i z \notin B$ ขัดแย้ง

ตัวอย่าง : Example 1.73–1.77

- 1.73 $B = \{0^n 1^n\}$: $s = 0^p 1^p$
  - กรณี y มีแต่ 0 : $xyyz$ มี 0 มากกว่า
  - กรณี y มีแต่ 1 : เช่นกัน
  - กรณี y มีทั้งคู่ : $xyyz$ ผิดลำดับ
  - (ใช้เงื่อนไข 3 จะเหลือกรณีเดียว)

- **1.74** $C = \{w \mid$ จำนวน 0 = จำนวน $1\}$: $s = 0^p1^p$ ต้องใช้เงื่อนไข 3 : $|xy| \leq p$ ทำให้ y มีแต่ 0 $xyyz \notin C$ ($s = (01)^p$ ใช้ไม่ได้ เพราะ pump ได้) ทางเลือก : ถ้า C regular แล้ว $C \cap 0^*1^* = B$ ต้อง regular ขัดแย้ง
- **1.75** $F = \{ww \mid w \in \{0,1\}^*\}$: $s = 0^p10^p1$ y มีแต่ 0 $xyyz \notin F$ ($s = 0^p0^p$ ใช้ไม่ได้)
- **1.76** $D = \{1^{n^2} \mid n \geq 0\}$: $s = 1^{p^2}$ $|xy^2z| \leq p^2 + p < (p+1)^2$ และ $> p^2$ จึงไม่เป็น perfect square
- **1.77** $E = \{0^i1^j \mid i > j\}$: $s = 0^{p+1}1^p$
  - y มีแต่ 0
  - pump ขึ้นไม่ช่วย
  - ต้อง pump down : $i = 0$ ได้ xz ที่จำนวน 0 $\leq$ จำนวน 1
  - $xz \notin E$

บทที่ S9

Context-Free Grammar (CFG)

อ้างอิง : Sipser 3rd ed. §2.1 (pp. 101–107)

S9.1 Context-free grammars

- บทที่ 2 แนะนำ **context-free grammar** : วิธีบรรยายภาษาที่แรงกว่า FA และ regular expression บรรยายโครงสร้างแบบ recursive ได้
- การใช้งาน : นิยามและ compile ภาษาโปรแกรม **parser** ดึงความหมายของโปรแกรมออกมาก่อน compile
- ตัวอย่าง G_1 :
  - $A \to 0A1$
  - $A \to B$
  - $B \to \#$
- grammar ประกอบด้วย **substitution rules** (**productions**)
  - ฝั่งซ้ายเป็น **variable**
  - ฝั่งขวาเป็น string ของ variable และ **terminal**
  - variable ของกฎแรกคือ **start variable**
- วิธีสร้าง string :
  1. เขียน start variable
  2. เลือก variable ที่เขียนไว้ แล้วแทนด้วยฝั่งขวาของกฎ
  3. ทำซ้ำจนไม่เหลือ variable
- ลำดับการแทนเรียก **derivation** เช่น $A \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000B111 \Rightarrow 000\#111$
- แสดงเป็นภาพด้วย **parse tree**
- $L(G_1) = \{0^n\#1^n \mid n \geq 0\}$
- ภาษาที่ CFG สร้างได้เรียก **context-free language (CFL)**
- ตัวอย่าง G_2 : ส่วนหนึ่งของภาษาอังกฤษ ⟨SENTENCE⟩ → ⟨NOUN-PHRASE⟩⟨VERB-PHRASE⟩ ฯลฯ สร้างได้เช่น `a boy sees`, `the boy sees a flower`

S9.2 Formal definition of a context-free grammar

Definition 2.2

Context-free grammar คือ 4-tuple (V, Σ, R, S)

1. V = เซตจำกัดของ **variables**
2. Σ = เซตจำกัดของ **terminals** $(V \cap \Sigma = \emptyset)$
3. R = เซตจำกัดของ **rules** แต่ละกฎคือ variable กับ string ของ variable และ terminal
4. $S \in V$ = **start variable**

- ถ้า $A \to w$ เป็นกฎ : uAv **yields** uwv เขียน $uAv \Rightarrow uwv$
- u **derives** v เขียน $u \overset{*}{\Rightarrow} v$ ถ้า $u = v$ หรือมี $u \Rightarrow u_1 \Rightarrow \cdots \Rightarrow u_k \Rightarrow v$
- **Language of the grammar** : $L(G) = \{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}$
- มักเขียนแค่กฎ
  - variable คือตัวที่อยู่ฝั่งซ้าย

- start variable คือฝั่งซ้ายของกฎแรก
- กฎฝั่งซ้ายเดียวกันรวมด้วย |

S9.3 Examples of context-free grammars

- **Example 2.3** : $G_3 = (\{S\}, \{a, b\}, R, S), \quad S \to aSb \mid SS \mid \varepsilon$
  สร้าง $abab$, $aaabbb$, $aababb$ มอง a เป็น "(" และ b เป็น ")" : $L(G_3)$ = วงเล็บที่ซ้อนกันถูกต้อง
- **Example 2.4** : $G_4 = (V, \Sigma, R, \langle \text{EXPR} \rangle), \quad \Sigma = \{a, +, \times, (,)\}$

$$\begin{aligned} \langle \text{EXPR} \rangle &\to \langle \text{EXPR} \rangle + \langle \text{TERM} \rangle \mid \langle \text{TERM} \rangle \\ \langle \text{TERM} \rangle &\to \langle \text{TERM} \rangle \times \langle \text{FACTOR} \rangle \mid \langle \text{FACTOR} \rangle \\ \langle \text{FACTOR} \rangle &\to (\langle \text{EXPR} \rangle) \mid a \end{aligned}$$

สร้าง $a + a \times a$ และ $(a + a) \times a$ parse tree จัดกลุ่ม $\times$ ก่อน $+$ ตามลำดับความสำคัญ

S9.4 Designing context-free grammars

- **เทคนิค 1 : แตกเป็น union** สร้าง grammar ของแต่ละชิ้น แล้วรวมด้วย $S \to S_1 \mid S_2 \mid \cdots \mid S_k$
  เช่น $\{0^n 1^n\} \cup \{1^n 0^n\}$: $S \to S_1 \mid S_2, \quad S_1 \to 0S_1 1 \mid \varepsilon, \quad S_2 \to 1S_2 0 \mid \varepsilon$
- **เทคนิค 2 : ภาษา regular**
  - สร้าง DFA ก่อนแล้วแปลง : variable R_i ต่อ state q_i
  - กฎ $R_i \to aR_j$ ถ้า $\delta(q_i, a) = q_j$
  - กฎ $R_i \to \varepsilon$ ถ้า q_i เป็น accept
  - start variable คือ R_0
- **เทคนิค 3 : substring ที่ "linked" กัน** (ต้องจำจำนวนไม่จำกัดของฝั่งหนึ่งเพื่อเทียบกับอีกฝั่ง เช่น $0^n 1^n$) ใช้กฎรูป $R \to uRv$
- **เทคนิค 4 : โครงสร้าง recursive** (เช่นนิพจน์ใน Example 2.4) วาง variable ที่สร้างโครงสร้างนั้นไว้ตรงตำแหน่งที่มันปรากฏซ้อนได้

บทที่ S10

Context-Free Languages (CFL)

อ้างอิง : Sipser 3rd ed. §2.1 (pp. 107–110)

S10.1 Ambiguity

- grammar อาจสร้าง string เดียวกันได้หลายแบบ : หลาย parse tree : หลายความหมาย ไม่ดีสำหรับภาษาโปรแกรม
- ตัวอย่าง G_5 : $\langle \text{EXPR} \rangle \to \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$
  สร้าง $a + a \times a$ ได้ 2 parse tree (G_5 ไม่บังคับลำดับความสำคัญ ต่างจาก G_4 ที่ไม่ ambiguous)
- derivation ต่างกันแต่ parse tree เดียวกันไม่นับว่า ambiguous จึงกำหนดลำดับการแทน :
- Leftmost derivation : ทุกขั้นแทน variable **ตัวซ้ายสุด**

Definition 2.7

w ถูก **derived ambiguously** ใน G ถ้ามี leftmost derivation ต่างกัน 2 แบบขึ้นไป G เป็น **ambiguous** ถ้าสร้างบาง string แบบ ambiguous

- บางครั้งหา grammar ที่ไม่ ambiguous สำหรับภาษาเดียวกันได้ แต่บางภาษาสร้างได้ด้วย grammar ที่ ambiguous เท่านั้น เรียก **inherently ambiguous** เช่น $\{a^i b^j c^k \mid i = j \text{ หรือ } j = k\}$

S10.2 Chomsky normal form

Definition 2.8

CFG อยู่ใน **Chomsky normal form (CNF)** ถ้าทุกกฎอยู่ในรูป

$$A \to BC \qquad \text{หรือ} \qquad A \to a$$

a เป็น terminal ; A, B, C เป็น variable ; B, C ห้ามเป็น start variable อนุญาต $S \to \varepsilon$ เมื่อ S เป็น start variable

Theorem 2.9

ทุก context-free language สร้างได้ด้วย CFG ใน Chomsky normal form

ขั้นตอนแปลง (จากบทพิสูจน์)

1. **เพิ่ม start variable ใหม่** $S_0 \to S$ (รับประกันว่า start variable ไม่อยู่ฝั่งขวา)
2. **ตัด** ε**-rule** $A \to \varepsilon$ (A ไม่ใช่ start) : ทุกกฎที่มี A ฝั่งขวา เพิ่มกฎที่ลบ A ออก (ทุกชุดของตำแหน่ง)
   เช่น $R \to uAvAw$ เพิ่ม $R \to uvAw \mid uAvw \mid uvw$ ถ้ามี $R \to A$ เพิ่ม $R \to \varepsilon$ (เว้นแต่เคยตัดไปแล้ว)
3. **ตัด unit rule** $A \to B$: ทุกกฎ $B \to u$ เพิ่ม $A \to u$ (เว้นแต่เป็น unit rule ที่ตัดไปแล้ว)
4. **แปลงกฎที่เหลือ** : $A \to u_1 u_2 \cdots u_k$, $k \geq 3$ แทนด้วย $A \to u_1 A_1$, $A_1 \to u_2 A_2$, ..., $A_{k-2} \to u_{k-1} u_k$
   terminal u_i ในกฎที่ยาว ≥ 2 แทนด้วย variable ใหม่ U_i พร้อมกฎ $U_i \to u_i$

ตัวอย่าง : Example 2.10 : แปลง G_6 เป็น CNF

- G_6 : $S \to ASA \mid aB$
- $A \to B \mid S$
- $B \to b \mid \varepsilon$

- ขั้น 1 : เพิ่ม $S_0 \to S$
- ขั้น 2 : ตัด $B \to \varepsilon$ ได้ $S \to ASA \mid aB \mid a, \quad A \to B \mid S \mid \varepsilon$ แล้วตัด $A \to \varepsilon$ ได้ $S \to ASA \mid aB \mid a \mid SA \mid AS \mid S$
- ขั้น 3 : ตัด $S \to S$,
  - $S_0 \to S$,
  - $A \to B$,
  - $A \to S$
- ขั้น 4 : แตกกฎยาว ผลลัพธ์

$$S_0 \to AA_1 \mid UB \mid a \mid SA \mid AS \qquad S \to AA_1 \mid UB \mid a \mid SA \mid AS$$
$$A \to b \mid AA_1 \mid UB \mid a \mid SA \mid AS \qquad A_1 \to SA \qquad U \to a \qquad B \to b$$

★ Claude สอนเพิ่ม : ทำไม CNF สำคัญ

- ใน CNF ทุก derivation ของ string ยาว $n \geq 1$ ใช้ $2n - 1$ **ขั้นพอดี** ใช้พิสูจน์ว่า A_{CFG} decidable (Theorem 4.7)

บทที่ S11

Pushdown Automata (PDA)

อ้างอิง : Sipser 3rd ed. §2.2 (pp. 112–116)

S11.1 Pushdown automata

- **Pushdown automaton** = NFA + **stack** stack ให้ความจำไม่จำกัด แต่เข้าถึงได้แค่ตัวบนสุด (last in, first out)
- **Push** = เขียนลงบน stack
- **Pop** = เอาตัวบนสุดออก
- PDA กำลังเท่ากับ CFG จึงพิสูจน์ว่าภาษาเป็น context-free ได้ 2 ทาง : ให้ CFG หรือให้ PDA
- $\{0^n 1^n\}$: push 0 ทุกตัวที่อ่าน เจอ 1 ให้ pop 0 ทีละตัว input หมดพร้อม stack ว่าง : accept
- PDA เป็น **nondeterministic** ได้
  - และ **deterministic PDA กำลังน้อยกว่า** nondeterministic PDA
  - (ต่างจาก FA)
  - เล่มนี้เน้นแบบ nondeterministic เพราะเท่ากับ CFG

S11.2 Formal definition of a pushdown automaton

Definition 2.13

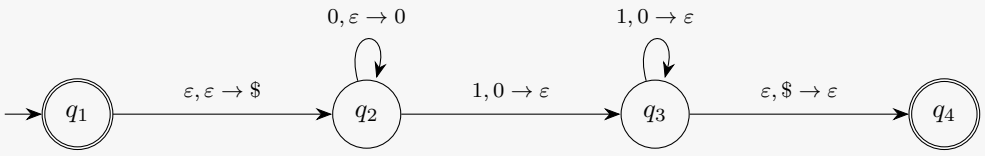
PDA คือ 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ ทุกเซตจำกัด

1. Q = states
2. Σ = input alphabet
3. Γ = **stack alphabet**
4. $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathcal{P}(Q \times \Gamma_\varepsilon)$
5. $q_0 \in Q$
6. $F \subseteq Q$

- M **accepts** w ถ้าเขียน $w = w_1 \cdots w_m$ ($w_i \in \Sigma_\varepsilon$) และมี state $r_0, \ldots, r_m$ กับ string $s_0, \ldots, s_m \in \Gamma^*$ (เนื้อหา stack) ที่
  1. $r_0 = q_0$ และ $s_0 = \varepsilon$ (เริ่มด้วย stack ว่าง)
  2. $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)$ โดย $s_i = at$, $s_{i+1} = bt$ ($a, b \in \Gamma_\varepsilon$, $t \in \Gamma^*$)
  3. $r_m \in F$
- ป้าย $a, b \to c$: อ่าน a จาก input, pop b, push c
  - $a = \varepsilon$ ไม่อ่าน input
  - $b = \varepsilon$ ไม่ pop
  - $c = \varepsilon$ ไม่ push
- PDA ทดสอบ stack ว่างตรง ๆ ไม่ได้ : push $ ไว้ก่อน แล้วเจอ $ อีกครั้งคือ stack ว่าง ทดสอบจบ input ไม่ได้เช่นกัน : accept มีผลเฉพาะเมื่ออยู่ที่ accept state ตอน input หมด

S11.3 Examples of pushdown automata

ตัวอย่าง : Example 2.14 : M_1 **รับ** $\{0^n1^n \mid n \geq 0\}$

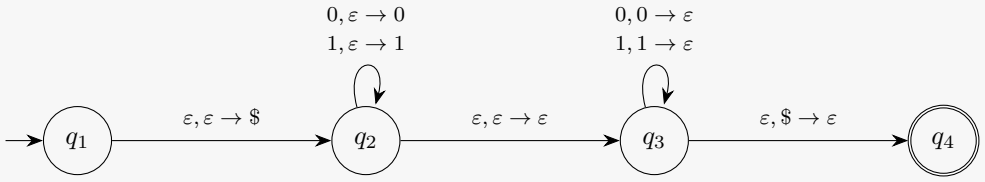


- $Q = \{q_1, q_2, q_3, q_4\}$,
  - $\Sigma = \{0, 1\}$,
  - $\Gamma = \{0, \$\}$,
  - $F = \{q_1, q_4\}$ (q_1 accept เพื่อรับ ε)

ตัวอย่าง : Example 2.16 : M_2 **รับ** $\{a^i b^j c^k \mid i, j, k \geq 0$ **และ** $i = j$ **หรือ** $i = k\}$

- push \$ แล้ว push a ทุกตัว จากนั้น**เดา**แบบ nondeterministic ว่าจะจับคู่ a กับ b หรือกับ c
- กิ่ง $i = j$: pop a ต่อ b หนึ่งตัว เจอ \$ แล้วอ่าน c ที่เหลือ
- กิ่ง $i = k$: อ่าน b ข้ามไป pop a ต่อ c หนึ่งตัว เจอ \$ แล้ว accept
- **nondeterminism จำเป็น**สำหรับภาษานี้

ตัวอย่าง : Example 2.18 : M_3 **รับ** $\{ww^{\mathcal{R}} \mid w \in \{0, 1\}^*\}$



- push ครึ่งแรก **เดา**ว่าถึงกึ่งกลางแล้ว ($\varepsilon, \varepsilon \to \varepsilon$) pop เทียบกับครึ่งหลัง

บทที่ S12

Equivalence of PDA and CFG

อ้างอิง : Sipser 3rd ed. §2.2 (pp. 117–124)

S12.1 Equivalence with context-free grammars

Theorem 2.20

ภาษาเป็น context-free $\iff$ มี pushdown automaton recognize มัน

Lemma 2.21 : context-free $\Rightarrow$ มี PDA

- แนวคิด : PDA P จำลอง derivation stack เก็บ **intermediate string** ส่วนที่เริ่มจาก variable ตัวแรก terminal ที่อยู่ข้างหน้าถูกจับคู่กับ input ทันที
- คำอธิบายแบบ informal ของ P :
  1. push \$ และ start variable ลง stack
  2. ทำซ้ำ :
     (a) บนสุดเป็น variable A : เลือกกฎของ A แบบ nondeterministic แล้วแทน A ด้วยฝั่งขวา
     (b) บนสุดเป็น terminal a : อ่าน input ตัวถัดไปเทียบกับ a ตรงให้ทำต่อ ไม่ตรงให้ reject กิ่งนี้
     (c) บนสุดเป็น \$: เข้า accept state (accept ถ้า input หมดแล้ว)
- ต้อง push ทั้ง string $u = u_1 \cdots u_l$ ในครั้งเดียว : ใช้ state เสริม เขียนย่อ $(r, u) \in \delta(q, a, s)$

Construction

$Q = \{q_{\text{start}}, q_{\text{loop}}, q_{\text{accept}}\} \cup E$ (E = state เสริมสำหรับ push หลายตัว)

- $\delta(q_{\text{start}}, \varepsilon, \varepsilon) = \{(q_{\text{loop}}, S\$)\}$
- $\delta(q_{\text{loop}}, \varepsilon, A) = \{(q_{\text{loop}}, w) \mid A \to w \text{ เป็นกฎใน } R\}$
- $\delta(q_{\text{loop}}, a, a) = \{(q_{\text{loop}}, \varepsilon)\}$
- ทุก terminal a
- $\delta(q_{\text{loop}}, \varepsilon, \$) = \{(q_{\text{accept}}, \varepsilon)\}$

- Example 2.25 : G : $S \to aTb \mid b$, $T \to Ta \mid \varepsilon$
  q_{loop} มี loop : $\varepsilon, S \to aTb$ (ใช้ state เสริม) $\varepsilon, S \to b$ $\varepsilon, T \to Ta$ $\varepsilon, T \to \varepsilon$ $a, a \to \varepsilon$ $b, b \to \varepsilon$

Lemma 2.27 : มี PDA $\Rightarrow$ context-free

- แนวคิด : ทุกคู่ state p, q มี variable A_{pq} ที่สร้าง**ทุก string ที่พา P จาก p (stack ว่าง) ไป q (stack ว่าง)**
- ปรับ P ก่อนให้มี 3 คุณสมบัติ :
  1. มี accept state เดียว q_{accept}
  2. ทำ stack ให้ว่างก่อน accept
  3. ทุก transition **push หรือ pop อย่างใดอย่างหนึ่ง** ไม่ทำทั้งคู่และไม่ทำเปล่า
- วิธีปรับข้อ 3 : transition ที่ทั้ง pop และ push แตกเป็น 2 ขั้นผ่าน state ใหม่ transition ที่ไม่ทำอะไรกับ stack แตกเป็น push แล้ว pop สัญลักษณ์อะไรก็ได้
- การคำนวณจาก p ถึง q : ขั้นแรกต้อง push ขั้นสุดท้ายต้อง pop มี 2 กรณี
  - ตัวที่ pop ตอนท้ายคือตัวที่ push ตอนแรก (stack ว่างเฉพาะต้นกับปลาย) : กฎ $A_{pq} \to aA_{rs}b$
  - stack ว่างกลางทางที่ state r : กฎ $A_{pq} \to A_{pr}A_{rq}$

Construction

ให้ $P = (Q, \Sigma, \Gamma, \delta, q_0, \{q_{\text{accept}}\})$ สร้าง G ที่มี variables $\{A_{pq} \mid p, q \in Q\}$, start variable $A_{q_0, q_{\text{accept}}}$ และกฎ

1. ทุก $p, q, r, s \in Q$, $u \in \Gamma$, $a, b \in \Sigma_\varepsilon$: ถ้า $(r, u) \in \delta(p, a, \varepsilon)$ และ $(q, \varepsilon) \in \delta(s, b, u)$ ใส่ $A_{pq} \to aA_{rs}b$
2. ทุก $p, q, r \in Q$: ใส่ $A_{pq} \to A_{pr}A_{rq}$
3. ทุก $p \in Q$: ใส่ $A_{pp} \to \varepsilon$

- **Claim 2.30** : ถ้า A_{pq} สร้าง x แล้ว x พา P จาก p (stack ว่าง) ไป q (stack ว่าง) induction บนจำนวนขั้นของ derivation
- **Claim 2.31** : ถ้า x พา P จาก p (stack ว่าง) ไป q (stack ว่าง) แล้ว A_{pq} สร้าง x induction บนจำนวนขั้นของการคำนวณ
- **Corollary 2.32** : ทุก regular language เป็น context-free (FA คือ PDA ที่ไม่ใช้ stack)
- ภาพรวม (Figure 2.33) : regular languages $\subsetneq$ context-free languages

บทที่ S13

Non-Context-Free Languages

อ้างอิง : Sipser 3rd ed. §2.3 (pp. 125–129)

S13.1 The pumping lemma for context-free languages

Theorem 2.34

ถ้า A เป็น context-free แล้วมี p (pumping length) ที่ทุก $s \in A$ ที่ $|s| \geq p$ แบ่งเป็น $s = uvxyz$ ได้ โดย

1. $uv^ixy^iz \in A$ ทุก $i \geq 0$
2. $|vy| > 0$
3. $|vxy| \leq p$

- เงื่อนไข 2 : v หรือ y ตัวใดตัวหนึ่งไม่ว่าง
- เงื่อนไข 3 : ช่วง v, x, y รวมกันยาวไม่เกิน p
- **Proof idea** : s ยาวมาก : parse tree สูงมาก : มี path ยาวจาก root ถึง leaf path นั้นต้องมี variable R **ซ้ำ** (pigeonhole) แทน subtree ใต้ R ตัวล่างด้วย subtree ใต้ R ตัวบนได้ซ้ำ ๆ (pump up) หรือกลับกัน (pump down)
- **Proof** : ให้ b = จำนวน symbol สูงสุดในฝั่งขวาของกฎ ($b \geq 2$) tree สูง h มี leaf ไม่เกิน b^h ตั้ง

$$p = b^{|V|+1}$$

$|s| \geq p > b^{|V|}$: parse tree สูงอย่างน้อย $|V|+1$ เลือก parse tree ที่มี node น้อยสุด path ยาวสุดมี variable อย่างน้อย $|V|+1$ ตัว : มี R ซ้ำใน $|V|+1$ ตัวล่างสุด

- เงื่อนไข 1 : สลับ subtree ได้
- เงื่อนไข 2 : ถ้า $v = y = \varepsilon$ จะได้ tree ที่เล็กกว่า ขัดกับการเลือก tree เล็กสุด
- เงื่อนไข 3 : subtree ใต้ R ตัวบนสูงไม่เกิน $|V|+1$ จึงสร้าง string ยาวไม่เกิน $b^{|V|+1} = p$

ตัวอย่าง : Example 2.36–2.38

- 2.36 $B = \{a^nb^nc^n \mid n \geq 0\}$: $s = a^pb^pc^p$
  - กรณี 1 : v และ y แต่ละตัวมี symbol ชนิดเดียว : uv^2xy^2z มี a, b, c ไม่เท่ากัน (เพิ่มได้ไม่เกิน 2 ชนิด)
  - กรณี 2 : v หรือ y มีมากกว่า 1 ชนิด : uv^2xy^2z ผิดลำดับ
- 2.37 $C = \{a^ib^jc^k \mid 0 \leq i \leq j \leq k\}$: $s = a^pb^pc^p$ ต้อง pump ทั้งขึ้นและลง
  - กรณี 1 (v, y มีชนิดเดียวต่อตัว) : มีอย่างน้อย 1 ชนิดที่ไม่อยู่ใน v, y
    - ไม่มี a : pump ลง uv^0xy^0z b หรือ c ลดลง แต่ a เท่าเดิม
    - ไม่มี b : ถ้ามี a pump ขึ้นได้ a มากกว่า b ถ้ามี c pump ลงได้ b มากกว่า c
    - ไม่มี c : pump ขึ้น a หรือ b มากกว่า c
  - กรณี 2 : v หรือ y มีมากกว่า 1 ชนิด : ผิดลำดับ
- 2.38 $D = \{ww \mid w \in \{0,1\}^*\}$: $s = 0^p1^p0^p1^p$ ($s = 0^p10^p1$ ใช้ไม่ได้ เพราะ pump ได้)
  - vxy อยู่ครึ่งแรก : uv^2xy^2z ดัน 1 ไปอยู่ตำแหน่งแรกของครึ่งหลัง ไม่อยู่ในรูป ww
  - vxy อยู่ครึ่งหลัง : ดัน 0 ไปอยู่ตำแหน่งสุดท้ายของครึ่งแรก
  - vxy คร่อมกึ่งกลาง : pump ลงได้ $0^p1^i0^j1^p$ ที่ i, j ไม่เป็น p ทั้งคู่ ไม่อยู่ในรูป ww

บทที่ S14

Turing Machines

อ้างอิง : Sipser 3rd ed. §3.1 (pp. 165–175)

S14.1 Turing machines

- เสนอโดย Alan Turing (1936) คล้าย FA แต่มี**ความจำไม่จำกัดและไม่มีข้อจำกัดการเข้าถึง** เป็น model ของคอมพิวเตอร์ทั่วไป
- มี **tape** ยาวไม่สิ้นสุด และ **tape head** ที่อ่าน เขียน และเลื่อนได้ เริ่มต้น tape มีแค่ input ที่เหลือเป็น **blank** (␣)
- ความต่างจาก finite automata :
  1. TM ทั้ง**เขียน**และอ่าน tape ได้
  2. head เลื่อนได้ทั้ง**ซ้ายและขวา**
  3. tape **ไม่สิ้นสุด**
  4. state พิเศษสำหรับ accept และ reject **มีผลทันที**
- ตัวอย่างเปิดบท : M_1 สำหรับ $B = \{w\#w \mid w \in \{0,1\}^*\}$
  M_1 = “On input w :
  1. วิ่งซิกแซกข้าม # เทียบตำแหน่งที่ตรงกันของสองฝั่ง ไม่ตรงหรือไม่มี # ให้ reject ขีดฆ่าตัวที่เทียบแล้ว
  2. เมื่อฝั่งซ้ายถูกขีดหมด ตรวจว่าฝั่งขวาไม่เหลือ เหลือให้ reject ไม่เหลือให้ accept”

S14.2 Formal definition of a Turing machine

Definition 3.3

Turing machine คือ 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ Q, Σ, Γ จำกัด

1. Q = states
2. Σ = input alphabet **ไม่มี** ␣
3. Γ = tape alphabet, $\text{␣} \in \Gamma$, $\Sigma \subseteq \Gamma$
4. $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\text{L}, \text{R}\}$
5. q_0 = start state
6. q_{accept}
7. q_{reject}, $q_{\text{reject}} \neq q_{\text{accept}}$

- $\delta(q, a) = (r, b, \text{L})$: อยู่ state q อ่าน a : เขียน b ทับ ไป state r เลื่อนซ้าย
- head อยู่ซ้ายสุดแล้วสั่งเลื่อนซ้าย : อยู่ที่เดิม
- คำนวณต่อจนเข้า accept หรือ reject state ไม่เข้าก็ทำไปตลอดกาล
- **Configuration** : สถานะปัจจุบัน + เนื้อหา tape + ตำแหน่ง head เขียน $u\,q\,v$: tape คือ uv, state q, head อยู่ที่ตัวแรกของ v เช่น $1011\,q_7\,01111$
- C_1 **yields** C_2 : ไปได้ในขั้นเดียว ให้ $a, b, c \in \Gamma$, $u, v \in \Gamma^*$

Yields

$\delta(q_i, b) = (q_j, c, \text{L})$: $ua\,q_i\,bv$ yields $u\,q_j\,acv$ $\delta(q_i, b) = (q_j, c, \text{R})$: $ua\,q_i\,bv$ yields $uac\,q_j\,v$

ขอบซ้าย : $q_i\,bv$ yields $q_j\,cv$ (เลื่อนซ้าย) หรือ $c\,q_j\,v$ (เลื่อนขวา) ขอบขวา : $ua\,q_i$ เท่ากับ $ua\,q_i$␣

- **Start configuration** : $q_0\, w$
- **Accepting configuration** : state เป็น q_{accept}
- **Rejecting configuration** : state เป็น q_{reject} ทั้งสองเป็น **halting configurations**
- M **accepts** w ถ้ามี $C_1, \ldots, C_k$: C_1 เป็น start configuration, C_i yields C_{i+1}, C_k เป็น accepting configuration
- $L(M)$ = เซตของ string ที่ M accept = **language recognized by** M

Definition 3.5 และ 3.6

- ภาษาเป็น **Turing-recognizable** ถ้ามี TM recognize มัน (ชื่ออื่น : recursively enumerable)
- ภาษาเป็น **Turing-decidable** (decidable) ถ้ามี TM **decide** มัน (ชื่ออื่น : recursive)

- ผลลัพธ์ของ TM มี 3 แบบ : **accept, reject, loop** (loop = ไม่หยุด)
- TM ที่หยุดทุก input เรียก **decider** decider ที่ recognize ภาษา เรียกว่า **decide** ภาษานั้น
- ทุก decidable language เป็น Turing-recognizable

S14.3 Examples of Turing machines

ตัวอย่าง : Example 3.7 : M_2 **decide** $A = \{0^{2^n} \mid n \geq 0\}$

M_2 = "On input w :

1. กวาดซ้ายไปขวา ขีดฆ่า 0 ตัวเว้นตัว
2. ถ้าในขั้น 1 tape มี 0 ตัวเดียว : *accept*
3. ถ้ามี 0 มากกว่า 1 ตัวและเป็นจำนวนคี่ : *reject*
4. กลับ head ไปซ้ายสุด
5. กลับไปขั้น 1"

- แต่ละรอบหารจำนวน 0 ด้วย 2
- $Q = \{q_1, \ldots, q_5, q_{\text{accept}}, q_{\text{reject}}\}$, $\Sigma = \{0\}$, $\Gamma = \{0, x, \sqcup\}$

	อ่าน 0	อ่าน x	อ่าน $\sqcup$
q_1	$(q_2, \sqcup, \text{R})$	$(q_{\text{reject}}, x, \text{R})$	$(q_{\text{reject}}, \sqcup, \text{R})$
q_2	(q_3, x, R)	(q_2, x, R)	$(q_{\text{accept}}, \sqcup, \text{R})$
q_3	$(q_4, 0, \text{R})$	(q_3, x, R)	$(q_5, \sqcup, \text{L})$
q_4	(q_3, x, R)	(q_4, x, R)	$(q_{\text{reject}}, \sqcup, \text{R})$
q_5	$(q_5, 0, \text{L})$	(q_5, x, L)	$(q_2, \sqcup, \text{R})$

- เขียน $\sqcup$ ทับ 0 ตัวซ้ายสุดเพื่อใช้หาขอบซ้ายของ tape
- รันบน 0000 : $q_1\,0000$, $\sqcup q_2\,000$, $\sqcup x\,q_3\,00$, $\sqcup x0\,q_4\,0$, $\sqcup x0x\,q_3\,\sqcup$, $\sqcup x0\,q_5\,x$, ..., $\sqcup xxx\,q_2\,\sqcup$, $\sqcup xxx\,\sqcup\, q_{\text{accept}}$ (21 ขั้น)

ตัวอย่าง : Example 3.9 : M_1 **แบบ formal decide** $B = \{w\#w\}$

- $Q = \{q_1, \ldots, q_8, q_{\text{accept}}, q_{\text{reject}}\}$,
  - $\Sigma = \{0, 1, \#\}$,
  - $\Gamma = \{0, 1, \#, x, \sqcup\}$
  - ช่องที่ไม่ได้เขียนไป q_{reject}

	0	1	#	x	⊔
q_1	(q_2, x, R)	(q_3, x, R)	$(q_8, \#, \mathsf{R})$		
q_2	R	R	$(q_4, \#, \mathsf{R})$		
q_3	R	R	$(q_5, \#, \mathsf{R})$		
q_4	(q_6, x, L)			R	
q_5		(q_6, x, L)		R	
q_6	L	L	$(q_7, \#, \mathsf{L})$	L	
q_7	L	L		(q_1, x, R)	
q_8				R	$(q_{\text{accept}}, ⊔, \mathsf{R})$

(R หรือ L เดี่ยว ๆ = อยู่ state เดิม ไม่เปลี่ยน tape แล้วเลื่อน)

- q_2, q_4 = จำว่าตัวที่ขีดคือ 0
- q_3, q_5 = จำว่าเป็น 1
- q_6, q_7 = เดินกลับ
- q_8 = ตรวจว่าฝั่งขวาหมด

ตัวอย่าง : Example 3.11 : M_3 decide $C = \{a^i b^j c^k \mid i \times j = k$ และ $i, j, k \geq 1\}$

1. กวาดซ้ายไปขวา ตรวจว่าอยู่ในรูป $a^+b^+c^+$ ไม่ใช่ให้ *reject*
2. กลับ head ไปซ้ายสุด
3. ขีดฆ่า a หนึ่งตัว
   - วิ่งไปหา b
   - สลับไปมาระหว่าง b กับ c ขีดฆ่าอย่างละตัวจน b หมด
   - ถ้า c หมดก่อน b ให้ *reject*
4. คืนค่า b ที่ขีดไว้ ถ้ายังมี a กลับไปขั้น 3 ถ้า a หมด ตรวจว่า c ถูกขีดหมด : หมดให้ *accept* ไม่หมดให้ *reject*

- หาขอบซ้ายของ tape : ทำเครื่องหมายบนตัวซ้ายสุด หรือใช้ความจริงที่ว่าเลื่อนซ้ายที่ขอบแล้ว head ไม่ขยับ

ตัวอย่าง : Example 3.12 : M_4 แก้ element distinctness problem

$E = \{\#x_1\#x_2\#\cdots\#x_l \mid x_i \in \{0,1\}^*$ และ $x_i \neq x_j$ ทุก $i \neq j\}$

1. ทำ mark บนตัวซ้ายสุด
   - ถ้าเป็น ⊔ ให้ *accept*
   - ถ้าเป็น # ทำต่อ
   - อย่างอื่น *reject*
2. วิ่งขวาไป # ตัวถัดไป ทำ mark ตัวที่สอง ถ้าไม่มี (มีแค่ x_1) ให้ *accept*
3. เทียบ string ทางขวาของ # ที่ mark ทั้งสองแบบซิกแซก ถ้าเท่ากัน *reject*
4. เลื่อน mark ตัวขวาไป # ถัดไป ถ้าไม่มี เลื่อน mark ตัวซ้ายไป # ถัดไป และ mark ตัวขวาไป # หลังจากนั้น ถ้าไม่มีให้ตัวขวา แปลว่าเทียบครบแล้ว *accept*
5. กลับไปขั้น 3

- เทคนิค **marking** : เพิ่ม symbol # ใน tape alphabet ทำได้กับ symbol ใดก็ได้

บทที่ S15

Variants of Turing Machines

อ้างอิง : Sipser 3rd ed. §3.2 (pp. 176–181)

S15.1 Variants of Turing machines

- มีนิยามแบบอื่นของ TM หลายแบบ ทั้งหมด**กำลังเท่ากับแบบเดิม** : recognize ภาษา class เดียวกัน คุณสมบัตินี้เรียก robustness
- ตัวอย่าง : อนุญาตให้ head **อยู่กับที่** $\delta : Q \times \Gamma \to Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}, \mathrm{S}\}$ ไม่เพิ่มกำลัง เพราะแทน S ด้วยการเลื่อนขวาแล้วเลื่อนซ้ายได้
- จะแสดงว่าสอง model เท่ากัน : แสดงว่าแต่ละตัว **simulate** อีกตัวได้

S15.2 Multitape Turing machines

- มีหลาย tape แต่ละ tape มี head ของตัวเอง input อยู่บน tape 1 ที่เหลือว่าง $\delta : Q \times \Gamma^k \to Q \times \Gamma^k \times \{\mathrm{L}, \mathrm{R}, \mathrm{S}\}^k$

Theorem 3.13

ทุก multitape TM มี single-tape TM ที่ equivalent

- **พิสูจน์** : S เก็บเนื้อหา k tape ต่อกันบน tape เดียว คั่นด้วย # ใช้ symbol ที่มี**จุดข้างบน**บอกตำแหน่ง head เสมือนของแต่ละ tape
  S = "On input $w = w_1 \cdots w_n$:
  1. จัด tape ให้เป็น $\#\dot{w}_1 w_2 \cdots w_n \# \dot{\sqcup} \# \dot{\sqcup} \# \cdots \#$
  2. จำลอง 1 ขั้น : กวาดจาก # ตัวแรกถึงตัวที่ $k+1$ เพื่ออ่าน symbol ใต้ head เสมือนทุกตัว แล้วกวาดอีกรอบเพื่ออัปเดตตาม δ ของ M
  3. ถ้า head เสมือนเลื่อนขวาไปชน # : เขียน $\sqcup$ แล้วเลื่อนเนื้อหาที่เหลือทั้งหมดไปขวา 1 ช่อง"
- Corollary 3.15 : ภาษาเป็น Turing-recognizable $\iff$ มี multitape TM recognize มัน

S15.3 Nondeterministic Turing machines

- $\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{\mathrm{L}, \mathrm{R}\})$ การคำนวณเป็น tree accept ถ้า**มีกิ่งใดกิ่งหนึ่ง**ถึง accept state

Theorem 3.16

ทุก nondeterministic TM มี deterministic TM ที่ equivalent

- **Proof idea** : D ลองทุกกิ่งของ N ต้องค้นแบบ breadth-first (depth-first อาจติดในกิ่งที่ไม่จบแล้วพลาดกิ่งที่ accept)
- **Proof** : D มี 3 tape
  - tape 1 : input ไม่เคยเปลี่ยน
  - tape 2 : สำเนา tape ของ N บนกิ่งที่กำลังจำลอง
  - tape 3 : address ของ node ใน tree เป็น string บน $\Gamma_b = \{1, \ldots, b\}$ (b = จำนวนทางเลือกสูงสุดของ δ) เช่น 231 = ลูกคนที่ 2 แล้ว 3 แล้ว 1

  1. tape 1 มี w tape 2, 3 ว่าง
  2. copy tape 1 ไป tape 2 ตั้ง tape 3 เป็น ε
  3. จำลอง N บน tape 2 โดยเลือกทางตาม symbol บน tape 3 ถ้า symbol หมด หรือทางเลือกไม่มี หรือเจอ rejecting

configuration ไปขั้น 4 เจอ accepting configuration ให้ *accept*

4. เปลี่ยน tape 3 เป็น string ถัดไปตาม string ordering กลับไปขั้น 2

- Corollary 3.18 : ภาษาเป็น Turing-recognizable $\iff$ มี nondeterministic TM recognize มัน
- NTM เป็น decider ถ้า**ทุกกิ่งหยุดบนทุก input**
- Corollary 3.19 : ภาษาเป็น decidable $\iff$ มี nondeterministic TM decide มัน

S15.4 Enumerators

- Enumerator = TM ที่มี **printer** ต่ออยู่ เริ่มด้วย tape ว่าง พิมพ์ string ออกมาเรื่อย ๆ (อาจไม่สิ้นสุด ซ้ำได้ ลำดับใดก็ได้)
- ภาษาที่ E enumerate = เซตของ string ทั้งหมดที่มันพิมพ์ ที่มาของชื่อ recursively enumerable

Theorem 3.21

ภาษาเป็น Turing-recognizable $\iff$ มี enumerator enumerate มัน

- ($\Leftarrow$) M = "On input w :
  1. รัน E ทุกครั้งที่ E พิมพ์ string ให้เทียบกับ w
  2. ถ้า w ปรากฏ *accept*"
- ($\Rightarrow$) ให้ $s_1, s_2, s_3, \ldots$ เป็นรายการของทุก string ใน Σ^*
  E = "ไม่สนใจ input :
  1. ทำซ้ำสำหรับ $i = 1, 2, 3, \ldots$
  2. รัน M i **ขั้น**บนแต่ละ input $s_1, \ldots, s_i$
  3. ถ้าตัวใด accept ให้พิมพ์ s_j นั้น"
- ต้องจำกัดจำนวนขั้น เพราะ M อาจ loop บนบาง s_j

S15.5 Equivalence with other models

- มี model การคำนวณทั่วไปอีกมาก ทุกตัวมีจุดร่วมคือ **เข้าถึงความจำไม่จำกัดได้อย่างไม่มีข้อจำกัด** และทำงานได้จำนวนจำกัดต่อขั้น
- model ที่มีคุณสมบัตินี้ simulate กันได้ทั้งหมด เหมือนภาษาโปรแกรม : อัลกอริทึมที่เขียนได้ในภาษาหนึ่ง เขียนในอีกภาษาได้เสมอ
- ผลสำคัญ : class ของอัลกอริทึมที่บรรยายได้**ไม่ขึ้นกับ model ที่เลือก**

★ Claude สอนเพิ่ม : §3.3 ที่ต้องรู้ก่อนอ่านบทถัดไป (นอกรายการ Suggested reading)

- Church–Turing thesis : แนวคิด "อัลกอริทึม" แบบ intuitive = อัลกอริทึมของ Turing machine
- บรรยาย TM ได้ 3 ระดับ : formal description (state, δ ครบ) implementation description (บอกการเลื่อน head และการเก็บข้อมูล) high-level description (บรรยายอัลกอริทึม)
- input ของ TM เป็น string เสมอ object O เข้ารหัสเป็น $\langle O \rangle$ หลาย object เป็น $\langle O_1, \ldots, O_k \rangle$

บทที่ S16

Decidability and Undecidability

อ้างอิง : Sipser 3rd ed. §4.1–4.2 (pp. 194–209)

S16.1 §4.1 Decidable languages

- แสดงปัญหาเป็นภาษา : "DFA B accept w หรือไม่" เท่ากับ "$\langle B, w\rangle \in A_{\text{DFA}}$ หรือไม่"
- พิสูจน์ decidable : ให้ TM (อัลกอริทึม) ที่**หยุดเสมอ**

Decidable problems concerning regular languages

Theorem 4.1–4.5

- 4.1 $A_{\text{DFA}} = \{\langle B, w\rangle \mid B \text{ เป็น DFA ที่ accept } w\}$: decidable
- 4.2 $A_{\text{NFA}} = \{\langle B, w\rangle \mid B \text{ เป็น NFA ที่ accept } w\}$: decidable
- 4.3 $A_{\text{REX}} = \{\langle R, w\rangle \mid R \text{ เป็น regular expression ที่สร้าง } w\}$: decidable
- 4.4 $E_{\text{DFA}} = \{\langle A\rangle \mid A \text{ เป็น DFA และ } L(A) = \emptyset\}$: decidable
- 4.5 $EQ_{\text{DFA}} = \{\langle A, B\rangle \mid A, B \text{ เป็น DFA และ } L(A) = L(B)\}$: decidable

- 4.1 M = "On input $\langle B, w\rangle$:
  1. simulate B บน w
  2. จบที่ accept state ให้ *accept* ไม่งั้น *reject*" (เก็บ state ปัจจุบันและตำแหน่งใน w บน tape)
- 4.2 N = "On input $\langle B, w\rangle$:
  1. แปลง NFA B เป็น DFA C (Theorem 1.39)
  2. รัน M บน $\langle C, w\rangle$
  3. ตอบตาม M" (ใช้ M เป็น **subroutine**)
- 4.3 P = "On input $\langle R, w\rangle$:
  1. แปลง R เป็น NFA A (Theorem 1.54)
  2. รัน N บน $\langle A, w\rangle$
  3. ตอบตาม N"
- 4.4 T = "On input $\langle A\rangle$:
  1. mark start state
  2. ทำซ้ำจนไม่มี state ใหม่ถูก mark :
  3. mark state ที่มี transition เข้าจาก state ที่ mark แล้ว
  4. ถ้าไม่มี accept state ถูก mark ให้ *accept* ไม่งั้น *reject*"
- 4.5 สร้าง DFA C ที่รับ symmetric difference

$$L(C) = \big(L(A) \cap \overline{L(B)}\big) \cup \big(\overline{L(A)} \cap L(B)\big)$$

$L(C) = \emptyset \iff L(A) = L(B)$ (regular ปิดภายใต้ complement, union, intersection)
F = "On input $\langle A, B\rangle$:
1. สร้าง C
2. รัน T บน $\langle C\rangle$
3. T accept ให้ *accept* ไม่งั้น *reject*"

Decidable problems concerning context-free languages

Theorem 4.7–4.9

- 4.7 $A_{\text{CFG}} = \{\langle G, w\rangle \mid G \text{ เป็น CFG ที่สร้าง } w\}$: decidable
- 4.8 $E_{\text{CFG}} = \{\langle G\rangle \mid G \text{ เป็น CFG และ } L(G) = \emptyset\}$: decidable
- 4.9 ทุก context-free language เป็น decidable

- 4.7 ลองทุก derivation ไม่ได้ (มีไม่จำกัด ได้แค่ recognizer) ใช้ CNF : derivation ของ w ยาว n มี $2n-1$ ขั้น
  S = "On input $\langle G, w\rangle$:
  1. แปลง G เป็น CNF
  2. ไล่ทุก derivation ที่มี $2n-1$ ขั้น ($n = |w|$) ถ้า $n = 0$ ไล่ derivation 1 ขั้น
  3. ถ้ามีตัวใดสร้าง w ให้ *accept* ไม่งั้น *reject*"
- 4.8 ลองทุก w ไม่ได้ ใช้ marking : หาว่า variable ใดสร้าง string ของ terminal ได้
  R = "On input $\langle G\rangle$:
  1. mark terminal ทุกตัว
  2. ทำซ้ำจนไม่มี variable ใหม่ถูก mark :
  3. mark A ถ้ามีกฎ $A \to U_1 \cdots U_k$ ที่ทุก U_i ถูก mark แล้ว
  4. ถ้า start variable ไม่ถูก mark ให้ *accept* ไม่งั้น *reject*"
- $EQ_{\text{CFG}} = \{\langle G, H\rangle \mid L(G) = L(H)\}$: **ไม่ decidable** วิธีของ 4.5 ใช้ไม่ได้ เพราะ CFL ไม่ปิดภายใต้ complement และ intersection
- 4.9 ห้ามแปลง PDA เป็น TM ตรง ๆ (บางกิ่งของ PDA อาจไม่หยุด) ให้ G เป็น CFG ของ A
  M_G = "On input w :
  1. รัน S บน $\langle G, w\rangle$
  2. ตอบตาม S"

Figure 4.10 : ความสัมพันธ์ของ class

regular $\subsetneq$ context-free $\subsetneq$ decidable $\subsetneq$ Turing-recognizable

S16.2 §4.2 Undecidability

Theorem 4.11

$A_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ เป็น TM และ } M \text{ accept } w\}$: **undecidable**

- A_{TM} เป็น Turing-recognizable จึงแสดงว่า recognizer แรงกว่า decider
  U = "On input $\langle M, w\rangle$:
  1. simulate M บน w
  2. M เข้า accept state ให้ *accept* ; เข้า reject state ให้ *reject*"
- U loop ถ้า M loop บน w จึงไม่ใช่ decider U คือ **universal Turing machine** : simulate TM ใดก็ได้จากคำบรรยาย

The diagonalization method

- เทคนิคของ Georg Cantor (1873) สำหรับเทียบขนาดของเซตอนันต์
- Definition 4.12 : $f : A \to B$ เป็น **one-to-one** ถ้า $f(a) \neq f(b)$ เมื่อ $a \neq b$ เป็น **onto** ถ้าทุก $b \in B$ มี a ที่ $f(a) = b$
  A, B **ขนาดเท่ากัน**ถ้ามี f ที่เป็นทั้งสองอย่าง (**correspondence**)
- Example 4.13 : $\mathcal{N}$ กับเลขคู่ $\mathcal{E} = \{2, 4, 6, \dots\}$ ขนาดเท่ากัน $f(n) = 2n$

- Definition 4.14 : A เป็น countable ถ้าจำกัด หรือขนาดเท่ากับ $\mathcal{N}$
- Example 4.15 : $\mathcal{Q} = \{m/n \mid m, n \in \mathcal{N}\}$ เป็น countable วาง m/n ในตารางอนันต์แล้วไล่ตาม**แนวทแยง** ข้ามตัวที่ซ้ำ
- Theorem 4.17 : $\mathcal{R}$ เป็น uncountable
  - สมมติมี correspondence $f : \mathcal{N} \to \mathcal{R}$
  - สร้าง $x \in (0, 1)$ โดยให้ทศนิยมตำแหน่งที่ n ต่างจากตำแหน่งที่ n ของ $f(n)$
  - $x \neq f(n)$ ทุก n
  - ขัดแย้ง
  - (เลี่ยงเลข 0 และ 9)
- Corollary 4.18 : มีภาษาที่ไม่ Turing-recognizable
  - เซตของ TM ทั้งหมด countable : Σ^* countable และแต่ละ TM มี encoding $\langle M \rangle$
  - เซตของภาษาทั้งหมด $\mathcal{L}$ uncountable : เซต $\mathcal{B}$ ของ infinite binary sequence uncountable (diagonalization) และ $\mathcal{L}$ ขนาดเท่า $\mathcal{B}$ ผ่าน characteristic sequence χ_A (บิตที่ i เป็น 1 เมื่อ $s_i \in A$)
  - TM แต่ละตัว recognize ภาษาเดียว ภาษามีมากกว่า TM บางภาษาไม่มี TM recognize

An undecidable language

- **พิสูจน์ Theorem 4.11** : สมมติ H เป็น decider ของ A_{TM}

$$H(\langle M, w \rangle) = \begin{cases} \textit{accept} & M \text{ accept } w \\ \textit{reject} & M \text{ ไม่ accept } w \end{cases}$$

- สร้าง D = "On input $\langle M \rangle$:
  1. รัน H บน $\langle M, \langle M \rangle \rangle$
  2. ตอบ**ตรงข้าม**กับ H"

$$D(\langle M \rangle) = \begin{cases} \textit{accept} & M \text{ ไม่ accept } \langle M \rangle \\ \textit{reject} & M \text{ accept } \langle M \rangle \end{cases}$$

- รัน D บน $\langle D \rangle$: D accept $\langle D \rangle \iff D$ ไม่ accept $\langle D \rangle$ ขัดแย้ง จึงไม่มี D และไม่มี H
- diagonalization อยู่ตรงไหน : ตารางที่แถวเป็น $M_1, M_2, \ldots$ คอลัมน์เป็น $\langle M_1 \rangle, \langle M_2 \rangle, \ldots$ (Figure 4.19–4.21)
  - ช่อง (i, j) คือผลของ $H(\langle M_i, \langle M_j \rangle \rangle)$
  - D ตอบตรงข้ามกับ**ช่องบนแนวทแยง**
  - D ต้องอยู่ในตารางด้วย แต่ช่อง $(D, \langle D \rangle)$ ต้องตรงข้ามกับตัวเอง

A Turing-unrecognizable language

- ภาษาเป็น co-Turing-recognizable ถ้าเป็น complement ของภาษาที่ Turing-recognizable

Theorem 4.22 และ Corollary 4.23

- 4.22 ภาษาเป็น decidable $\iff$ เป็นทั้ง Turing-recognizable และ co-Turing-recognizable
- 4.23 $\overline{A_{\text{TM}}}$ ไม่ Turing-recognizable

- 4.22 ($\Rightarrow$) A decidable : A และ $\overline{A}$ decidable จึง recognizable ทั้งคู่
  ($\Leftarrow$) ให้ M_1 recognize A, M_2 recognize $\overline{A}$
  M = "On input w :
  1. รัน M_1 และ M_2 บน w **ขนานกัน** (สลับทีละขั้น)
  2. M_1 accept ให้ *accept* ; M_2 accept ให้ *reject*"

  ทุก w อยู่ใน A หรือ $\overline{A}$: ตัวใดตัวหนึ่งต้อง accept M จึงหยุดเสมอ
- 4.23 A_{TM} recognizable ถ้า $\overline{A_{\text{TM}}}$ recognizable ด้วย จะได้ A_{TM} decidable ขัดกับ Theorem 4.11